

第 8 章: Lists and Dictionaries (列表与字典)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章深入讲解 Python 的两大主力容器类型——列表 (list) 与字典 (dictionary)。list 是按"位置"组织的有序可变序列, dict 是按"键"组织的可变映射 (mapping), 它们几乎出现在每一段真实 Python 代码中。本章是理解第 4、5 章"对象类型概览"的完整展开, 也是后续学习推导式、迭代、for 语句乃至面向对象的前提。

8.1 开篇引言 (Chapter Opening)

英文原文

Now that we've explored numbers and strings, this chapter moves on to give the full story on Python's list and dictionary objects—collections of other objects, and the main workhorses in almost all Python scripts. As you'll find, both are remarkably flexible: they can be changed in place, can grow and shrink on demand, and may contain and be nested in any other kind of object. By leveraging these built-in object types, you can create and process rich information structures in your scripts without having to define new object type

s of your own.

中文翻译

既然我们已经探索过数字和字符串，本章将继续深入讲解 Python 的列表 (list) 和字典 (dictionary) 对象——它们是"其他对象的集合"，也是几乎所有 Python 脚本中的主力干将。你会发现，两者都极其灵活：它们可以在原地 (in place) 被修改，可以按需增长和收缩，并且可以包含任何其他类型的对象，也能被嵌套在任何其他类型的对象之中。借助这些内置对象类型，你可以在脚本中创建和处理丰富的信息结构，而无需自行定义新的对象类型。

深度理解

·核心概念：本章的主角是两类"容器" (container) 类型。容器区别于数字、字符串的要点是：容器装着"别的对象"。list 与 dict 之所以几乎出现在所有脚本里，是因为真实程序处理的几乎都是"一组数据"或"一组带标签的数据"。

·底层视角："changed in place" (原地修改) 是本章最重要的关键词——它意味着对象本身被修改，而不是产生新对象。这背后是 Python 的引用语义 (reference semantics)：变量名只是指向对象的引用，原地修改会影响所有指向该对象的引用。

·生活例子：list 好比一条按顺序排列的置物架 (每个格子按编号存取物品)；dict 好比一组合上写了标签的收纳盒 (按标签找东西，不用管盒子摆在哪个位置)。“原地修改”就像在架子上直接换掉某格物品，而不是重新买一架新柜

子。

8.2 列表 (Lists)

英文原文

The first stop on this chapter's tour is the Python list. Lists are Python's most flexible ordered collection object type. Unlike strings, lists can contain any sort of object: numbers, strings, and even other lists. Also, unlike strings, lists may be changed in place by assignment to offsets and slices, list method calls, deletion statements, and more—they are mutable objects. Python lists do the work of many of the collection data structures you might have to implement manually in lower-level languages such as C. Here is a quick look at their main properties. Python lists are: Ordered collections of arbitrary objects From a functional view, lists are just places to collect other objects so you can treat them as groups. Lists also maintain a left-to-right positional ordering among the items they contain. Accessed by offset Just as with strings, you can fetch a component object from a list by indexing the list on the object's offset. Because items in lists are ordered by their positions, you can also do tasks such as slicing and concatenation. Variable-length, heterogeneous, and arbitrarily nestable Unlike strings, lists can grow and shrink in place (their lengths can vary), and they

can contain any sort of object, not just one-character strings (they're heterogeneous). Because lists can both contain and be contained by other collection objects, they also support arbitrary nesting; you can create lists of lists of lists, and so on. Of the category "mutable sequence" In terms of our type category qualifiers, lists are mutable (i.e., can be changed in place) and can respond to all the sequence operations used with strings, such as indexing, slicing, and concatenation. In fact, sequence operations work the same on lists as they do on strings; the only difference is that sequence operations such as concatenation and slicing return new lists instead of new strings when applied to lists. Because lists are mutable, however, they also support other operations that strings don't, such as deletion, expansion, and index assignment operations, which change the lists in place. Arrays of object references Technically, Python lists contain zero or more references to other objects. Lists might remind you of arrays of pointers (addresses) if you have a background in some other languages, and fetching an item from a Python list is about as fast as indexing a C array. In fact, lists really are arrays inside the standard CPython interpreter, not linked structures. As we learned in Chapter 6, though, Python always follows a reference to an object whenever the reference is used, so your program deals only with objects. Whenever you assign an object to a data structure component or variable

name, Python always stores a reference to that same object, not a copy of it (though the object stored may be a copy of another, if you requested one before the store).

中文翻译

本章之旅的第一站是 Python 的列表 (list)。列表是 Python 最灵活的有序集合对象类型。与字符串不同，列表可以包含任何种类的对象：数字、字符串，甚至其他列表。同样与字符串不同，列表可以通过对偏移 (offset) 和切片 (slice) 的赋值、列表方法调用、删除语句等方式原地修改——它们是可变 (mutable) 对象。Python 列表承担了你在 C 等低级语言中可能不得不手动实现的许多集合数据结构的工作。这里快速浏览一下它的主要特性。Python 列表是：任意对象的有序集合 (Ordered collections of arbitrary objects) 从功能上看，列表只是收集其他对象的地方，使你可以把它们当作一个整体来处理。列表还维护着其包含项之间从左到右的位置顺序。按偏移访问 (Accessed by offset) 与字符串一样，你可以通过在对象的偏移处对列表进行索引 (indexing) 来取出列表中的某个组件对象。由于列表中的项按位置排序，你还可以进行切片 (slicing) 和拼接 (concatenation) 等操作。可变长度、异构、可任意嵌套 (Variable-length, heterogeneous, and arbitrarily nestable) 与字符串不同，列表可以原地增长和收缩 (它们的长度可以变化)，并且可以包含任何种类的对象，而不只是单字符字符串 (它们是异构的, heterogeneous)。因为列表既可以包含其他集合对象，也可以被其他集合对象包含，所以它们还支持任意嵌套：你可以创建列表的列表的列表

，以此类推。属于"可变序列"类别 (Of the category "mutable sequence") 用我们的类型类别限定词来说，列表是可变的（即可以原地修改），并且能响应所有用于字符串的序列（sequence）操作，比如索引、切片和拼接。事实上，序列操作在列表上和在字符串上的工作方式完全相同；唯一的区别是：当序列操作（如拼接和切片）作用于列表时，返回的是新的列表，而不是新的字符串。但由于列表是可变的，它们还支持字符串所不支持的其他操作，比如删除、扩展和索引赋值等会原地改变列表的操作。对象引用的数组（Arrays of object references）从技术上讲，Python 列表包含零个或多个对其他对象的引用（references）。如果你有使用其他语言的经验，列表可能会让你想起指针（地址）数组，而且从 Python 列表中取出一项的速度几乎和索引 C 数组一样快。事实上，在标准 CPython 解释器内部，列表确实就是数组，而不是链表结构。不过正如我们在第 6 章学到的，每当引用被使用时，Python 总是沿着引用去跟随对象，所以你的程序只与对象打交道。每当你把一个对象赋给某个数据结构组件或变量名时，Python 总是存储对该同一个对象的引用，而不是它的副本（不过，如果你在存储前显式请求了一份拷贝，那么存储的对象也可能是另一份拷贝）。

深度理解

·核心概念：列表有五大身份标签——有序、按偏移访问、可变长度、异构、可变。前四点决定了"能做什么"，最后一点决定了"能怎么改"。与字符串对比记忆：字符串是有序、按偏移访问、不可变（immutable）的序列；列表 = 字符串的全部能力 + 可变能力。

·底层实现：在 CPython 中，list 是 PyListObject，内部持有 PyObject obitem——一个指向对象指针的 C 动态数组（realloc 管理内存）。所以“取第 i 项”的复杂度是 O(1)，和索引 C 数组一样快；list 不是链表，也因此插入头部需要整体搬移元素（memmove）。

·设计原因：为什么“存储的是引用而不是对象本身”？因为 Python 中一切都是对象，如果每次赋值都深拷贝，内存和性能都不可接受。引用共享让 Python 语言极简：赋值 `x = y` 永远只是“贴标签”，这也是后面理解别名（aliasing）陷阱的根源。

·实际问题：列表承担了 C 中“数组 + 链表 + 栈 + 队列”多种结构的职责，让脚本程序员不必手动管理内存、分配大小。这也正是 Python 生产力高的原因之一。

·初学者误区：① 误以为“列表里存的是值”，实际存的是引用——`a = [1]; b = a; b.append(2)` 后 a 也变了；② 误以为列表是链表，其实它是动态数组；③ 误以为 list 里的元素类型必须一致——实际上可以任意混搭（异构）。

Table 8-1 (表 8-1：常用列表字面量与操作)

As a preview and reference, Table 8-1 summarizes common and representative list object operations. It is fairly complete, but for the full story, consult the Python standard-library manual, or run a `help(list)` or `dir(list)` call interactively for a complete list of list methods—you can pass in a real list, or the word list, which is the name of the list data type. The set of methods here is especially prone to change, so be sure to cross-c

heck in the future.

作为预览和参考，表 8-1 总结了常见且有代表性的列表对象操作。它相当完整，但要了解全貌，请查阅 Python 标准库手册，或交互式地运行 `help(list)` 或 `dir(list)` 调用以获取完整的列表方法清单——你可以传入一个真实的列表，也可以直接传入 `list` 这个词（它是列表数据类型的名字）。这里列出的方法尤其容易随版本变化，所以将来务必再核对。

Operation	Interpretation (英文解释)	中文解释
<code>L = []</code>	An empty list	空列表
<code>L = [123,'abc', 1.23, {}]</code>	Four items: indexes 0..3	四项：索引 0..3
<code>L = ['Pat', 40.0, ['dev ','mgr']]</code>	Nested sublists	嵌套子列表
<code>L = list('code')</code>	List of an iterable's items, list of successive integers	由可迭代对象 (iterable) 的项构成的列表、由连续整数构成的列表
<code>L = list(range(-4, 4))</code>	(同上)	(同上)
<code>L[i], L[i][j], L[i:j], len(L)</code>	Index, index of index, slice, length	索引、索引的索引、切片、长度
<code>L1 + L2, L * 3</code>	Concatenate, repeat	拼接、重复
<code>L1 > L2, L1 == L2</code>	Comparisons: magnitude, equality	比较：大小、相等
<code>3 in L, for x in L: print(x)</code>	Membership, iteration	成员测试、迭代
<code>L.append(4)</code>	Methods: growing	方法：增长
<code>L.extend([5, 6, 7]), L.insert(i, X)</code>	(同上)	(同上)
<code>L.index(X), L.count(X)</code>	Methods: searching	方法：搜索
<code>L.sort(), L.reverse(), L.copy(), L.clear()</code>	Methods: sorting, reversing, copying, clearing	方法：排序、反转、复制、清空
<code>L.pop(i), L.remove(X), del L[i], del L[i:j], L[i:j] = []</code>	Methods, statements: shrinking	方法与语句：收缩

<code>L[i] = 3, L[i:j] = [4, 5, 6]</code>	Index assignment, slice assignment	索引赋值、切片赋值
<code>L = [x, 0, y, z]</code>	Iterable unpacking	可迭代对象解包 (unpacking)
<code>L = [x² for x in range(5)]</code>	List comprehensions and maps	列表推导式 (comprehension) 与 map
<code>list(map(ord, 'python'))</code>	(同上)	(同上)

When written down as a literal expression, a list is coded as a series of objects (really, expressions that return objects) in square brackets, separated by commas. For instance, the second row in Table 8-1 assigns the variable `L` to a four-item list. A nested list is coded as a nested square-bracketed series (row 3), and the empty list is just a square-bracket pair with nothing inside (row 1). [1] Many of the operations in Table 8-1 should look familiar, as they are the same sequence operations we put to work on strings earlier—indexing, concatenation, iteration, and so on. Lists also respond to list-specific method calls (which provide utilities such as sorting, reversing, adding items to the end, etc.), as well as in-place change operations (deleting items, assignment to indexes and slices, and so forth). Again, lists have these tools for change operations because they are a mutable object type.

当以字面量 (literal) 表达式书写时，列表被写成方括号中的一系列对象（实际上是返回对象的表达式），彼此用逗号分隔。例如，表 8-1 的第二行把变量 `L` 赋给一个四项列表。嵌套列表被写成一串嵌套的方括号系列（第 3 行），而空列表只是一对中间什么也没有的方括号（第 1 行）。[1]

表 8-1 中的许多操作应该看起来很眼熟，因为它们就是我们之前在字符串上用过的相同序列操作——索引、拼接、迭代等等。列表还响应列表特有的方法调用（这些方法提供排序、反转、在末尾添加项等实用功能），以及原地修改操作（删除项、对索引和切片赋值等）。再次强调：列表之所以拥有这些修改工具，正是因为它是一种可变对象类型。

深度理解

·核心概念：表 8-1 是本章第一张“操作地图”，与第 4 章概览一脉相承。注意操作分三类：序列通用操作（索引、切片、拼接、成员测试、迭代）、列表特有方法（append/extend/sort/reverse 等）、原地修改语句（del、索引/切片赋值）。

·底层实现：字面量 [1, 2, 3] 在解释器执行时，会先为每个元素表达式求值得到对象，然后调用 PyListNew 分配一个连续的内存块（指针数组），把引用按序填入。空列表 [] 只是分配一个长度 0 的数组——注意它仍然会预留少量容量。

·设计原因：dir(list)/help(list) 可以随时查到完整方法清单，这体现了 Python“自文档化”（self-documenting）的传统——不需要背 API。

·实际问题：这张表宣告了一个重要事实：你不再需要像 C 那样手写链表、动态数组、栈——一个 list 全搞定。

·初学者误区：① 看到 L1 > L1 以为笔误，其实原书印刷如此，指“两个列表比较大小”；② 误以为 L[0:2] 修改了原列表——切片是生成新列表的只读操作，只有“切片赋值”

(L[0:2] = ...) 才是修改原列表。

8.3 列表实战 (Lists in Action)

英文原文

Probably the best way to understand lists is to see them at work. Let's once again turn to some simple interpreter interactions to illustrate the operations in Table 8-1.

中文翻译

理解列表的最好方式，大概就是看它实际运行。让我们再次回到简单的解释器交互，来演示表 8-1 中的操作。

深度理解

- 核心概念：本节用 REPL (Read-Eval-Print Loop, 读取-求值-打印循环) 会话教学——这是学习 Python 的黄金方式：每条语句输入后立即看到结果，把"猜"变成"验证"。
- 底层实现：REPL 中每输入一行，解释器就编译成字节码 (bytecode) 并执行，然后调用 repr() 打印结果的字符串形式。因此 >>> [1,2,3] + [4,5,6] 会打印出 [1, 2, 3, 4, 5, 6]。
- 设计原因：交互式探索降低了试错成本，这是 Python 被选为"胶水语言"和教学语言的重要原因。

·实际问题：阅读本书时，务必跟着在终端里敲一遍；看到报错（TypeError、IndexError）不是坏事，报错本身就是最好的老师。

·初学者误区：分不清 REPL 中的 `>>>` 提示符与代码文件——写 `.py` 文件时不需要输入 `>>>`。

8.3.1 基本列表操作（Basic List Operations）

英文原文

Because they are sequences, lists support many of the same operations as strings, which means we don't have to repeat all the operation details again here. In short, though, lists respond to the `+` and operators much like strings—they mean concatenation and repetition here too, except that the result is a new list, not a string:

```
$ python3 # Launch a REPL >> len([1, 2, 3]) # Length 3
>> [1, 2, 3] + [4, 5, 6] # Concatenation [1, 2, 3, 4, 5, 6]
>> ['Py!'] * 4 # Repetition ['Py!','Py!','Py!','Py!']
```

Although the `+` operator works the same for lists and strings, it's important to know that it expects the same sort of sequence on both sides—otherwise, you get a type error when the code runs. For instance, you cannot concatenate a list and a string unless you first convert the list to a string (using tools such as `str` or `formatting`) or convert the string to a list (the `list` built-in function does the trick):

```
>> str([1, 2]) + '34' # Same as '[1, 2]' + '34' '[1, 2]34'>>
[1, 2] + list('34') # Same as [1, 2] + ['3','4'] [1, 2,'3','4']
```

As suggested in prior chapters, lists also support comparisons, which automatically compare all parts from left to right until a result is known. In the following, for instance, a nested 3 is greater than a nested 2, and the one-item list [1] at the end is considered less because it's shorter (though it broadly prefers the term horizontally challenged):

```
>> L = [1, [2, 3], 4] >> (L == [1, [2, 3], 4]), (L > [1, [2,
2], 4]), (L > [1]) (True, True, True)
```

中文翻译

因为列表是序列 (sequence)，它们支持与字符串相同的大量操作，这意味着我们不必在这里重复所有操作细节。不过简而言之，列表对 + 和运算符的响应与字符串非常相似——在这里它们同样表示拼接和重复，只不过结果是新的列表，而不是字符串：\$ python3 # 启动一个 REPL >>> len([1, 2, 3]) # 长度3 >>> [1, 2, 3] + [4, 5, 6] # 拼接[1, 2, 3, 4, 5, 6] >>> ['Py!'] * 4 # 重复['Py!','Py!','Py!','Py!'] 虽然 + 运算符对列表和字符串的工作方式相同，但重要的是要知道：它要求两侧是同一种序列——否则代码运行时会报类型错误。例如，你不能直接拼接列表和字符串，除非先把列表转换成字符串（使用 str 或格式化等工具），或者把字符串转换成列表（list 内置函数就能做到）：>>> str([1, 2]) + '34' # 等价于'[1, 2]' + '34' '[1, 2]34'>>> [1, 2] + li

`st('34') # 等价于 [1, 2] + ['3','4'] [1, 2,'3','4']` 正如前面章节提到的，列表还支持比较（comparisons），它会自动从左到右逐项比较所有部分，直到得出结果为止。例如在下面的代码中，嵌套的 3 大于嵌套的 2，而末尾那个单项列表 [1] 因为更短而被视为更小（尽管它更喜欢用“横向发展受限”这个委婉说法）：
`>>> L = [1, [2, 3], 4] >>> (L == [1, [2, 3], 4]), (L > [1, [2, 2], 4]), (L > [1]) (True, True, True)`

深度理解

·核心概念：`+`（拼接）、`*`（重复）、`==/>`（比较）、`len()`（长度）这些“序列通用操作”在列表上全部可用，但注意它们都产生新列表，不修改原列表——这与字符串一致。

·底层实现：`+` 拼接时，解释器调用 `listconcat`：先按两列表长度之和分配新数组，再两次 `memcpy` 把引用拷贝过去——新建对象，原列表不动。重复同理（`listrepeat`）。比较操作则按“字典序”（lexicographic）逐元素调用各自类型的 `eq/lt`，直到某个位置分出胜负；若两个列表完全相同长度也一样，则相等。

·设计原因：为什么 `+` 不允许“列表 + 字符串”？因为拼接的语义是“同类型序列合并”，隐式转换会掩盖类型错误（`type error`）。Python 哲学：显式优于隐式——你自己决定怎么转换。

·实际问题：反复用 `+` 拼接大量数据会很慢（每次都要复制整个列表），此时应改用 `append` 或 `extend`（第 8.3.4 节）。

·初学者误区：① `L > [1]` 中 `[1, [2, 3], 4] > [1]` 返回 `True`

——先比较第 0 项 ($1 == 1$ 持平)，继续比较时右边的列表"没有下一项"了，于是"较长的列表更大"；② 误解"horizontally challenged"这个玩笑——原文借 "vertically challenged" (矮的委婉语) 调侃短列表，属幽默文字，不影响语义。

代码分析

```
#
len([1, 2, 3])                # 3
[1, 2, 3] + [4, 5, 6]         # [1, 2, 3, 4, 5, 6]
['Py!'] * 4                   # 4

# +
str([1, 2]) + '34'            # '[1, 2]34'
[1, 2] + list('34')           # [1, 2, '3', '4']

#
L = [1, [2, 3], 4]
(L == [1, [2, 3], 4])         # True
(L > [1, [2, 2], 4])          # True 1 3 > 2
(L > [1])                      # True
```

解释器如何处理： $[1, 2, 3] + [4, 5, 6]$ 会被编译为调用字节码 `BINARYOP +`，解释器随即调用 `PyListType` 的 `sqconcat` 槽位 (`listconcat`)，分配大小 $= 3 + 3 = 6$ 的新数组，把两个列表的引用逐项填入；没有对象被复制，只有引用被复制。4 走 `listrepeat`，同样新建列表。 $L == [1, [2, 3], 4]$ 走 `listrichcompare`：先比较长度（相同），再逐元素用 `PyObjectRichCompareBool` 比较——数字 $1 == 1$ ，内层列表 $[2, 3] == [2, 3]$ 递归走同样的比较流程。

·设计思想: "同类型才能 +"这条规则看似麻烦, 实则
程序早期就能暴露数据结构设计错误——比如把"人名列表"
和"数字列表"意外拼在一起。

8.3.2 索引与切片 (Indexing and Slicing)

英文原文

Because lists are sequences, indexing and slicing also work the same way for lists as they do for strings. For lists, though, the result of indexing is whatever type of object lives at the offset you specify (not a one-character string), and slicing a list always returns a new list (not a string):

```
>> L = ['hack','Hack','HACK!'] >> L[2] # Offsets start  
at zero'HACK!'
```

```
>> L[-2] # Negative: count from the right'Hack'>> L[  
1:] # Slicing fetches sections ['Hack','HACK!']
```

New here: because you can nest lists and other object types within lists, you will sometimes need to string together index operations to go deeper into a data structure. For example, one of the simplest ways to represent matrixes (multidimensional arrays) in Python is as lists with nested sublists. Here's a basic 3×3 two-dimensional list-based array—a reprise from Chapter 4:

```
>> matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```


With one index, you get an entire row (really, a nested sublist), and with two, you get an item within the row:

```
>> matrix[1] [4, 5, 6] >> matrix[1][1] 5 >> matrix[2][0] 7
```

As demoed earlier, lists can naturally span multiple lines if you want them to because they are contained by a pair of brackets; watch for more on syntax like this in the next part of the book (and ignore the "..." if absent in your REPL):

```
>> matrix = [[1, 2, 3],  
... [4, 5, 6], ... [7, 8, 9]] >> matrix[1][1] 5
```

For more on matrixes, also watch for a dictionary-based alternative later in this chapter, which can be more efficient when matrixes are largely empty. We'll also continue this thread in Chapter 20 where we'll write additional matrix code, especially with list comprehensions. And for high-powered numeric work, the NumPy extension mentioned in Chapters 1, 4, and 5 provides other ways to handle matrixes.

中文翻译

因为列表是序列，索引和切片对列表的工作方式也与字符串完全相同。只不过对于列表：索引的结果是位于你指定偏移处的任何类型的对象（而不是单字符字符串），切片的返回值则总是一个新列表（而不是字符串）：

```
>>> L = ['hack', 'Hack', 'HACK!'] >>> L[2] # 偏移从零开始'HACK!'
```

```
>>> L
```

`[-2]` # 负数：从右边数'Hack'>>> `L[1:]` # 切片取出区间['Hack','HACK!'] 这里有个新东西：因为可以在列表中嵌套列表和其他对象类型，有时你需要把索引操作串联起来，以深入数据结构更深的层次。例如，在 Python 中表示矩阵（多维数组）的最简单方式之一，就是用嵌套子列表的列表。下面是一个基本的 3×3 二维列表式数组——第 4 章的老面孔：
`>>> matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]` 用一个索引，你得到一整行（实际上是一个嵌套子列表）；用两个索引，你得到行内的一个元素：
`>>> matrix[1]` `[4, 5, 6]`
`>>> matrix[1][1]` `5` `>>> matrix[2][0]` `7` 如前所示，列表可以自然地跨越多行书写——因为它们被一对括号包围着；下一部分书中你会看到更多这类语法（如果你的 REPL 里没有显示 "...", 忽略即可）：
`>>> matrix = [[1, 2, 3], ... [4, 5, 6], ... [7, 8, 9]]` `>>> matrix[1][1]` `5` 关于矩阵还有更多内容：本章稍后会看到一个基于字典的替代方案，当矩阵大部分为空时它可能更高效。我们还会在第 20 章继续这个话题，在那里将编写更多矩阵代码，尤其是使用列表推导式。另外，第 1、4、5 章提到的 NumPy 扩展为高性能数值计算提供了处理矩阵的其他方式。

深度理解

·核心概念：索引 (`L[i]`) 与切片 (`L[i:j]`) 对列表的语义与字符串完全一致，包括负索引。区别只有两点：索引返回"那个对象"（类型不定），切片永远返回新列表。

·底层实现：索引 `L[i]` 走 `listsubscript`：先做边界检查（越界抛 `IndexError`），然后直接取指针数组的第 i 个元素——这就是 $O(1)$ 的原因。`matrix[1][1]` 是两次索引：先取出第 1 行（子列表对象），再对它索引取第 1 项。切片

- `L[1:]` 则分配一个新数组，用 `memcpy` 复制区间内的引用。
- 设计原因：为什么 `L[1:]` 必须返回新列表而不是“视图”（view）？因为列表可变，若切片是视图，修改切片会悄悄修改原列表，极易出 bug。Python 选择了“切片即复制”的安全语义（而 NumPy 出于性能选择了视图语义，这是两种不同权衡）。
- 实际问题：用嵌套列表表达矩阵简单直观，但当矩阵稀疏（大部分位置为空）时浪费严重——本章后面的字典方案用“只存有值的坐标”解决。
- 初学者误区：① `matrix[1][1]` 里第一个 `[1]` 选出的是“行”（子列表），不是“第 1 行第 1 列”——两步走；② 负索引 `L[-2]` 从右数，`-0` 就是 `0`；③ 越界索引报 `IndexError`，但切片越界不报错（返回截断结果）——这是 Python 切片的宽容特性。

代码分析

```
L = ['hack', 'Hack', 'HACK!']
L[2]          # 'HACK!' 2
L[-2]         # 'Hack'-2   len(L)-2 = 1
L[1:]         # ['Hack', 'HACK!']

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
matrix[1]     # [4, 5, 6]" 1 "
matrix[1][1]  # 5 1
matrix[2][0]  # 7 2 0
```

- 解释器如何处理：`matrix[1]` 编译为 `BINARYSUBSCR` 字节码。解释器先用 `listsubscript` 校验索引 `1` 在 `[0, 3)` 内

，然后返回 `obitem[1]`——这个引用指向一个 `PyListObject`（即 `[4, 5, 6]`）。第二次 `[1]` 在同一对象上重复该流程。注意每次索引都是一次“指针解引用 + 边界检查”，嵌套越深开销越大，但每次都是 $O(1)$ 。

·设计思想：多行书写列表时，逗号必须在行尾，这是语法规则——解释器靠“方括号未闭合”判断表达式尚未结束，从而允许换行，因此也不需要加分号或续行符。

8.3.3 原地修改列表 (Changing Lists in Place)

英文原文

Because lists are mutable, they support operations that change a list object in place. That is, list operations in this section and others all modify the list object directly—overwriting its former value—without requiring that you make a new copy, as you had to for strings. Because Python deals only in object references, this distinction between changing an object in place and creating a new object matters; as discussed in Chapter 6, if you change an object in place, you might impact more than one reference to it at the same time.

中文翻译

因为列表是可变的，它们支持原地修改列表对象的操作。也就是说，本节以及其他章节中的列表操作都会直接修改列表对象本身——覆盖它先前的值——而无需像字符串那样制作新副本。由于 Python 只处理对象引用，“原地修改对象”

与"创建新对象"之间的区别至关重要；正如第 6 章讨论过的，如果你原地修改一个对象，可能会同时影响到指向它的多个引用。

深度理解

·核心概念：可变 (mutable) 与不可变 (immutable) 的本质区别不是"语法上允不允许改"，而是"修改是否可能影响其他引用"。字符串是 immutable 的，所以 `s += 'x'` 实际上是创建新字符串再重新绑定变量；列表是 mutable 的，`L.append(x)` 直接修改对象本身。

·底层实现：list 的原地操作直接改写 `PyListObject` 内部的 `obitem` 数组（写入引用、搬移元素、重新分配），对象的地址不变。这也是为什么两个变量指向同一列表时，一处修改处处可见。

·设计原因：可变类型带来性能与表达力——不需要反复复制大数据；代价是需要小心别名 (aliasing) 带来的副作用。Python 让程序员自己管理这个权衡。

·实际问题：函数参数传递也是引用传递：`def f(L): L.append(1)` 会修改调用者的列表。想避免就必须显式复制 (`L[:]`、`list(L)`、`L.copy()`)。

·初学者误区：把"赋值"理解为"复制"——`b = a` 后以为 `b` 是 `a` 的副本，改 `b` 却惊愕地发现 `a` 也变了。这是 Python 新手第一大内存模型误区。

索引与切片赋值 (Index and slice assignments)

英文原文

First up in this category is a twist on the indexing and slicing we've already explored. When using a list, you can change its contents by assigning to either a particular item (offset) or an entire section (slice):

```
>>> L = ['code','Code','CODE!'] >>> L[1] = 'Hack' # Index assignment >>> L # Replaces item 1 ['code','Hack','CODE!']
```

```
>>> L[0:2] = ['write','Python'] # Slice assignment: "delete+insert" >>> L # Replaces items 0,1 ['write','Python','CODE!']
```

Both index and slice assignments are in-place changes—they modify the subject list directly, rather than generating a new list object for the result. Index assignment in Python works much as it does in most other languages: Python replaces the single object reference at the designated offset with a new one; the offset's reference is changed. Slice assignment, the last operation in the preceding example, replaces an entire section of a list in a single step. Because it can be a bit complex, it is perhaps best thought of as a combination of two steps: 1. Deletion: The slice you specify to the left of the `=` is deleted. 2. Insertion: The new items contained in the iterable object to the right of the `=` are inserted into the list on the left, at the place where

e the old slice was deleted. [2] This isn't what really happens, but it can help clarify why the number of items inserted doesn't have to match the number of items deleted. For instance, given a list `L` of two or more items, an assignment `L[1:2]=[4,5]` replaces one item with two—it's as though Python first deletes the one-item slice at `[1:2]` (from offset 1, up to but not including offset 2), then inserts both 4 and 5 where the deleted slice used to be. The net effect makes the list larger. This also explains why the second slice assignment in the following is really an insert—Python replaces an empty slice at `[1:1]` with two items; and why the third is really a deletion—Python deletes the slice (the item at offset 1), and then inserts nothing:

```
>>> L = [1, 2, 3] >>> L[1:2] = [4, 5] # Replacement/insertion
>>> L
[1, 4, 5, 3]

>>> L[1:1] = [6, 7] # Insertion (replace nothing)
>>> L
[1, 6, 7, 4, 5, 3]

>>> L[1:2] = [] # Deletion (insert nothing)
>>> L
[1, 7, 4, 5, 3]
```

In effect, slice assignment replaces an entire section, or "column," all at once—even if the column or its replacement is empty. Because the length of the sequence being assigned does not have to match the length of the slice being assigned to, slice assignment can be used to replace (by overwriting), expand (by inserting), or shrink (by deleting) the subject list. It's a pow

erful operation, but frankly, one that you may not see very often in practice. There are often more straightforward and mnemonic ways to replace, insert, and delete (including concatenation expressions, and the insert, pop, and remove list methods coming up soon), which Python programmers tend to prefer in practice. On the other hand, this operation can be used as a sort of in-place concatenation at the front of the list—per the next section's method coverage, something the list's extend does more mnemonically but at list end:

```
>>> L = [1] >>> L[:0] = [2, 3, 4] # Insert all at :0, an
empty slice at front >>> L [2, 3, 4, 1]

>>> L[len(L):] = [5, 6, 7] # Insert all at len(L):, an empty
slice at end >>> L [2, 3, 4, 1, 5, 6, 7]

>>> L.extend([8, 9, 10]) # Insert all at end, named method
(see also +=) >>> L [2, 3, 4, 1, 5, 6, 7, 8, 9, 10]
```

中文翻译

这一类里第一个要讲的是对已经探索过的索引与切片的一个“变体”。使用列表时，你可以通过给某个特定项（偏移）或整个区间（切片）赋值来改变其内容：>>> L = ['code', 'Code', 'CODE!'] >>> L[1] = 'Hack' # 索引赋值>>> L # 替换第 1 项['code', 'Hack', 'CODE!'] >>> L[0:2] = ['write', 'Python'] # 切片赋值：“删除+插入”>>> L # 替换第 0、1 项['write', 'Python', 'CODE!'] 索引赋值和切片赋值都是原

地修改——它们直接修改目标列表本身，而不是为结果生成新的列表对象。Python 的索引赋值与其他大多数语言的工作方式类似：Python 用一个新的对象引用替换指定偏移处的那一个对象引用；该偏移处的引用被改变。切片赋值（上面示例中的最后一个操作）在一步之内替换列表的整个区间。因为它可能有点复杂，最好把它想成两个步骤的组合：

1. 删除 (Deletion)：= 左侧你指定的切片被删除。
2. 插入 (Insertion)：= 右侧可迭代对象中包含的新项被插入到左侧列表中——插入位置正是旧切片被删除的地方。

[2] 这不是真正的实现过程，但它有助于解释为什么“插入的项数”不必与“删除的项数”相等。例如，给定一个含有两项以上的列表 L，赋值 `L[1:2]=[4,5]` 用两个项替换一个项——就好像 Python 先删除 `[1:2]` 处的一项切片（从偏移 1 起、到但不包括偏移 2），然后把 4 和 5 都插入到被删除切片原来的位置。净效果是让列表变大了。这也解释了为什么下面代码中的第二个切片赋值实际上是插入——Python 用两个项替换了 `[1:1]` 处的空切片；以及为什么第三个实际上是删除——Python 删除该切片（偏移 1 处的项），然后什么也不插入：

```
>>> L = [1, 2, 3]
>>> L[1:2] = [4, 5] # 替换/插入
>>> L
[1, 4, 5, 3]
>>> L[1:1] = [6, 7] # 插入（替换“无”）
>>> L
[1, 6, 7, 4, 5, 3]
>>> L[1:2] = [] # 删除（插入“无”）
>>> L
[1, 7, 4, 5, 3]
```

实际上，切片赋值一次性替换整个区间或“列”——即使该区间或其替换内容为空。因为被赋值的序列长度不必与目标切片长度相等，所以切片赋值可以用来替换（覆盖）、扩展（插入）或收缩（删除）目标列表。这是一个强大的操作，但坦率地说，你在实践中可能不常看到它。通常有更直接、更好记的方式来做替换、插入和删除（包括拼接表达式，以及即将登场的 `insert`、`pop`

、remove 等列表方法)，Python 程序员在实践中往往更喜欢那些方式。另一方面，这个操作还可以用作列表前端的“原地拼接”——按照下一节的方法介绍，列表的 extend 方法做同样的事但更有可读性，只不过它作用于列表末尾：>>> L = [1] >>> L[:0] = [2, 3, 4] # 全部插入到 :0 处——前端的一个空切片>>> L [2, 3, 4, 1] >>> L[len(L):] = [5, 6, 7] # 全部插入到 len(L): 处——末尾的一个空切片>>> L [2, 3, 4, 1, 5, 6, 7] >>> L.extend([8, 9, 10]) # 全部插入到末尾：命名方法（也可用 +=）>>> L [2, 3, 4, 1, 5, 6, 7, 8, 9, 10]

深度理解

- 核心概念：索引赋值 $L[i] = x$ 只能改“已有位置”；切片赋值 $L[i:j] = \text{iterable}$ 是“删除 + 插入”的组合，两侧长度无需匹配——这就是它既能替换又能扩容又能收缩的原因。
- 底层实现：索引赋值走 listassubscript：校验边界后直接把 obitem[i] 换成新引用，并把旧对象引用计数减一（可能触发垃圾回收）。切片赋值走 listassslice：若新元素比旧切片多，需要 realloc 扩容，然后 memmove 把后续元素整体后移腾出空间，再逐个填入新引用；若新元素更少，则把后续元素整体前移。这就是“插入中段是 $O(n)$ ”的物理原因。
- 设计原因：把切片赋值设计为“删除+插入”而非“等长替换”，是为了让一种操作统一三种语义（replace/expand/shrink）——API 最小化，语法最少。
- 实际问题： $L[:0] = \text{items}$ 是“头部批量插入”的合法写法；但日常代码里 $\text{insert}(0, x)$ （头部插一个）和 extend （

尾部批量插) 可读性更好。切片赋值最常用于"原地批量替换" (如 `L[2:5] = newitems`) 。

·初学者误区：① 混淆 `L[1:2] = [4,5]` (原地改列表) 与 `L[1:2]` (只读切片)；② 以为切片赋值要求长度匹配——恰恰相反，这正是它的威力所在；③ 对不存在的索引赋值 `L[5] = x` 会报 `IndexError`，但 `L[5:5] = [x]` 却合法 (那是插入) 。

代码分析

```
L = ['code', 'Code', 'CODE!']
L[1] = 'Hack'           # 1  'Hack'
L                        # ['code', 'Hack', 'CODE!']

L = [1, 2, 3]
L[1:2] = [4, 5]         # 1 ([2]) 2  →
L                        # [1, 4, 5, 3]
L[1:1] = [6, 7]         # 2  →
L                        # [1, 6, 7, 4, 5, 3]
L[1:2] = []             # 1  0  →
L                        # [1, 7, 4, 5, 3]

L = [1]
L[:0] = [2, 3, 4]       # +
L                        # [2, 3, 4, 1]
L[len(L):] = [5, 6, 7]  # extend
L                        # [2, 3, 4, 1, 5, 6, 7]
```

·解释器如何处理：`L[1:2] = [4, 5]` 编译为 `STORESUBSCR` 字节码，左侧切片作为 `PySlice` 对象传入 `listassubscript` (走 `listasslice` 路径)。关键点：解释器会先求值右侧 `[4, 5]` (得到新列表对象)，再执行删除-插入；当左右有重叠 (如 `L[2:5] = L[3:6]`) 时，先取右侧值再删除左侧，

因此仍然正确。插入位置在中间时，memmove 将尾部元素后移，腾出连续空间后逐个填入引用。

·设计思想：“删除+插入”模型让初学者可以只记一条规则：任何 $L[i:j] = \text{anything}$ ，都是“先删 $i..j$ 再插入”。这不完全是内部实现，但作为心智模型完全够用。

列表方法调用 (List method calls)

英文原文

Like strings, Python list objects also support type-specific method calls, most of which change the subject list itself:

```
>>> L = ['write']
>>> L.append('Python') # Append: add one item at the end
>>> L ['write','Python']
>>> L.extend(['code','goodly']) # Extend: add many items at the end
>>> L ['write','Python','code','goodly']
>>> L.sort() # Sort: order list items ('P' < 'c')
>>> L ['Python','code','goodly','write']
```

Methods were introduced in Chapter 7. In brief, they are functions (really, object attributes that reference functions) that are associated with and act upon particular subject objects (the objects through which they are called). Methods provide type-specific tools; the list methods presented here, for instance, are generally available only for lists. Demoed in the preceding code, append is perhaps the most commonly used list method. It simply tacks a single item (really, an object reference) on to the end of the subject list in place, and the list exp

ands to make room for the addition. Unlike concatenation, `append` expects you to pass in one object, not a list, and uses it literally. In fact, the effect of `L.append(X)` is similar to `L+[X]`, but the former changes `L` in place, while the latter makes a new list. [3] By contrast, the `extend` method adds multiple items at the end of the list, again in place. Technically, `extend` always iterates through and adds each item in the passed iterable object, whereas `append` simply adds a single item as is without iterating—a distinction that will be more meaningful in Chapter 14. For now, it's enough to know that `extend` adds many items, and `append` adds one. Also ahead: Chapter 11 covers the `+=` statement, which does in-place assignment for lists too, and is yet another way to add many items that redundantly mirrors `extend`. The `sort` method orders the list's items but merits a section of its own.

中文翻译

与字符串一样，Python 列表对象也支持类型特有的方法调用，其中大多数会修改列表对象本身：

```
>>> L = ['write']
>>> L.append('Python') # 追加 (Append)：在末尾添加一个项
>>> L ['write','Python']
>>> L.extend(['code','goodly']) # 扩展 (Extend)：在末尾添加多个项
>>> L ['write','Python','code','goodly']
>>> L.sort() # 排序 (Sort)：排列列表项 ('P' < 'c')
>>> L ['Python','code','goodly','write']
```

方法 (method) 在第 7 章介绍过。简而言之，方法是与特定目标对象（即调用它的那个对象）相关联

、并作用于该对象的函数（实际上是引用函数的对象属性）。方法提供类型特有的工具；例如这里介绍的列表方法，一般只对列表可用。上面代码演示过的 `append` 大概是最常用的列表方法。它只是把一个单个项（实际上是一个对象引用）原地添加到目标列表末尾，列表会扩展以容纳新增项。与拼接不同，`append` 要求你传入一个对象而不是一个列表，并且原样使用它。事实上，`L.append(X)` 的效果类似于 `L+[X]`，但前者原地修改 `L`，后者则生成一个新列表。[3]相比之下，`extend` 方法在列表末尾添加多个项，同样是原地操作。从技术上讲，`extend` 总是遍历传入的可迭代（iterable）对象并把其中每一项逐个加入，而 `append` 只是不加遍历地按原样添加单个项——这个区别到第 14 章会更有意义。现在，只要知道 `extend` 加多项、`append` 加一项就够了。另外预告一下：第 11 章会讲 `+=` 语句，它对列表也做原地赋值，是又一种添加多项的方式，与 `extend` 功能冗余、互为镜像。`sort` 方法会排列列表的项，但它值得单独一节来讨论。

深度理解

- 核心概念：方法 = “挂在对象身上的函数”。`L.append(X)` 中 `L` 是主体（subject），`append` 是对这个列表进行操作的专用工具。列表方法大多是原地修改且返回 `None`。
- 底层实现：`append` 走 `listappend`：若容量已满则先扩容（`listresize`），再把引用写入 `obitem[obsize]`，`obsize++`。均摊 $O(1)$ 。`extend` 走 `listextend`：它先计算可迭代对象总长，一次性预扩容（避免多次小扩容），再逐个填入。

·设计原因：为什么 append 要"传一个对象"而 extend 要"传可迭代对象"？append([1,2]) 会把 [1,2] 整体当作一项塞进去（嵌套），extend([1,2]) 则把两项展平加入。两种需求都常见，于是提供两个方法而非一个"魔法"方法——显式优于隐式。

·实际问题：文件读出的行列表需要去空白、字典需要按键排序、日志需要收集——append/extend/sort 是日常三件套。

·初学者误区：① L = L.append(4) 会把列表变成 None——append 返回 None，这是本书反复强调的经典陷阱；② 混淆 L.append(x) 与 L.extend([x]) 在"单元素"时等价，但 L.append([1,2]) 与 L.extend([1,2]) 完全不同；③ 用 + 在循环里拼列表，导致 $O(n^2)$ 性能灾难，应该用 append/extend。

代码分析

```
L = ['write']
L.append('Python')           # None
L                             # ['write', 'Python']

L.extend(['code', 'goodly']) #
L                             # ['write', 'Python', 'code', 'goodl...

L.sort()                     # 'P'(80) < 'c'(99) ASCII
L                             # ['Python', 'code', 'goodly', 'writ...

L.append(['x'])              # →
L.extend(['y'])              # extend →
```

·解释器如何处理：L.append('Python') 编译为 LOADME

THOD append + CALL 字节码。解释器先在 L 的类型 (PyListType) 上查找名为 append 的属性——找到函数对象后，把 L 本身作为第一个参数 (self) 传入，再把 'Python' 作为第二个参数。进入 listappend: 若 obsize == allocated (容量已满)，调用 listresize 扩容 (新容量 ≈ 原容量 + 原容量的 1/8 加一个小常数，即约 12.5% 的增幅)，然后把新引用写入数组末尾。函数返回 None——所以 L = L.append(...) 会把 None 赋给 L。

·设计思想："返回 None 的原地方法"与"返回新对象的操作" (如 +、sorted) 的区分，是 Python API 的固定模式：改原地的以结尾的风格 (如 sort/reverse) 绝不返回值，返回新值的 (sorted/reversed) 绝不动原对象。记住这条，能少踩一半的坑。

列表排序 (Sorting lists)

英文原文

As we've just witnessed, sort orders a list in place. It's a common tool that uses Python standard comparison tests (string comparisons in most examples here, but other objects work in sorts too), and by default sorts in ascending order. You can modify sort behavior by passing in keyword arguments—a special name=value syntax in function calls that we've used in earlier chapters, gives values by name, and is often used for configuration options.

In sorts, the reverse argument allows sorts to be made in descending instead of ascending order, and the key argument gives a one-argument function that returns the value to be used in sorting—the string object's standard lower case converter in the following (though its newer casefold may handle some types of Unicode text better):

```
>> L = ['abc','ABD','aBe'] >> L.sort() # Sort with mixed case
>> L ['ABD','aBe','abc'] >> L = ['abc','ABD','aBe']
>> L.sort(key=str.lower) # Normalize to lowercase
>> L ['abc','ABD','aBe'] >> L = ['abc','ABD','aBe']
>> L.sort(key=str.lower, reverse=True) # And change sort order
>> L ['aBe','ABD','abc']
```

The sort key argument can also be useful when sorting lists of dictionaries, to select a sort value by indexing each dictionary on a field along the way. We'll study dictionaries later in this chapter, and you'll learn more about keyword function arguments in Part IV.

Two notes of caution here: first, sorts won't work by default with mixed types. In Python, magnitude comparison of mixed types is an error, as it's generally ambiguous. Because sorting uses these comparisons internally, though, sorting mixed-type lists fails by proxy. To work around this limitation, use the key argument to code value transformations during the sort.

The following simply converts all items to strings with the str built-in, but key can be an arbitrary function

of your making (and is often coded inline with the lambda expression you'll meet later in this book):

```
>> L = [1,'hack', 2] >> L.sort() # Mixed-type sorts fail by default
TypeError:'<' not supported between instances of'str' and'int'
>> L.sort(key=str) >> L # Enable mixed-type sorts: all str
[1, 2,'hack']
```

Second, beware that `append`, `extend`, and `sort` change the associated list object in place, but don't return the modified list as a result (technically, they return the `None` placeholder object introduced in Chapter 4). If you run `L = L.append(X)`, you won't get the modified value of `L`; in fact, you'll lose the reference to the list altogether! When you use methods like these, objects are changed as a side effect, so there's no reason to reassign.

Partly because of such constraints, sorting is also available in Python as the `sorted` built-in function, which sorts any collection (not just lists) and returns a new list for the result (instead of changing the collection in place):

```
>> L = ['abc','ABD','aBe'] >> sorted(L) # Sorting built-in function
['ABD','aBe','abc'] >> L # L is not modified
['abc','ABD','aBe'] >> sorted(L, key=str.lower, reverse=True) # Same arguments as list.sort
['aBe','ABD','abc'] >> L = ['abc','ABD','aBe'] >> sorted([x.lower() for x in L], reverse=True) # Pretransform items: differs!
['abe','abd','abc']
```

Notice the last example here—we can convert to lowercase prior to the sort with a list comprehension (introduced in Chapter 4 and expanded shortly), but the result does not contain the original list's values as it does with the `key` argument. The latter is applied temporarily during the sort, instead of changing the values to be sorted altogether. As we move along, you'll see roles in which the sorted built-in can sometimes be more useful than the `sort` method.

中文翻译

正如我们刚刚看到的，`sort` 会原地排列列表。它是一个常用工具，使用 Python 标准的比较测试（这里大多数例子是字符串比较，但其他对象也能参与排序），默认按升序排序。你可以通过传入关键字参数（keyword arguments）来修改排序行为——关键字参数是函数调用中一种特殊的 `name=value` 语法，我们在前面章节已经用过，它按名称传值，常用于配置选项。在排序中，`reverse` 参数允许按降序而非升序排序；`key` 参数接收一个单参数函数，该函数返回用于排序的值——下面的例子用的是字符串对象标准的 `lower` 转小写转换器（不过它较新的 `casefold` 方法在处理某些类型的 Unicode 文本时可能更好）：

```
>>> L = ['abc','ABD','aBe']
>>> L.sort() # 混合大小写排序
>>> L
['ABD','aBe','abc']
>>> L = ['abc','ABD','aBe']
>>> L.sort(key=str.lower) # 先归一化为小写再排序
>>> L
['abc','ABD','aBe']
>>> L = ['abc','ABD','aBe']
>>> L.sort(key=str.lower, reverse=True) # 顺便改变排序方向
>>> L
['aBe','ABD','abc']
```

排序的 `key` 参数在排序“字典列表”时也很有用——

—可以沿路用字段索引每个字典来选取排序值。我们稍后会本章学习字典，第 IV 部分还会学到更多关于关键字函数参数的内容。这里有两个注意事项：第一，默认情况下混合类型的排序行不通。在 Python 中，混合类型的“大小比较”是错误，因为这种比较通常是含混不清的。而排序内部要用到这些比较，所以混合类型列表的排序会连带失败。要绕过这个限制，可以用 key 参数在排序期间编码值变换。下面只是用 str 内置函数把所有项转成字符串，但 key 可以是任意你自定义的函数（而且常常用本书后面会讲的 lambda 表达式内联编写）：

```
>>> L = [1,'hack', 2] >>> L.sort()
# 混合类型排序默认失败TypeError:'<' not supported between instances of 'str' and 'int' >>> L.sort(key=str) >>> L
# 启用混合类型排序：全转成 str [1, 2,'hack']
```

第二，小心 append、extend 和 sort 会原地修改关联的列表对象，但不会把修改后的列表作为结果返回（严格说是返回第 4 章介绍的 None 占位对象）。如果你写 L = L.append(X)，你得到的不是修改后的 L 的值；事实上，你会把列表的引用整个弄丢！使用这类方法时，对象是通过副作用（side effect）被修改的，所以没有必要重新赋值。部分出于这类限制，Python 还提供了 sorted 内置函数来排序，它可以对任何集合排序（不只是列表），并返回一个新的列表作为结果（而不是原地修改集合）：

```
>>> L = ['abc','ABD','aBe'] >>> sorted(L) # 排序的内置函数['ABD','aBe','abc'] >>> L
# L 没有被修改['abc','ABD','aBe'] >>> sorted(L, key=str.lower, reverse=True) # 参数与 list.sort 相同['aBe','ABD','abc'] >>> L = ['abc','ABD','aBe'] >>> sorted([x.lower() for x in L], reverse=True) # 先变换项再排序：结果不同！['abe','abd','abc']
```

注意最后一个例子——我们

可以先用列表推导式在排序前转成小写（第 4 章引入，稍后会展开），但结果并不像 `key` 参数那样包含原始列表的值。`key` 是在排序过程中临时应用的，而不是把要排序的值整体改掉。随着学习的深入，你会看到 `sorted` 内置函数在某些场景下比 `sort` 方法更有用。

深度理解

- 核心概念：`sort`（原地排序，返回 `None`）与 `sorted`（返回新列表，不动原数据）是一对孪生工具；`key` 参数是“变换比较值但不改变数据”的机制。

- 底层实现：CPython 的 `list.sort` 使用 Timsort 算法（由 Tim Peters 设计，Python 之父 Guido 的同事，也以“时间之禅”闻名）。Timsort 是归并排序与插入排序的混合：对已大致有序的数据（run）表现接近 $O(n)$ ，最坏 $O(n \log n)$ ，且稳定（相等元素保持原相对顺序）。`key` 参数底层通过“装饰-排序-取消装饰”（`decorate-sort-undecorate`，即 DSU）技巧实现：为每个元素计算一次 `key` 存入临时结构，避免比较时反复调用 `key` 函数。

- 设计原因：为什么 `sorted` 与 `sort` 都要存在？因为“修改原数据”和“保留原数据”是两种同样常见的需求，Python 选择把选择权交给调用者，并保持命名规律（`sort/reverse` 原地、`sorted/reversed` 返回新值）。`key` 参数的设计则避免了 DSU 手动技巧——这正是内置机制替代手写模式（pattern）的范例。

- 实际问题：稳定排序是真实世界刚需——比如先按姓氏排、再按年龄排，第二次排序不会破坏第一次的结果。`rever`

`se=True` 是"降序"的推荐写法，而不是 `sort()` 后 `reverse()` (多一次 $O(n)$ 遍历)。

·初学者误区：① `L.sort()` 后再 `print(L.sort())` 看到 `None` 就以为排序失败；② 把 `sorted` 用于原地需求，忘了赋值给变量导致排序结果丢失；③ 以为 `L.sort(reverse=True)` 之后列表顺序是"先 `sort` 再 `reverse`"——不，是真正的降序；④ 忘记字符串排序按 Unicode 码点（ASCII 大写字母在小写字母之前），所以 `'ABD' < 'abc'`。

代码分析

```
L = ['abc', 'ABD', 'aBe']
L.sort()                                #
L                                      # ['ABD', 'aBe', 'abc']

L.sort(key=str.lower)                  # key
L                                      # ['abc', 'ABD', 'aBe']

L.sort(key=str.lower, reverse=True)
L                                      # ['aBe', 'ABD', 'abc']

L = [1, 'hack', 2]
L.sort()                               # TypeError: int str
L.sort(key=str)                        # str()
L                                      # [1, 2, 'hack'] '1'<'2'<'hack'

sorted(L)                              # L
sorted(L, key=str.lower, reverse=True)
sorted([x.lower() for x in L], reverse=True) #
```

·解释器如何处理：`L.sort(key=str.lower, reverse=True)` 执行时：① 解释器把 `str.lower` (一个"未绑定方法"对象，本身是可调对象) 保存为 `key`；② Timsort 排序开始

前，为每个元素调用一次 key，得到装饰后的比较值序列（这是 DSU 技巧，保证 key 函数每个元素只调用一次， $O(n)$ 次而非 $O(n \log n)$ 次）；③ 用这些临时值做比较完成排序；④ 排序结束后丢弃临时值，列表里仍是原始对象。sorted 则先对传入的任意可迭代对象构建一个新列表，再走同样的流程，返回新列表。

·设计思想：key 与“先变换再排序”的区别是临时 vs 永久——key 只在比较瞬间生效。这个“临时变换”思想也是后面函数式编程（map/filter）的原型。

8.3.4 更多列表方法 (More List Methods)

英文原文

Like strings, lists have other methods that perform other specialized operations. For instance, reverse reverses the list in place, and pop deletes one item at the end (by default). Just like sorting, there is also a reversed built-in function that does not change the list in place.

Confusingly, though, reversed does not return a new list like sorted; it instead returns an iterable object that produces results on demand, and must be wrapped in a list call to collect its results if a real list is needed (e.g., for indexing, or display at the REPL here):

```
>> L = [1, 2, 3, 4, 5] >> L [1, 2, 3, 4, 5] >> L.pop() #  
Delete and return an item: @ -1 by default 5 >> L [1,
```

```
2, 3, 4] >> L.reverse() # In-place reversal >> L [4, 3, 2, 1] >> list(reversed(L)) # Reversal built-in with a result (iterable) [1, 2, 3, 4] >> L # L was unchanged [4, 3, 2, 1]
```

In some types of programs, the list pop method is used in conjunction with append to implement a quick last-in-first-out (LIFO) stack structure. The end of the list serves as the "top" of the stack:

```
>> L = [] >> L.append(1) # Push onto stack >> L.append(2) >> L [1, 2] >> L.pop() # Pop off stack 2 >> L [1]
```

The pop method also accepts an optional offset of the item to be deleted and returned (the default is the last item at offset -1). Other list methods remove an item by value (remove), insert an item at an offset (insert), count the number of occurrences (count), and search for an item's offset (index—a search for the index of an item, not to be confused with indexing itself, despite the name!):

```
>> L = ['hack','Py','code'] >> L.index('Py') # Index of an object (search/find) 1 >> L.insert(1,'more') # Insert at offset/position >> L ['hack','more','Py','code'] >> L.remove('code') # Delete by value >> L ['hack','more','Py'] >> L.pop(1) # Delete by offset/position 'more' >> L ['hack','Py'] >> L.count('Py') # Number of occurrences 1
```


Note that unlike other list methods, `count` and `index` do not change the list itself, but return information about its content. Run a `help(list)` or `dir(list)` in your REPL, see Python reference resources, or experiment with these calls interactively on your own to learn more about list methods.

中文翻译

与字符串一样，列表还有其他执行专门操作的方法。例如，`reverse` 原地反转列表，`pop`（默认）删除末尾的一个项并返回它。和排序类似，也有一个不会原地修改列表的 `reversed` 内置函数。不过让人困惑的是，`reversed` 不像 `sorted` 那样返回一个新列表；它返回的是一个按需产生结果的迭代对象，如果确实需要真实列表（比如要索引，或要在 REPL 中显示），必须用 `list` 调用把它包装起来收集结果：

```
>>> L = [1, 2, 3, 4, 5] >>> L [1, 2, 3, 4, 5] >>> L.pop() # 删除并返回一项：默认是 -1 处 5 >>> L [1, 2, 3, 4] >>> L.reverse() # 原地反转 >>> L [4, 3, 2, 1] >>> list(reversed(L)) # 反转内置函数（返回可迭代对象） [1, 2, 3, 4] >>> L # L 未被修改 [4, 3, 2, 1]
```

在某些类型的程序中，列表的 `pop` 方法与 `append` 配合使用，可以实现一个快速的后进先出（last-in-first-out, LIFO）栈（stack）结构。列表的末尾充当栈的“栈顶”：

```
>>> L = [] >>> L.append(1) # 压栈（Push） >>> L.append(2) >>> L [1, 2] >>> L.pop() # 出栈（Pop） 2 >>> L [1]
```

`pop` 方法还接受一个可选的偏移参数，指定要删除并返回的项的位置（默认是偏移 -1 处的最后一项）。其他列表方法包括：按值删除项（`remove`）、在偏移处插入项（`insert`）、统计出现次数（`count`）

、搜索某项的偏移 (index——这是"某项的索引"的搜索/查找, 尽管名字叫 index, 可别和索引操作本身搞混!) : >
>> L = ['hack','Py','code'] >>> L.index('Py') # 某个对象的索引 (搜索/查找) 1 >>> L.insert(1,'more') # 在偏移/位置处插入 >>> L ['hack','more','Py','code'] >>> L.remove('code') # 按值删除 >>> L ['hack','more','Py'] >>> L.pop(1) # 按偏移/位置删除'more' >>> L ['hack','Py'] >>> L.count('Py') # 出现次数1 注意, 与其他列表方法不同, count 和 index 并不修改列表本身, 而是返回关于其内容的信息。在你的 REPL 中运行 help(list) 或 dir(list), 查阅 Python 参考资源, 或自己交互式地实验这些调用, 来了解更多列表方法。

深度理解

·核心概念: 列表方法按行为分四类: 增长 (append/extend/insert)、删除 (pop/remove/clear)、重排 (sort/reverse)、查询 (index/count, 不动数据)。查询类返回信息, 其余返回 None。

·底层实现: pop() 走 listpop: 取出 obitem[obsize-1], obsize--, 返回该引用; 从中间 pop 则需 memmove 前移后续元素 (O(n))。insert(i, x) 走 listinsert: 先扩容 (如需), 再 memmove 把 i 及之后的元素后移一格, 空出位置写入新引用。remove(x) 是线性查找 (从前往后逐个 PyObjectRichCompareBool 比较相等), 找到后删除; 找不到抛 ValueError。index 同样是 O(n) 线性查找。

·设计原因: 为什么默认 pop() 是弹出末尾? 因为列表作为

数组，尾部操作是 $O(1)$ 、头部操作是 $O(n)$ ——把"默认最快"设为默认行为，引导用户写高效代码。这也让"列表当栈用"顺理成章。

·实际问题：栈 (stack) 是最常用的数据结构之一——函数调用栈、表达式求值、撤销 (undo) 历史都是 LIFO。Python 里 `list.append + list.pop` 就是零依赖的栈实现。若需要 FIFO 队列，用 `collections.deque` (两端都是 $O(1)$) 。

·初学者误区：① 以为 `remove` 删除所有匹配项——它只删第一个匹配；② 以为 `index` 返回后列表会变——它只是查找；③ `pop()` 与 `pop(0)` 性能差异巨大 (尾部 $O(1)$ ，头部 $O(n)$)；④ 对空列表 `pop()` 会抛 `IndexError: pop from empty list`。

代码分析

```
L = [1, 2, 3, 4, 5]
L.pop()           # 50(1)
L                 # [1, 2, 3, 4]
L.reverse()       # None
L                 # [4, 3, 2, 1]
list(reversed(L)) # [1, 2, 3, 4]reversed list()

# Stackpush = appendpop = pop
L = []
L.append(1)       #
L.append(2)       #
L.pop()           # 2

L = ['hack', 'Py', 'code']
L.index('Py')     # 1 'Py'
L.insert(1, 'more') #
```

```
L                # ['hack', 'more', 'Py', 'code']
L.remove('code')  # 'code'
L.pop(1)         # 'more'
L.count('Py')    # 1
```

·解释器如何处理：L.insert(1,'more') 时，解释器调用 list insert：先确保容量（必要时 listsize 扩容），随后用 memmove(obitem+2, obitem+1, (n-1)sizeof(ptr)) 把索引 1 及之后的引用整体后移一格，再把'more'的引用写入 obitem[1]——中段插入是 O(n) 的物理原因就在这一行 memmove。L.pop(1) 则反向操作：取出引用后，memmove 把后续元素前移覆盖空位。L.remove('code') 则进入循环：对每个元素调用相等比较，找到第一个相等的就执行同样的删除逻辑。

·设计思想：列表 = 栈 (Stack) + 动态数组，这是 Python 语法层提供的"瑞士军刀"。但记住它的 O(n) 操作都在中段/头部，编写大数据量代码时要按此选择结构 (deque、heapq、sortedcontainers 等)。

8.3.5 迭代、推导式与解包 (Iteration, Comprehensions, and Unpacking)

英文原文

Lists also respond to other sequence operations we used on strings in the prior chapter, including iteration tools, and are regularly used in conjunction with the range built-in previewed in Chapter 4:

```
>> 3 in [1, 2, 3] # Membership True >> for x in [1, 2, 3]:
```

```
... print(x, end='') # Iteration ... 1 2 3 >> list(range(5))  
# Counter generators (0...N-1) [0, 1, 2, 3, 4]
```

We will talk more formally about for iteration and the range built-in in Chapter 13, because they are related to statement syntax. Per Chapter 4 previews, though, for loops step through items in any sequence (or other iterable) from left to right, executing one or more statements for each item; and range makes a series of integers, for use in a variety of roles, including counter loops (and coerced by list to surrender its values for display here).

List comprehensions and maps The last items in Table 8-1, list comprehensions and map calls, were also introduced in Chapter 4 and are covered in full in Chapters 14 and 20. Their basic usage is straightforward, though—list comprehensions are close kin to for loops, and build a new list by applying an expression to each item in a sequence (really, any iterable):

```
>> res = [x 4 for x in 'code'] # List comprehension >  
> res ['cccc','oooo','dddd','eeee']
```

As a preview, this expression is equivalent to a for loop that builds up a list of results manually with the append we met earlier, but as you'll learn in later chapters, list comprehensions are simpler to code and may run faster:

```
>> res = [] # Equivalent loop code
>> for x in 'code':
# Append expression results
... res.append(x 4) ...
>> res ['cccc','oooo','dddd','eeee']
```

Comprehension also support extensions like if filters and nested for loops, and the map built-in function does similar work, but applies a function instead of an expression to items in a sequence (or other iterable) and collects all the results in a new list—if we wrap it in list to force it to produce all its results:

```
>> [x 4 for x in 'program' if x >='p'] # Filter items with
if clauses ['pppp','rrrr','rrrr']
>> [x + y for x in 'py' for y in '312'] # Nested loops: see Chapter 14!
['p3','p1','p2','y3','y1','y2']
>> list(map(abs, [-1, -2, 0, 1, 2])) # Map a function across a
sequence [1, 2, 0, 1, 2]
```

While these are powerful tools, their loop equivalents also make them optional. Stay tuned for the related dictionary comprehension later in this chapter, and much more on comprehensions and maps later in this book.

List-literal unpacking As of Python 3.5, list literals also support a syntax that unpacks the contents of any iterable (including sequences like lists and strings) at the top level of the list being created. The effect flattens the starred item in the new list:

```
>> L = ['code','hack']
>> [L, 2, 3, L] # Normal assign
```

```

ments nest items [['code','hack'], 2, 3, ['code','hack']]
>> [L, 2, 3, L] # Iterable unpacking in list literal ['code',
',hack', 2, 3,'code','hack'] >> S='code' # Any sequence (or other iterable)
>> ['Py', S, range(3)] ['P','y','c','o','d','e', 0, 1, 2]

```

Importantly, and as is so often true in Python today, unpacking is never required. The following simple concatenation, for example, has the same effect as the preceding's first unpacking—which is redundant in this case:

```

>> L + [2, 3] + L # What unpacking does, for lists ['code','hack', 2, 3,'code','hack']

```

For nonlist iterables, unpacking without requires conversions because `+` expects lists on both sides, but it's not much extra work, especially given the rarity of such code:

```

>> list('Py') + list(S) + list(range(3)) # What unpacking does, for nonlists ['P','y','c','o','d','e', 0, 1, 2]

```

Loops can achieve the same goals as unpacking too; they may be more verbose, but are also more general purpose:

```

>> M = [] >> for x in ('Py', S, range(3)): M.extend(x)
# Loops v special-case hacks
... >> M ['P','y','c','o','d','e', 0, 1, 2]

```

Notice two things about this example. First, it codes a tuple of three items as the sequence that the `for` loop

op steps through (the tuple's outer parentheses are optional but explicit). Second, the for loop in this code avoids a line by moving its body up to its header, using a syntax rule we'll define formally in the next part of this book (tl;dr: this works only for simple statements, like calls to print and extend).

Because we're not quite ready for the full iteration story, we'll postpone further details for now, but watch for similar unpacking expressions for dictionaries later in this chapter. Here, though, you should already be able to weigh for yourself whether the alternatives to unpacking were sufficiently subpar to warrant the convolution.

中文翻译

列表还响应我们在上一章用于字符串的其他序列操作，包括迭代 (iteration) 工具，并且经常与第 4 章预览过的 range 内置函数配合使用：

```
>>> 3 in [1, 2, 3] # 成员测试 True
>>> for x in [1, 2, 3]: ... print(x, end=" ") # 迭代... 1 2 3
>>> list(range(5)) # 计数器生成器 (0...N-1) [0, 1, 2, 3, 4]
```

我们会在第 13 章更正式地讨论 for 迭代和 range 内置函数，因为它们与语句语法相关。不过按第 4 章的预览：for 循环从左到右遍历任何序列（或其他可迭代对象）中的项，对每一项执行一条或多条语句；range 生成一系列整数，可用于多种角色，包括计数器循环（这里用 list 强制它交出全部值以便显示）。列表推导式与 map 表 8-1 中最后几项——列表推导式 (list comprehension) 和 map 调用——同样在第 4 章介绍过，并在第 14、20 章完整展开。

不过它们的基本用法很直接——列表推导式是 for 循环的近亲，通过对序列（实际上是任何可迭代对象）中的每一项应用一个表达式来构建新列表：

```
>>> res = [x 4 for x in 'code'] # 列表推导式
>>> res ['cccc','oooo','dddd','eeee']
```

作为预览，这个表达式等价于一个用前面学过的 append 手动构建结果列表的 for 循环，但正如你将在后面章节学到的，列表推导式写起来更简洁，运行也可能更快：

```
>>> res = [] # 等价的循环代码
>>> for x in 'code': # 追加表达式结果...
...     res.append(x 4) ...
>>> res ['cccc','oooo','dddd','eeee']
```

推导式还支持扩展语法，比如 if 过滤和嵌套 for 循环；map 内置函数做类似的工作，但它是对序列（或其他可迭代对象）中的项应用一个函数而不是表达式，并把所有结果收集到一个新列表中——前提是我们用 list 包装它，强制它产出全部结果：

```
>>> [x 4 for x in 'program' if x >='p'] # 用 if 子句过滤项
['pppp','rrrr','rrrr']
>>> [x + y for x in 'py' for y in '312'] # 嵌套循环：详见第 14 章!
['p3','p1','p2','y3','y1','y2']
>>> list(map(abs, [-1, -2, 0, 1, 2])) # 把一个函数映射到序列上
[1, 2, 0, 1, 2]
```

虽然这些都是强大的工具，但对应的循环版本也让它们变得“可有可无”。敬请期待本章后面的字典推导式，以及本书后面关于推导式和 map 的大量内容。列表字面量解包从 Python 3.5 起，列表字面量还支持一种语法，可以在正在创建的列表的顶层解包（unpack）任何可迭代对象（包括列表、字符串等序列）的内容。其效果是让带星号的项在新列表中“摊平”：

```
>>> L = ['code','hack']
>>> [L, 2, 3, L] # 普通赋值会嵌套项
[['code','hack'], 2, 3, ['code','hack']]
>>> [L, 2, 3, L] # 列表字面量中的可迭代解包
['code','hack', 2, 3,'code','hack']
>>> S = 'code' # 任何序列（或其他可迭代对象）>
```

>> ['Py', S, range(3)] ['P','y','c','o','d','e', 0, 1, 2] 重要的是（在今天 Python 里经常如此），解包从来不是必需的。例如，下面简单的拼接与前面第一个解包效果相同——在这种情况下它其实是冗余的：>>> L + [2, 3] + L # 解包对列表做的事['code','hack', 2, 3,'code','hack'] 对于非列表的可迭代对象，不用就需要转换，因为 + 要求两侧都是列表；但额外工作量并不大，尤其考虑到这类代码本来就少见：>>> list('Py') + list(S) + list(range(3)) # 解包对非列表做的事['P','y','c','o','d','e', 0, 1, 2] 循环也能达到与解包相同的目标；它可能更冗长，但也更通用：>>> M = []>>> for x in ('Py', S, range(3)): M.extend(x) # 循环 vs 特例技巧...>>> M ['P','y','c','o','d','e', 0, 1, 2] 注意这个例子的两件事。第一，它写了一个三项的元组（tuple）作为 for 循环遍历的序列（元组外层括号可省略但这里显式写出了）。第二，这个 for 循环把循环体移到头部从而省掉一行，用到的语法规则我们将在本书下一部分正式定义（长话短说：这种写法只对简单语句有效，比如调用 print 和 extend）。因为我们还没有准备好讲完整的迭代故事，这里先推迟更多细节，但请注意本章稍后字典的解包表达式与之类似。不过到这里，你已经可以自己掂量：解包的替代方案是否差到值得引入这么绕的语法。

深度理解

·核心概念：本节是“列表的三种批量处理风格”——成员测试/迭代（in/for）、推导式（comprehension，表达式驱动）、解包（，把可迭代对象摊进字面量）。三者都建立在“可迭代协议”之上。

·底层实现：3 in [1,2,3] 走 listcontains：线性搜索，逐个比较相等，O(n)。for 循环走迭代协议：调用 iter 获得迭代器，循环调用 next 直到抛出 StopIteration。推导式在 CPython 中有专门的字节码 (LISTAPPEND 等)，比等价 for 循环少几层查找，所以通常更快。map 返回惰性迭代器 (lazy iterator)，必须 list() 才产出全部结果。

·设计原因：为什么 3.5 要引入解包？因为它消灭了一整类"拼接 + 转换"的样板代码 (list(a) + b + list(c))，让混合任意可迭代对象变得自然。而 Python 一贯哲学是：语法糖是可选优化，不是必需。

·实际问题：推导式是 Python 程序员使用频率最高的写法之一——"对每个 x 做 f(x)"的意图用一行表达。if 过滤 (filter)、嵌套 for (笛卡尔积) 都是高频模式。解包在函数调用 (f(args)) 和合并多个列表 ([a, b]) 时最实用。

·初学者误区：① 在推导式中试图"修改原列表"——推导式永远生成新列表；② 嵌套推导式的阅读顺序是"从左到右" (外层 for 在前，先执行)，容易与直觉相反；③ 以为 [x for x in lst if x] 的 if 能"短路"——它只是过滤；④ 在循环里用 M.extend(x) 时忘了 x 必须是可迭代对象，传数字会 TypeError。

代码分析

```

3 in [1, 2, 3]                # True
for x in [1, 2, 3]:
    print(x, end=' ')        # 1 2 3

res = [x * 4 for x in 'code'] # 'c','o','d','e' 4
res                                     # ['cccc', 'oooo', 'dddd', 'eeee...

res = []
for x in 'code':              #
    res.append(x * 4)

[x * 4 for x in 'program' if x >= 'p'] # >= 'p'
[x + y for x in 'py' for y in '312']   #
list(map(abs, [-1, -2, 0, 1, 2]))      # map

L = ['code', 'hack']
[*L, 2, 3, *L]                      # ['code', 'hack', 2, 3, 'code', 'hac...
['*Py', *'code', *range(3)]         # range

```

·解释器如何处理：推导式 `[x 4 for x in 'code']` 被编译成独立的"推导式代码对象"：创建空列表 → 创建迭代器 → FORITER 循环：取下一个 `x`，若不等于哨兵（`StopIteration`）则计算 `x 4`，用 `LISTAPPEND` 追加 → 循环结束返回列表。由于迭代和追加都在 C 层完成、且无需维护循环变量的名字空间查找，它通常比手动 `for + append` 快。`[L, 2, 3, L]` 则被编译为：创建列表，`LISTEXTEND` 把 `L` 的项批量加入（相当于 `extend`），再 `LISTAPPEND` 加 `2、3`，再 `LISTEXTEND` 加 `L`。

·设计思想：解包 = "顶层展平"，（字典解包）稍后登场 = "按键合并"。这两类语法让"组合数据结构"成为一等公民操作，是函数式风格在 Python 语法层的体现。

8.3.6 其他列表操作 (Other List Operations)

英文原文

Because lists are mutable, you can also use the `del` statement to delete an item or section in place:

```
>> L = ['hack','more','Py','code'] >> del L[0] # Delete
one item (in place) >> L ['more','Py','code'] >> del L[
1:] # Delete an entire section >> L ['more']
```

As covered earlier, because slice assignment is a deletion plus an insertion, you can also delete a section of a list by assigning an empty list to a slice (`L[i:j]=[]`); Python deletes the slice named on the left, and then inserts nothing. Assigning an empty list to an index, on the other hand, just stores a reference to the empty list object in the specified slot, rather than deleting an item:

```
>> L = ['hack','more','Py','code'] >> L[0:1] = [] # Delete via slice assignment
>> L ['more','Py','code'] >> L[1:] = [] # Delete all but first
>> L ['more'] >> L[0] = [] # Replace item, don't delete!
>> L [[]]
```

Although all the operations just discussed are typical, there may be additional list methods and operations not illustrated here. The method toolbox may also change over time, and in fact has: the newer `L.copy()` method makes a top-level copy of the list, much like `L[:]` and `list(L)`, but is symmetric with copy methods in

sets and dictionaries. For a comprehensive and up-to-date list of type tools, you should always consult Python's manuals.

And because it's such a common hurdle, this book is compelled to remind you once again that all the in-place change operations discussed here work only for mutable objects: they won't work on strings (or tuples, coming up in Chapter 9), no matter how hard you try. Mutability is an inherent yes/no property of each object type—including the subject of this chapter's next section.

中文翻译

因为列表是可变的，你还可以用 `del` 语句原地删除一个项或一个区间：
`>>> L = ['hack','more','Py','code'] >>> del L[0] # 删除一个项（原地）>>> L ['more','Py','code'] >>> del L[1:] # 删除整个区间>>> L ['more']` 正如前面讲过的，因为切片赋值是“删除 + 插入”，你也可以通过把一个空列表赋给某个切片来删除列表的一段 (`L[i:j]=[]`)；Python 删除左边指定的切片，然后什么也不插入。另一方面，把空列表赋给一个索引，只是在指定槽位存入对空列表对象的引用，而不是删除一个项：
`>>> L = ['hack','more','Py','code'] >>> L[0:1] = [] # 通过切片赋值删除>>> L ['more','Py','code'] >>> L[1:] = [] # 删除除第一个外的所有项>>> L ['more'] >>> L[0] = [] # 替换项，不是删除！>>> L [[]]` 虽然上面讨论的所有操作都很典型，但可能还有这里没有演示的列表方法和操作。方法工具箱也会随时间变化，事实上确实如此：较新的 `L.copy()` 方法制作列表的顶

层副本，非常类似于 `L[:]` 和 `list(L)`，但与集合和字典中的 `copy` 方法保持对称。要获得全面而最新的类型工具清单，你应当始终查阅 Python 手册。而且因为它是一个常见的绊脚石，本书不得不再次提醒你：这里讨论的所有原地修改操作只对可变对象有效：它们对字符串（或第 9 章即将登场的元组 tuple）不起作用，无论你多努力地尝试。可变性是每个对象类型固有的"是/否"属性——包括本章下一节的主角

。

深度理解

·核心概念：删除手段的家族谱：`del L[i]`（删一项）、`del L[i:j]`（删一段）、`L[i:j] = []`（用切片赋值删）、`L.pop()/L.remove()`（方法删除）。`L[i] = []` 是"替换"，不是"删除"——它把空列表存进槽位。

·底层实现：`del` 是语句（statement）而不是函数，它编译成 `DELETESUBSCR` 字节码；对 list 它走 `listassubscript` 的删除路径：取出引用、递减引用计数（可能触发对象的析构与内存回收）、`memmove` 前移后续元素、`obsz--`。`del L[1:]` 则直接 `memmove` 剩余元素并整体收缩

。

·设计原因：为什么既有 `del L[i]` 又有 `L.pop(i)`？`pop` 要"返回被删的值"（用于栈等场景），`del` 纯删除更高效、意图更清晰。Python 不搞"一个函数参数化所有"，而是提供多把钥匙。

·实际问题：`L.copy()`、`L[:]`、`list(L)` 三者都是浅拷贝（shallow copy）——只复制引用数组，不复制内层对象。若

内层是可变对象（如嵌套列表），修改内层仍会"共享"。
深拷贝需要 `copy.deepcopy`。

·初学者误区：① `L[0] = []` 与 `del L[0]` 分不清——前者列表长度不变（变成 `[[]]`），后者长度减一；② 对字符串执行 `del s[0]` 抛 `TypeError:'str' object doesn't support item deletion`——这是"不可变"最直观的报错教育；③ 以为 `copy()` 是深拷贝，改嵌套列表时惊掉下巴。

代码分析

```
L = ['hack', 'more', 'Py', 'code']
del L[0]                # 0 → ['more', 'Py', 'code']
del L[1:]               # 1 → ['more']

L = ['hack', 'more', 'Py', 'code']
L[0:1] = []            # del L[0]
L[1:] = []             # 1
L[0] = []              # "" → [[]]

L.copy()               # L[:]
list(L)               #
L[:]                  #
```

·解释器如何处理：`del L[1:]` 编译为 `DELETESUBSCR`，解释器发现下标是切片对象，调用 `listasslice` 的删除路径：被删区间的引用逐一递减引用计数（若引用计数归零，则该对象被回收，其 `del` 等终结逻辑在此时执行），然后 `memmove` 把尾部元素前移，最后可能触发 `listresize` 收缩内存。注意：Python 的"删除"不是清空内存，而是"解除引用 + 让引用计数（以及更底层的内存分配器）去处理回收"。

·设计思想：浅拷贝 vs 深拷贝的区分至关重要：Python 的复制默认"浅"——这与 C 的 memcopy 语义一致，效率高；需要深拷贝时显式请求。这正是"显式优于隐式"在内存模型上的体现。

8.4 字典 (Dictionaries)

英文原文

Along with lists, dictionaries are one of the most flexible built-in object types in Python. If you think of lists as order-based collections of objects, you can think of dictionaries as key-based collections; the chief distinction is that in dictionaries, items are stored and fetched by key, instead of by positional offset. While lists can serve roles similar to arrays in other languages, dictionaries can take the place of records, search tables, and any other sort of aggregation where item names are more meaningful than item positions. For example, dictionaries can replace many of the searching algorithms and data structures you might have to implement manually in lower-level languages—as a highly optimized built-in tool, indexing a dictionary is a very fast search operation. Dictionaries also sometimes do the work of records and symbol tables used in other languages, can be used to represent sparse (mostly empty) data structures, and much more. As a ru

ndown of their main properties, Python dictionaries are: Accessed by key, not offset position Dictionaries are sometimes called associative arrays or hashes (especially by users of other scripting languages). They associate a set of values with corresponding keys, so you can fetch an item out of a dictionary using the key under which you originally stored it. You use the same indexing operation to get components in a dictionary as you do in a list, but the index takes the form of a key, not a relative offset. Insertion-ordered collections of arbitrary objects Unlike in a list, keys in a dictionary are ordered only by the order in which they are inserted. This is not the same as the positional ordering of sequences like lists: items added to dictionaries later always show up at the end of the keys list (even if they appeared earlier in the past), and there is no way to insert a key into the middle of the keys list. Moreover, this ordering is new as of Python 3.7, before which keys' left-to-right order was pseudo random. Under either ordering regime, keys provide the symbolic (not physical) locations of items in a dictionary. Variable-length, heterogeneous, and arbitrarily nestable Like lists, dictionaries can grow and shrink in place (without new copies being made), they can contain objects of any type, and they support nesting to any depth (they can be freely mixed with lists, other dictionaries, and so on). Each key can have just one associated value, but that value can be a collection of multiple o

jects if needed, and a given value can be stored under any number of keys. Of the category "mutable mapping" You can change dictionaries in place by assigning to indexes (they are mutable), but they don't support the sequence operations that work on strings and lists. Because dictionaries are key-based collections, operations that depend on a fixed positional order (e.g., concatenation, slicing) don't make sense. Instead, dictionaries are the only built-in, core-type representatives of the mapping category—objects that map keys to values. Other mappings in Python are created by imported modules, not language syntax. Tables of object references (hash tables) If lists are arrays of object references that support access by position, dictionaries are tables of object references that support access by key. Internally, dictionaries are implemented as hash tables (data structures that support very fast retrieval), which start small and grow on demand. Moreover, Python employs optimized hashing algorithms to find keys, so retrieval is quick. Like lists, though, dictionaries store object references (not copies, unless you make them explicitly before storing).

中文翻译

与列表一起，字典（dictionary）是 Python 中最灵活的内置对象类型之一。如果把列表想成“基于位置的对象集合”，那么可以把字典想成“基于键（key）的对象集合”；最主要的区别是：在字典中，项按键存储和取出，而不是按位置偏

移。列表可以扮演其他语言中数组的角色，而字典可以取代记录 (record)、查找表 (search table)，以及任何"项的名字比位置更有意义"的聚合结构。例如，字典可以替代你在低级语言中不得不手动实现的许多搜索算法和数据结构——作为一个高度优化的内置工具，索引字典是一种非常快速的搜索操作。字典有时还扮演其他语言中记录和符号表 (symbol table) 的角色，可以用来表示稀疏的（大部分为空的）数据结构，还有很多其他用途。下面快速浏览它的一些主要特性。Python 字典是：按键访问，而非按偏移位置访问字典有时被称为关联数组 (associative arrays) 或哈希表 (hashes)（尤其是其他脚本语言的用户这么叫）。它们把一组值与其对应的键关联起来，所以你可以用当初存储项时所用的键来取出字典中的项。从字典中取组件所用的索引操作与列表中完全相同，只不过这里的"索引"是一个键，而不是相对偏移。按插入顺序排列的任意对象集合与列表不同，字典中的键只按它们被插入的顺序排列。这不同于列表这类序列的位置顺序：后加入字典的项总是出现在键列表的末尾（即使它们更早就出现过），而且没有办法把一个键插入到键列表的中间。此外，这种顺序是从 Python 3.7 才开始的新特性；在此之前，键的从左到右顺序是伪随机的。无论采用哪种顺序制度，键提供的都是字典中项的符号位置 (symbolic location) 而非物理位置。可变长度、异构、可任意嵌套与列表一样，字典可以原地增长和收缩（无需制作新副本），可以包含任何类型的对象，并支持任意深度的嵌套（可以自由地与列表、其他字典等混合）。每个键只能有一个关联值，但该值如果需要可以是一个包含多个对象的集合，而且同一个值可以存放在任意多个键名下。属于"可变映射"类别你可以通过给索引赋值来原地修改字典（它

们是可变的)，但它们不支持作用于字符串和列表的序列操作。因为字典是基于键的集合，依赖固定位置顺序的操作（如拼接、切片）没有意义。相反，字典是映射（mapping）类别——把键映射到值的对象——唯一的、核心类型的内置代表。Python 中的其他映射由导入的模块创建，而不是语言语法。对象引用的表（哈希表）如果说列表是支持按位置访问的对象引用数组，那么字典就是支持按键访问的对象引用表。在内部，字典被实现为哈希表（hash tables）（支持极速检索的数据结构），它们从很小开始并随需增长。此外，Python 采用优化的哈希算法来查找键，因此检索很快。不过与列表一样，字典存储的是对象引用（不是副本，除非你在存储前显式制作了一份）。

深度理解

·核心概念：列表 = "位置→对象"的映射；字典 = "键→值"的映射。五条特性给字典定位：按键访问、插入有序、可变的、异构、可嵌套、可变、哈希表实现。字典是 Python 中唯一的"核心映射（mapping）"类型。

·底层实现：CPython 的 dict 是 compact dict（紧凑字典）（3.6 起）：内部有两块数组——indices（索引表）和 entries（条目表）。插入顺序由"条目在 entries 中的下标"天然决定，因此按键迭代即按插入顺序。查找时先算键的哈希值，经 indices 定位到条目，只做一次（或几次）探测（见下文深度理解）。

·设计原因："符号位置而非物理位置"是字典的哲学核心：键是"名字"，排序/位置是附带品（3.7 之前甚至不保证）。这让字典可以表达"字段名→字段值"的记录，也让"按名

字查找"成为 $O(1)$ 而非线性搜索。

·实际问题：字典的"几乎 $O(1)$ 查找"取代了低级语言里手写哈希表、查找树 (search tree/table) 的大量工作；JSON 的天然对应物就是字典；程序配置、对象属性、符号表都是字典的典型应用。

·初学者误区：① 以为 dict 无序（那是 3.7 之前的旧印象）——现在它插入有序；② 以为"键必须排序/有位置"——其实"符号"比"位置"重要；③ 混淆"关联数组"这个词，其实它就是哈希表；④ 在 3.6 之前写的代码里"依赖 dict 顺序"的 bug 在新版本中反而"碰巧正确"，这是移植老代码时的经典陷阱。

Table 8-2 (表 8-2：常用字典字面量与操作)

For reference and preview, Table 8-2 summarizes some of the most common and representative dictionary operations and is relatively complete at this writing. As usual, though, you should consult Python's library manual or run a `dir(dict)` or `help(dict)` call for a complete list (dict is the built-in name of the dictionary type).

作为参考和预览，表 8-2 总结了部分最常见、最有代表性的字典操作，截至写作时它相对完整。不过照例，你应该查阅 Python 库手册，或运行 `dir(dict)` 或 `help(dict)` 调用来获取完整清单（dict 是字典类型的内置名字）。

Operation	Interpretation (英文解释)	中文解释	
D = {}	Empty dictionary	空字典	

D = {'name': 'Pat', 'age': 40.0}	Two-item dictionary	两项字典	
E = {'cto': {'name': 'Sue', 'age': 40}}	Nesting	嵌套	
D = dict(name='Bob', age=40)	Alternative construction techniques: keywords, key/value pairs, zipped key/value lists,	其他构造方法: 关键字、键/值对、zip 生成的键/值列表、键列表	
D = dict([('name', 'Bob'), ('age', 40)])	(同上)	(同上)	
D = dict(zip(keylist, valueslist))	(同上)	(同上)	
D = dict.fromkeys(['name', 'age'])	(同上)	(同上)	
D['name'], E['cto']['age']	Indexing by key, nested index	按键索引、嵌套索引	
'age' in D, for key in D: print(D[key])	Key membership and iteration	键成员测试与迭代	
D.keys(), D.values(), D.items()	Methods: all keys, all values, all key+value tuples	方法: 全部键、全部值、全部键值对元组	
D.copy()	copy (top-level)	复制 (顶层/浅拷贝)	
D.clear()	clear (remove all items)	清空 (移除所有项)	
D.update(D2)	merge by keys	按键合并	
D.get(key, default?)	fetch by key, if absent default (or None)	按键获取, 若不存在则返回默认值 (或 None)	
D.pop(key, default?)	remove by key, if absent default (or error)	按键移除, 若不存在则返回默认值 (或报错)	
D.setdefault(key, default?)	fetch by key, if absent set default (or None)	按键获取, 若不存在则设置默认值 (或 None)	

D.popitem()	remove/return any (key, value) pair; etc.	移除/返回任意一个键值对; 等	
len(D)	Length: number of stored entries	长度: 存储的条目数	
D[key] = 62	Adding keys, changing key values	添加键、改变键的值	
del D[key]	Deleting entries by key	按键删除条目	
D1 == D2	Comparisons: equality (only)	比较: 相等 (仅此而已)	
list(D.keys())	Dictionary views	字典视图 (view)	
D1.keys() & D2.keys()	(同上)	(同上)	
D = {'x': 1, 'y', 'z'}	Dictionary unpacking	字典解包	
D = {x: x ² for x in range(10)}	Dictionary comprehensions	字典推导式	
D \	E	Dictionary merge expression: copy + update	字典合并表达式: copy + update

Per Table 8-2, when coded as a literal expression, a dictionary is written as a series of key:value pairs separated by commas, and enclosed in curly braces. [4] An empty dictionary is an empty set of curly braces, and you can nest dictionaries by simply coding one as a value inside another dictionary, or within a list or tuple literal. There's a lot more to dictionaries than their literals, though, as the next section's tutorial will begin to reveal.

如表 8-2 所示, 当以字面量表达式书写时, 字典被写成一系列用逗号分隔的键:值对, 并用花括号括起来。[4] 空字典

就是一对空花括号；你可以通过把一个字典作为另一个字典的值，或写进列表、元组字面量中来实现嵌套。不过，字典的内涵远不止字面量，下一节的教程将开始揭示这一点。

深度理解

·核心概念：表 8-2 是字典的操作全景图：构造（字面量/`dict()`/`zip/fromkeys`）、访问（`D[k]`/`get`）、修改（赋值/`update`/`pop`/`del`/`clear`）、查询（`in`/`keys`/`values`/`items`/`len`）、视图（`view`）、推导式、解包、| 合并（3.9+）。

·底层实现：字典的哈希表有两个关键点：① 键必须可哈希（hashable）——即不可变，哈希值在其生命周期内不变，否则查找会“找不到自己”；② 哈希表以空间换时间：底层 slot 数量总是大于条目数（负载因子约 2/3），保证查找平均 $O(1)$ 。

·设计原因：为什么字面量用花括号 `{}`？历史上 `{1,2,3}` 是集合（set），`{'a':1}` 是字典——语法上靠“有无冒号”区分。空花括号 `{}` 是字典，空集合必须写 `set()`，这是一个著名的历史包袱，务必记住。

·实际问题：`dict.fromkeys(keys, default)` 用于“一次性初始化一群键为同一默认值”；`zip` 拼键值对列表；`in` 判键是否存在——这些是处理动态数据（从文件、网络来的运行时数据）的日常操作。

·初学者误区：① 用 `D[k]` 取不存在的键会 `KeyError`——想安全用 `get`；② 忘了键是“符号”，把顺序和排序的期望强加给字典；③ 花括号 `{}` 误当空集合用。

8.5 字典实战 (Dictionaries in Action)

英文原文

As Table 8-2 summarizes, dictionaries are indexed by key, and nested dictionary entries are referenced by a series of indexes (keys in square brackets). When Python creates a dictionary, it stores its items in a way that associates values with keys; to fetch a value back, you supply the key with which it is associated, not its relative position. Let's go back to the interactive prompt to see what this all looks like in code.

中文翻译

如表 8-2 所总结的，字典按键索引，嵌套字典条目通过一串索引（方括号里的键）引用。Python 创建字典时，会以“把值关联到键”的方式存储条目；要取回一个值，你提供与之关联的键，而不是它的相对位置。让我们回到交互提示符，看看这一切在代码里长什么样。

深度理解

- 核心概念：强调“以键取回，而非位置”——这是字典与列表在思维模型上最根本的分野。
- 底层实现：D['Py'] 的求值过程是：计算'Py' 的哈希值 → 用哈希值定位索引表 → 找到条目表中的槽 → 校验键确实相等（哈希碰撞时也走这里）→ 返回值引用。全程与“位置”无关。

·设计原因：把"取数"的口令从"第几个"升级为"叫什么名字"，是人类记忆的自然方式——这也是为什么字典在表达"记录"时胜过列表。

·实际问题：所有"名词查找"类需求（配置项、用户名→资料、状态码→描述）都用字典。

·初学者误区：把字典当作"自动排序的列表"——它是"名字→内容"的通讯录，排序语义只是附带。

8.5.1 基本字典操作 (Basic Dictionary Operations)

英文原文

In normal operation, you create dictionaries with literals and store and access items by key with indexing:

```
$ python3 >>> D = {'hack': 1, 'Py': 2, 'code': 3} # Make a dictionary
>>> D['Py'] # Fetch a value by key
>>> D # Insertion ordered {'hack': 1, 'Py': 2, 'code': 3}
```

Here, the dictionary is assigned to the variable `D`; the value of the key `'Py'` is the integer 2, and so on. We use the same square bracket syntax to index dictionaries by key as we did to index lists by offset, but here it means access by key, not by position. Notice the end of this example: unlike the randomly ordered sets we studied in Chapter 5, dictionary keys retain their insertion order today, but we need to cover some more basics before formalizing this. The built-in `len` func

tion works on dictionaries, too—it returns the number of values stored in the dictionary or, equivalently, the number of its keys. The dictionary in membership operator allows you to test for key existence, and the `keys` method returns all the keys in the dictionary. The latter of these can be useful for processing dictionaries in full, by fetching corresponding values in loops. Because the keys result can be treated like a normal list, it can also be sorted if order matters and the automatic insertion order doesn't do the job (more on sorting and dictionaries later):

```
>>> len(D) # Number of entries in dictionary 3
>>> 'code' in D # Key membership test True
>>> list(D.keys()) # Create a new list of D's keys ['hack', 'Py', 'code']
```

Notice the second step in this listing. As noted, the `in` membership test used for strings and lists also works on dictionaries, where it checks whether a key is stored. Technically, this works because dictionaries define a protocol run by `in` to lookup a key quickly. Other objects provide `in` protocols that reflect their common uses; files, for example, use iterators that read line by line. This will matter later in this book, when we reach iterators and classes. Also note the syntax of the last step in this listing. It uses `list` for similar reasons—the dictionary `keys` method returns an iterable “view” object that produces results on demand, instead of

f a physical list. The list call forces it to serve up all its values at once for display at the interactive REPL, though this call isn't required and may even be subpar in some other contexts. More on this, as well as other dictionary basics like comparisons, later in this chapter.

中文翻译

在正常操作中，你用字面量创建字典，并通过索引按键存储和访问项：\$ python3 >>> D = {'hack': 1, 'Py': 2, 'code': 3} # 创建一个字典 >>> D['Py'] # 按键取出一个值 2 >>> D # 插入有序{'hack': 1, 'Py': 2, 'code': 3} 这里，字典被赋给变量 D；键 'Py' 的值是整数 2，等等。我们使用与索引列表相同的方括号语法按键索引字典，但这里的意思是按键访问，而不是按位置。注意这个示例的结尾：与第 5 章学过的随机无序的集合 (set) 不同，字典键如今保持插入顺序，但我们需要先覆盖一些更基础的内容，再把它正式化。内置 len 函数也适用于字典——它返回字典中存储的值的数量，等价于键的数量。字典的 in 成员操作符允许你测试键是否存在，keys 方法返回字典中的所有键。最后这个对“完整处理字典”很有用——可以在循环中取出相应的值。因为 keys 的结果可以像普通列表一样对待，当顺序很重要而自动插入顺序不能满足时，也可以对它排序（关于排序和字典的更多内容在后面）：>>> len(D) # 字典中的条目数 3 >>> 'code' in D # 键成员测试 True >>> list(D.keys()) # 新建一个 D 的键列表 ['hack', 'Py', 'code'] 注意这个列表中第二步。如前所述，用于字符串和列表的 in 成员测试也适用于字典，它检查某个键是否被存储。从技术上讲，这是因

为字典为 `in` 定义了一个可以进行快速键查找的协议。其他对象提供的 `in` 协议反映各自的常见用途；例如，文件使用逐行读取的迭代器。这一点在本书后面讲到迭代器和类时会变得重要。还要注意列表中最后一步的语法。它使用 `list` 是出于类似的原因——字典的 `keys` 方法返回一个可迭代的“视图 (view)”对象，它按需产生结果，而不是物理列表。`list` 调用强制它一次交出全部值，以便在交互式 REPL 中显示；虽然这个调用并非必需，在某些其他情境下甚至可能不是最优。关于这一点，以及字典的其他基础知识（比如比较），本章后面会有更多内容。

深度理解

- 核心概念：字典的三个基本查询 API：`D[k]`（按键取值）、`in`（判键存在）、`len`（条目总数）、`keys()`（取所有键）。注意 `in` 查的是键而不是值——`'code' in D` 不检查有没有值是 `'code'`。
- 底层实现：`'code' in D` 走 `dictcontains`：算出 `'code'` 的哈希值 → 在哈希表中定位 → 若找到且键相等返回 `True`。这是 $O(1)$ 的，而列表的 `x in L` 是 $O(n)$ 线性查找——同样的语法，底层复杂度天差地别。`D.keys()` 返回 `dictkeys` 视图对象（懒加载，不占用 $O(n)$ 内存），`list()` 把它物化成真列表。
- 设计原因：“`in` 对不同的容器用不同的协议”体现了 Python 的鸭子协议思想：语言只规定语义（“是否存在”），具体实现由对象自己高效完成。这也是后面迭代器、类（多态）的伏笔。

·实际问题：想在循环里同时得到"键"和"值"，优先用 `for k, v in D.items()`（效率与可读性双优），而不是 `for k in D.keys()` 再 `D[k]` 二次查找。

·初学者误区：① `in` 误以为查值；② 忘了 `keys()` 返回的是视图不是列表，直接对它 `.sort()` 报 `AttributeError`（后面会细讲）；③ 用字典模拟"有没有某个值"时，忘了先判断键再取值（见 8.5.10 的稀疏矩阵与防 `KeyError` 三法）。

代码分析

```
D = {'hack': 1, 'Py': 2, 'code': 3} #  
D['Py']                             # 2"  
len(D)                              # 3  
'code' in D                         # True0(1)  
list(D.keys())                      # ['hack', 'Py', 'code']...
```

·解释器如何处理：`D['Py']` 编译为 `BINARYSUBSCR`，解释器调用 `PyDictGetItem`（底层走 `dictlookup`）：先调用字符串 `'Py'` 的 `hash` 得到哈希值 `h`，然后 `h & mask`（掩码定位，`mask = 表大小 - 1`，等价 `mod`）得到索引表槽位，从索引表取出条目下标，比较存储键与目标键（用 `==`，哈希碰撞时多个候选逐一比较）。命中即返回值引用；`dict` 的 `missing` 未定义，否则抛 `KeyError`。`'code' in D` 走 `dictcontains`，流程相同但只返回布尔。整体 `O(1)`。

·设计思想："索引"这个动词被复用到列表（位置）和字典（键）上，说明 Python 用同一套外观、不同的语义统一了容器访问——这是"少即是多"语法的体现。

8.5.2 原地修改字典 (Changing Dictionaries in Place)

英文原文

Let's continue with our interactive session. Dictionaries, like lists, are mutable, so you can change, expand, and shrink them in place without making new dictionaries: simply assign a value to a key to change or create an entry. The `del` statement works here, too; it deletes the entry associated with the key specified as an index.

Notice also the nesting of a list inside a dictionary in this example (the value of the key 'Py'); all collection data types in Python can nest inside each other arbitrarily, and can be changed independently of their containing objects:

```
>> D {'hack': 1, 'Py': 2, 'code': 3} >> D['Py'] = ['app', 'dev'] # Change entry (value=list)
>> D {'hack': 1, 'Py': ['app', 'dev'], 'code': 3} >> del D['code'] # Delete entry (also D.pop(k))
>> D {'hack': 1, 'Py': ['app', 'dev']} >> D['years'] = 32 # Add new entry
>> D {'hack': 1, 'Py': ['app', 'dev'], 'years': 32} >> D['Py'][0] = 'program' # Change a nested list in place
>> D {'hack': 1, 'Py': ['program', 'dev'], 'years': 32}
```

Like lists, assigning to an existing index in a dictionary changes its associated value. Unlike lists, however,

whenever you assign a new dictionary key (one that isn't already present) you create a new entry in the dictionary, as was done in the previous example for the key 'years'.

This doesn't work for lists because you can only assign to existing list offsets—Python considers an offset beyond the end of a list out of bounds and raises an error. As you learned earlier, to expand a list, you need to use tools such as the append method or slice assignment instead.

中文翻译

让我们继续这个交互会话。字典与列表一样是可变的，所以你可以原地修改、扩展和收缩它们，而无需制作新字典：只需给一个键赋值就能改变或创建一个条目。del 语句在这里也适用；它会删除与指定键（作为索引）相关联的条目。还要注意这个示例中字典里嵌套了一个列表（键 'Py' 的值）；Python 的所有集合数据类型都可以任意地互相嵌套，并且可以独立于其包含对象被修改：

```
>>> D {'hack': 1, 'Py': 2, 'code': 3} >>> D['Py'] = ['app', 'dev'] # 改变条目（值是个列表）>>> D {'hack': 1, 'Py': ['app', 'dev'], 'code': 3} >>> del D['code'] # 删除条目（也可用 D.pop(k)）>>> D {'hack': 1, 'Py': ['app', 'dev']} >>> D['years'] = 32 # 添加新条目>>> D {'hack': 1, 'Py': ['app', 'dev'], 'years': 32} >>> D['Py'][0] = 'program' # 原地修改嵌套列表>>> D {'hack': 1, 'Py': ['program', 'dev'], 'years': 32}
```

与列表一样，给字典中已有的索引赋值会改变它关联的值。与列表不同的是，每当你给一个新的字典键（尚未存在的键）赋值，就会

在字典中创建一个新条目，就像上面例子中对键'years'所做的那样。这对列表不成立，因为你只能给列表中已存在的偏移赋值——Python 认为超出列表末尾的偏移越界，会抛出错误。正如你之前学到的，要扩展列表，需要用诸如 `append` 方法或切片赋值之类的工具。

深度理解

·核心概念：赋值给键 = 创建或覆盖。一个语法 `D[k] = v` 同时完成"插入新键"和"更新旧键"两件事——这是 `dict` 与 `list` 在"增长方式"上的关键差异。

·底层实现：`D['Py'] = ['app','dev']` 走 `dictasssub`：若键已存在，找到旧条目，替换 `value` 引用，递减旧值的引用计数；若键不存在，触发插入路径：若负载因子超阈值则 `rehash` 扩容，再找空槽填入 (`hash, key, value`) 三元组。
`del D['code']` 走 `dictdelitem`：找到条目后，把该槽标记为"已删除哨兵" (`dummy`)，真正删除在 `rehash` 时清理。

·设计原因：为什么字典能"自动长出"新键而列表不能？因为列表的位置是连续物理内存，跳着赋 `L[99]` 就要预分配 100 格（或者浪费）；而字典用键的哈希定位，天然支持"稀疏分配"——只为实际存在的键占内存。这也是"字典可以当柔性列表用"的物理基础。

·实际问题：往字典加数据是"无脑赋值"：`config['timeout'] = 30` 不用先检查存在与否。而列表必须 `append`。这个差异在 8.5.10 里会被用来"模拟会长大的列表"。

·初学者误区：① 把"字典的键能自动新建"套用到列表上（

L[99]='x' 报 IndexError) ; ② 修改嵌套结构 D['Py'][0] 时忘了会共享修改——如果 D2 = D, 改 D['Py'] 也能看到; ③ 用 D[k] 更新值时以为旧值对象被销毁——如果旧值还被其他变量引用, 它仍然活着。

代码分析

```
D = {'hack': 1, 'Py': 2, 'code': 3}
D['Py'] = ['app', 'dev']      # →
del D['code']                 # 'code'
D['years'] = 32               # →
D['Py'][0] = 'program'        # ['app', 'dev'] 0
D                             # {'hack': 1, 'Py': ['program', 'dev'...
```

·解释器如何处理: D['Py'][0] = 'program' 有两步: ① BINARYSUBSCR 取出 D['Py']——返回一个指向列表 ['app', 'dev'] 的引用 (注意: 这是引用, 不是拷贝); ② STORESUBSCR 对该列表执行索引赋值。因为列表可变, 第 ① 步取到的对象被就地修改。整个过程不产生新的字典对象。

·设计思想: 容器里的嵌套结构是"引用树"而非"复制树"——修改内层对象不必重新构建整个外层。这让"以字典为骨架的复杂数据结构"可以就地更新, 代价是要时刻记得共享引用。

8.5.3 更多字典方法 (More Dictionary Methods)

英文原文

Dictionary methods provide a variety of type-specific tools. For instance, the dictionary values and items m

methods return all of the dictionary's values and (key, value) pair tuples, respectively; along with keys, these are useful in loops that need to step through dictionary entries one by one (we'll start coding such loops later in this chapter).

As with keys, these two methods also return iterable objects, and wrapping them in a list call collects their values all at once for display:

```
>> D = {'program': 1, 'script': 2, 'app': 3} >> list(D.keys()) # All keys ['program', 'script', 'app'] >> list(D.values()) # All values [1, 2, 3] >> list(D.items()) # All (key, value) tuples [('program', 1), ('script', 2), ('app', 3)]
```

In realistic programs that gather data as they run, you often won't be able to predict what will be in a dictionary before the program is launched, much less when it's coded. Fetching a nonexistent key is normally an error, but the `get` method returns a default value—`None`, or a passed-in default—if the key doesn't exist.

It's an easy way to fill in a default for a key that isn't present and avoid a missing-key error when your program can't anticipate contents ahead of time:

```
>> D.get('script') # A key that is present 2 >> print(D.get('code')) # A key that is missing None >> D.get('code', 4) # With a passed-in default 4
```

The `update` method provides something similar to `c`

concatenation for dictionaries (though it works in the realm of keys and values, not just values). It merges the keys and values of one dictionary into another, both adding new entries for new keys, and blindly overwriting values of the same key if there's a clash:

```
>> D {'program': 1,'script': 2,'app': 3} >> D2 = {'code':4,'hack':5,'app': 6} # New keys added, app:6 wins >>
D.update(D2) >> D {'program': 1,'script': 2,'app': 6,'code': 4,'hack': 5}
```

Finally, the dictionary pop method deletes a key from a dictionary and returns the value it had. It's similar to the list pop method, but it takes a key instead of an optional position:

```
>> D.pop('app') # Pop a dictionary key 6 >> D.pop('hack') # Delete key and return its value 5 >> D {'program': 1,'script': 2,'code': 4} >> L = ['aa','bb','cc','dd'] >>
L.pop() # Delete and return from the end'dd'>> L.pop(1) # Delete from a specific position'bb'>> L ['aa','cc']
```

Dictionaries also provide a copy method that, as you might guess, makes a copy; we'll revisit this in Chapter 9, as it's a way to avoid the potential side effects of shared references to the same dictionary. In fact, dictionaries come with more methods than the common ones demoed here, and over time may gain others beyond those listed in Table 8-2; again, see the Python library manual, dir and help, or other reference res

sources for a comprehensive list.

中文翻译

字典方法提供了各种各样的类型特有工具。例如，字典的 `values` 和 `items` 方法分别返回字典的全部值和（键，值）对元组；与 `keys` 一起，这些方法在“需要逐条遍历字典条目”的循环中非常有用（本章后面会开始编写这类循环）。与 `keys` 一样，这两个方法也返回可迭代对象，用 `list` 调用包装它们就能一次性收集全部值以便显示：

```
>>> D = {'program': 1, 'script': 2, 'app': 3} >>> list(D.keys()) # 全部键['program', 'script', 'app'] >>> list(D.values()) # 全部值[1, 2, 3] >>> list(D.items()) # 全部（键，值）元组[('program', 1), ('script', 2), ('app', 3)]
```

在真实程序里，数据是运行时收集的，你常常无法在程序启动前（更不用说在编码时）预知字典里会有什么。取出一个不存在的键通常是错误，但 `get` 方法会在键不存在时返回一个默认值——`None`，或一个传入的默认值。这在“你的程序无法提前预知内容”时，是一种为缺失键填充默认值、避免缺键错误的简便方式：

```
>>> D.get('script') # 存在的一个键2 >>> print(D.get('code')) # 缺失的一个键None >>> D.get('code', 4) # 传入默认值4
```

`update` 方法提供了类似“字典的拼接”的功能（不过它工作于键和值的领域，而不只是值）。它把一个字典的键和值合并（merge）进另一个字典：既为新键添加新条目，又在键冲突时盲目地覆盖同键的值：

```
>>> D {'program': 1, 'script': 2, 'app': 3} >>> D2 = {'code': 4, 'hack': 5, 'app': 6} # 新键被添加，app:6 获胜 >>> D.update(D2) >>> D {'program': 1, 'script': 2, 'app': 6, 'code': 4, 'hack': 5}
```

最后，字典的 `pop` 方法从字典中删除一个键并返回它原来的值

。它与列表的 pop 方法相似，但接收的是一个键而不是一个可选位置：

```
>>> D.pop('app') # 弹出字典键6 >>> D.  
pop('hack') # 删除键并返回其值5 >>> D {'program': 1,  
script': 2,'code': 4} >>> L = ['aa','bb','cc','dd'] >>> L.p  
op() # 从末尾删除并返回'dd'>>> L.pop(1) # 从指定位置  
删除'bb'>>> L ['aa','cc']
```

字典还提供一个 copy 方法做复制——谁都能猜到；我们会在第 9 章重新讨论它，因为它是避免"共享引用到同一字典"产生潜在副作用的一种方式。事实上，字典的方法比这里演示的这些常用方法还多，并且随着时间推移可能还会增加表 8-2 之外的新方法；同样，请查阅 Python 库手册、dir 和 help，或其他参考资源获取完整清单。

深度理解

·核心概念：字典的"安全带"三件套——get（安全取值）、update（合并）、pop（删除并取回）。它们共同解决"运行时数据不确定"这一现实问题。

·底层实现：D.get(k, default) 与 D[k] 走同一条哈希查找路径，差异只在末尾：找不到键时，get 返回默认值（不抛异常），D[k] 抛 KeyError。update 是"循环赋值"（for k in D2: D1[k] = D2[k]）的 C 层优化版。pop 在成功删除后返回值引用。

·设计原因：为什么不干脆让 D[k] 一直返回默认值？因为"取缺失键"往往意味着逻辑错误（拼错了键名、数据缺失），静默返回 None 会掩盖 bug；get 则是把"可能缺失"这个意图显式说出来。显式优于隐式在异常处理上的又一次体现。

·实际问题：统计词频的经典写法是利用 get 的默认值：count[w] = count.get(w, 0) + 1；合并配置时 update 让默认配置被用户配置覆盖；pop 常用于"消费式取出"（取走即删除）。

·初学者误区：① D.get('k') 找不到返回 None，但值本身可能是 None——无法区分"键不存在"与"值为 None"，需要时改用 if k in D；② update 是覆盖性合并，想要"不覆盖已有键"需先判断或 setdefault；③ pop 找不到键默认抛 KeyError，想安全要传第二个参数 D.pop(k, default)。

代码分析

```
D = {'program': 1, 'script': 2, 'app': 3}
list(D.keys())           # ['program', 'script', 'app']
list(D.values())         # [1, 2, 3]
list(D.items())          # [('program', 1), ('script', 2), ...

D.get('script')          # 2
D.get('code')            # None None
D.get('code', 4)         # 4

D2 = {'code': 4, 'hack': 5, 'app': 6}
D.update(D2)             # 'app' 6
D.pop('app')             # 6

L = ['aa', 'bb', 'cc', 'dd']
L.pop()                  # 'dd' pop
L.pop(1)                 # 'bb'
```

·解释器如何处理：D.get('code', 4) 编译为 LOADMETHOD get + CALL。解释器在 PyDictType 上找到 dict.get（一个 C 函数），传入 self=D、key='code'、default=

4. 底层 dictget: 调用 dictlookup 做哈希查找, 未命中则返回 default (这里未命中返回 4, 而 D.get('code') 未命中返回 None 常量)。D.update(D2) 走 dictmerge, 本质上是对 D2 的每个键执行插入/覆盖。

·设计思想: "get/pop 带默认值"是 Python 函数签名里 default? (可选参数) 的常见形态——用一个参数统一"常态"与"缺省"两类需求, 避免为每种情况发明新方法。

8.5.4 字典的其它构造方法 (Other Dictionary Makers)

英文原文

Because dictionaries are so useful, multiple ways to build them have emerged over time. The following summarizes the most common alternatives; its last two calls to the dict constructor (really, type name) have the same effect as the literal and key-assignment forms listed above them: {'name': 'Pat', 'age': 40} # 1) Traditional literal expression D = {} # 2) Assign by keys dynamically D['name'] = 'Pat' D['age'] = 40 dict(name='Pat', age=40) # 3) dict keyword-argument form dict([('name', 'Pat'), ('age', 40)]) # 4) dict key/value tuples form All four of these forms create the same two-key dictionary, but they are useful in differing circumstances:- The first is handy if you can spell out the entire dictionary ahead of time.- The second is of use if you need to create the dictionary one field at a time on t

he fly.- The third involves less typing than the first, but it requires all keys to be strings.- The last is useful if you need to build up keys and values as sequences at runtime. We met name=value keyword arguments earlier when sorting lists; the dict form in this summary that uses them is newer than literals and common in Python code, because it has less syntax (and hence less room for mistakes). The last form in the summary is also commonly used in conjunction with the zip function, to combine separate lists of keys and values obtained dynamically at runtime (parsed out of a data file's columns, for instance): dict(zip(keylist, valueslist)) # Zipped key/value tuples form (ahead) There is more on zipping dictionary keys in the next section. Provided all the key's values are the same initially, you can also create a dictionary with the following special form—simply pass in a list of keys and an initial value for all of the values (the default is None). Notice this is run from the dict type name, not an actual dictionary: >>> dict.fromkeys(['a','b'], 0) {'a': 0,'b': 0} Although you could get by with just literals and key assignments at this point in your Python career, you'll probably find uses for all of these dictionary-creation forms as you start applying them in realistic Python programs.

中文翻译

因为字典非常有用，随着时间的推移出现了多种构建它们的方式。下面总结了最常见的替代方案；其中对 dict 构造器

(实际上是类型名)的最后两次调用,与上面列出的字面量和按键赋值形式效果相同: `{'name':'Pat','age': 40} # 1)` 传统字面量表达式 `D = {} # 2)` 运行动态按键赋值 `D['name'] = 'Pat'` `D['age'] = 40` `dict(name='Pat', age=40) # 3)` `dict` 关键字参数形式 `dict([('name','Pat'), ('age', 40)]) # 4)` `dict` 键/值元组形式这四种形式创建的都是同一个双键字典,但它们在以下不同场景下各有用途: - 第一种:如果你能提前把整个字典完整写出来,很方便。 - 第二种:如果你需要在运行中一个字段一个字段地构建字典,很有用。 - 第三种:比第一种打字更少,但要求所有键都必须是字符串。 - 最后一种:如果你需要在运行时把键和值作为序列构建出来,很有用。我们在给列表排序时已经遇到过 `name=value` 关键字参数;本总结中使用它们的 `dict` 形式比字面量更新,在 Python 代码中很常见,因为它的语法更少(因此出错空间也更小)。总结中的最后一种形式也常与 `zip` 函数配合使用,用来把运行时动态获得的、彼此独立的键列表和值列表组合起来(例如从数据文件各列解析出来的): `dict(zip(keyslist, valueslist)) # zip 键/值元组形式` (后面会讲)下一节还会讲更多关于“用 `zip` 把键配成字典键”的内容。如果所有键的初始值都相同,你还可以用下面这个特殊形式创建字典——只需传入一个键列表和所有值共用的初始值(默认是 `None`)。注意这是从 `dict` 类型名调用,而不是从一个实际字典调用: `>>> dict.fromkeys(['a','b'], 0) {'a': 0, 'b': 0}` 虽然在你目前的 Python 生涯中,只用字面量和按键赋值也能应付,但当你开始把它们应用到真实 Python 程序时,你很可能会为这些字典创建形式的每一种都找到用武之地。

深度理解

·核心概念：造字典的“五大门路”：字面量（静态可写全）、逐步赋值（动态添加）、`dict(关键字)`（键须为字符串、打字少）、`dict(键值对序列)`（运行时拼装）、`fromkeys`（一键多值同初值）。

·底层实现：`dict(name='Pat', age=40)` 在解释器中走与字面量相同的构建路径，只是把关键字参数装进内部；`dict(iterable)` 则要求传入可迭代的键值对（如元组列表或 `zip` 对象），逐个插入。`dict.fromkeys` 是类方法（classmethod），内部循环把每个键插入且共享同一个默认值引用。

·设计原因：为什么需要这么多种构造方式？因为数据的来源千差万别：有的是硬编码常量（字面量），有的是运行时一行行解析（赋值/序列），有的是函数调用参数（关键字）。每种来源都有最顺手的表达。

·实际问题：`dict(zip(keys, values))` 是“两个平行列表合成字典”的标准答案——比如从 CSV 的表头与数据行构建行字典、从两个 API 返回的数组合成映射。`fromkeys` 常用于“统计前的初始化”（如哈希计数前的键集合）。

·初学者误区：① `dict(关键字)` 的键其实要满足标识符规则（必须是合法变量名），所以 `dict(key-with-dash=1)` 编译失败，而字面量 `{'key-with-dash': 1}` 可以；② `fromkeys(['a','b'], [])` 中所有键共享同一个空列表，改一个全变——这是共享引用陷阱的典型案列。

代码分析

```
# {'name': 'Pat', 'age': 40}
{'name': 'Pat', 'age': 40}          # 1)
D = {}                             # 2)
D['name'] = 'Pat'
D['age'] = 40
dict(name='Pat', age=40)            # 3)
dict([('name', 'Pat'), ('age', 40)]) # 4) zip

dict(zip(keylist, valueslist))      # →
dict.fromkeys(['a', 'b'], 0)        # {'a': 0, 'b': 0}

M = dict.fromkeys(['a', 'b'], [])   #
M['a'].append(1)                    # M['b'] [1]
```

·解释器如何处理：字面量 `{'name': 'Pat'}` 在编译期就已解析为"构建字典"的字节码序列（`BUILDMAP` + 存键值）。`dict(name='Pat')` 调用类型对象 `dict`（一个类），参数被封装后进入 `dict.init` 构建。`dict([(...), (...)])` 走 `PyDictSetItem` 逐对插入；若后续键重复，后者覆盖前者。`fromkeys` 内部把默认值对象只写一次并让所有键引用它——这也是"共享引用"bug 的根源。

·设计思想：类型名 `dict` 既能当"构造函数"又能当"类型转换器"（`dict(somedict)` 是新浅拷贝），这一专一名字的多重角色与 `list`、`str` 一脉相承。

8.5.5 字典字面量解包（Dictionary-literal unpacking）

英文原文

As of Python 3.5, dictionary literals also support a special syntax that unpacks the contents of another dictionary in its top level, similar to the in list literals we met earlier. Any number of can appear in a literal to unpack any number of dictionaries, and the rightmost's value wins when keys collide:

```
>>> D = dict(a=4, c=3) >>> {'a': 1, 'b': 2, D} # Dictionary-literal unpacking {'a': 4, 'b': 2, 'c': 3}
```

```
>>> dict(a=1, b=2) | D # Same, but with union operator {'a': 4, 'b': 2, 'c': 3}
```

As shown, the effect of is similar to both the update method we met earlier and the dictionary | union covered ahead and might avoid a follow-up update call in some contexts. It's also related to the extended-unpacking assignment statement you'll meet in Chapter 11. The also works in dict, but simply because unpacks keyword arguments in any function call (as usual in Python, this relies on larger concepts we'll reach later in this book). Unlike in {} literals though, in dict fails if any key appears twice:

```
>>> dict(a=1, {'b': 2}, dict(c=3)) # Works in dict(), as in all calls {'a': 1, 'b': 2, 'c': 3} # As long as no keys are repeated!
```

```
>>> dict(a=1, b=2, dict(a=4, c=3)) # Repeated keys:
TypeError: dict() got multiple values for keyword argu
```

ment'a'

The examples so far demo the many basic ways to create dictionaries, but there is yet another that's even more powerful, as the next section will explain.

中文翻译

从 Python 3.5 起，字典字面量也支持一种特殊的语法，可以在顶层解包另一个字典的内容——与我们之前看到的列表字面量里的类似。一个字面量中可以出现任意多个来解包任意多个字典，当键冲突时最右侧的值胜出：

```
>>> D = dict(a=4, c=3) >>> {'a': 1, 'b': 2, D} # 字典字面量解包{'a': 4, 'b': 2, 'c': 3} >>> dict(a=1, b=2) | D # 一样的效果，但用合并运算符{'a': 4, 'b': 2, 'c': 3} 如所示，的效果既类似于我们前面见过的 update 方法，也类似于后面要讲的字典 | 合并运算符，在某些场景下可以省掉一次后续的 update 调用。它还与第 11 章将介绍的"扩展解包赋值语句"相关。在 dict 里也能用，但那仅仅是因为在任何函数调用中都会解包关键字参数（和 Python 中通常的情况一样，这依赖本书后面才会讲到的更大概念）。不过与 {} 字面量不同，在 dict 中不允许同一个键出现两次：
```

```
>>> dict(a=1, {'b': 2}, dict(c=3)) # dict() 里可用，和所有调用一样{'a': 1, 'b': 2, 'c': 3} # 只要键不重复！ >>> dict(a=1, b=2, dict(a=4, c=3)) # 键重复了：TypeError: dict() got multiple values for key word argument 'a'
```

到目前为止，示例演示了很多创建字典的基本方式，但还有另一种更强大的——下一节会解释。

深度理解

·核心概念：解包是"字典合并"的一大流派：在构造时合并 `{a, b}`，一次成型。`update`（方法）是就地合并，`|`（运算符）是返回新字典合并——三者殊途同归。

·底层实现：`{D}` 在 CPython 中特化为 `BUILDMAPUNPACK` 字节码：解释器分配含两部分内容的空间，把右侧字典逐键拷贝进新字典（键冲突时后写覆盖先写——"右侧胜出"是这一执行顺序的直接结果）。

·设计原因：python 追求"一行表达式"的组合能力：`{defaults, overrides}` 是"先默认后覆盖"配置合并的标准表达，比先 `d = defaults.copy()` 再 `d.update(overrides)` 少两行。

·实际问题：函数调用 `f(d)` 把字典展开成关键字参数去调用函数；`{a, 'k': v}` 是"字典加一个键"的惯用表达式——这些在 API 封装、配置处理中被大量使用。

·初学者误区：① 混淆（列表解包/位置参数）与（字典解包/关键字参数）——星号个数决定语义；② 以为 `{D}` 是深拷贝——它仍是浅拷贝；③ 在 `dict(a=1, {'a':2})` 里踩"重复关键字参数"报错，而在 `{{'a':1}, {'a':2}}` 里却合法（后者右侧胜出）。

代码分析


```

D = dict(a=4, c=3)
{'a': 1, 'b': 2, **D}          # → {'a': 4, 'b': 2, 'c': 3}
dict(a=1, b=2) | D             # 3.9+

dict(a=1, **{'b': 2}, **dict(c=3))  # OK → {'a': 1, 'b': 2, ...}
dict(a=1, b=2, **dict(a=4, c=3))    # 'a'

def f(x, y): return x + y
f(**{'x': 1, 'y': 2})              # 3

```

·解释器如何处理：{'a': 1, 'b': 2, D} 编译为 BUILDMAP+BUILDMAPUNPACK 系列字节码。执行时解释器先建立空 dict，然后按字面量出现的先后顺序依次合并：先 {'a': 1, 'b': 2}，再把 D 的条目拷入——处理到 'a' 时发现已存在，覆盖为 4。因此"右侧胜出"不是魔法，而是"顺序写入、后写覆盖"的直接结果。而在 dict(...) 调用中，走的是函数调用的关键字参数解析（KWNAMES 机制），重复键在参数校验阶段直接抛 TypeError——这就是两者行为差异的根源。

·设计思想：同一种语法在不同上下文（字面量 vs 函数调用）有不同"契约"——字面量宽松合并，函数调用严格校验。理解"语法糖背后的求值顺序"比死记规则更重要。

8.5.6 字典推导式 (Dictionary Comprehensions)

英文原文

Like the set and list comprehensions we met in previous coverage, dictionary comprehensions run an implied loop, collecting the key/value results of expressions on each iteration and using them to fill out a new

dictionary. A loop variable allows the comprehension to use loop iteration values along the way.

The net effect lets us create new dictionaries with small bits of code that are simpler than the full-blown statements we'll study later in this book.

Abstractly, dictionary comprehensions map to for loops as follows, where both *k* and *v* can use loop variable *x*:

```
{k: v for x in iterable} # Dictionary comprehension  
new = {} # Equivalent loop code for x in iterable:  
new[k] = v
```

To illustrate, a standard way to initialize a dictionary dynamically is to combine its keys and values with `zip`, and pass the result to the `dict` call, per the last section.

The `zip` built-in function is the hook that allows us to construct a dictionary from key and value lists this way—if you cannot predict the set of keys and values in your code, you may be able to build them up as lists and `zip` them together.

We'll study `zip` in detail in the next part of this book; it's an iterable, so we must wrap it in a list call to show its results there, but its basic usage is otherwise straightforward:

```
>> list(zip(['a','b','c'], [1, 2, 3])) # Zip together keys and values  
[('a', 1), ('b', 2), ('c', 3)] >> D = dict(zip(['a',
```

```
b','c'], [1, 2, 3])) # Make a dict from zip result >> D {'a': 1, 'b': 2, 'c': 3}
```

You can achieve the same effect, though, with a dictionary comprehension expression. The following uses tuple assignment to unpack items into variables (per its Chapter 4 debut), as it scans the list of zipped pairs from left to right. The net effect builds a new dictionary with a key/value pair for every such pair in the zip result (the Python code reads almost the same as this natural-language description, but with a bit more formality):

```
>> D = {k: v for (k, v) in zip(['a','b','c'], [1, 2, 3])} >> D {'a': 1, 'b': 2, 'c': 3}
```

Comprehensions are longer to code in this case, but they're also more general than this example implies—we can use them to map a single stream of values to dictionaries and apply them to any kind of sequence (or other iterable), and collected keys can be computed with expressions just like collected values:

```
>> D = {x: x2 for x in [1, 2, 3, 4]} # Or: in range(1, 5)
>> D {1: 1, 2: 4, 3: 9, 4: 16} >> D = {c: c4 for c in 'HACK'} # Loop over any iterable
>> D {'H': 'HHHH', 'A': 'AAAA', 'C': 'CCCC', 'K': 'KKKK'} >> D = {c.lower(): (c + '!') for c in ['HACK', 'PY', 'CODE']} # Expr: expr
>> D {'hack': 'HACK!', 'py': 'PY!', 'code': 'CODE!'}
```

Dictionary comprehensions are also useful for initializing dictionaries from keys lists, in much the same way

ay as the `fromkeys` method we met at the end of the preceding section:

```
>> D = dict.fromkeys(['a','b','c'], 0) # Initialize dict fr
om keys >> D {'a': 0,'b': 0,'c': 0} >> D = {k: 0 for k in [
'a','b','c']} # Same, but with a comprehension >> D {'a
': 0,'b': 0,'c': 0} >> D = dict.fromkeys('code') # Other i
terables, default value >> D {'c': None,'o': None,'d': N
one,'e': None} >> D = {k: None for k in 'code'} >> D {'
c': None,'o': None,'d': None,'e': None}
```

Like its list and set relatives, dictionary comprehensions support additional syntax not shown here, including if clauses to filter values out of results, and nested for loops. Unfortunately, to truly understand dictionary comprehensions, you need to also know more about iteration statements and concepts in Python, and this book hasn't yet disclosed enough information to tell that story well.

You'll learn much more about all flavors of comprehensions—list, set, dictionary, and generator—in Chapters 14 and 20, so we'll defer further details until later.

中文翻译

与我们之前见过的集合 (set) 和列表推导式一样，字典推导式运行一个隐含的循环，在每次迭代中收集表达式的键/值结果，并用它们填充一个新的字典。循环变量让推导式能沿途使用循环迭代表达式的值。净效果是：我们可以用少量的代码创建新字典，比本书后面将研究的那种完整的语句形

式更简洁。抽象地说，字典推导式可映射为如下的 for 循环，其中 k 和 v 都可以使用循环变量 x：`{k: v for x in iterable}` # 字典推导式 `new = {}` # 等价的循环代码 `for x in iterable: new[k] = v` 为演示起见，动态初始化字典的标准方式是把键和值用 `zip` 组合起来，并把结果传给 `dict` 调用（按上一节所述）。`zip` 内置函数正是让我们能这样从键列表和值列表构建字典的“钩子”——如果你无法在代码中预知键和值的集合，可以先构建成列表再 `zip` 起来。我们会在本书下一部分详细研究 `zip`；它是可迭代对象，所以在那里展示结果时必须用 `list` 调用包装它，不过它的基本用法是很直白的：

```
>>> list(zip(['a','b','c'], [1, 2, 3])) # 把键和值 zip 在一起
[('a', 1), ('b', 2), ('c', 3)]
>>> D = dict(zip(['a','b','c'], [1, 2, 3])) # 从 zip 结果构造字典
>>> D {'a': 1, 'b': 2, 'c': 3}
```

不过，你也可以用字典推导式表达式达到同样的效果。下面的代码在从左到右扫描 `zip` 出的键值对列表时，使用元组赋值把项解包到变量中（这是它第 4 章的首次亮相）。净效果是为 `zip` 结果中的每一对都构建一个新的键/值字典（Python 代码读起来几乎和这段自然语言描述一样，只是更正式一点）：

```
>>> D = {k: v for (k, v) in zip(['a','b','c'], [1, 2, 3])}
>>> D {'a': 1, 'b': 2, 'c': 3}
```

在这个例子里推导式写起来更长，但它们比这个例子所暗示的更通用——我们可以用它把“单一的值流”映射成字典，也可以应用于任何种类的序列（或其他可迭代对象），而且收集到的键也像收集到的值一样可以用表达式计算：

```
>>> D = {x: x2 for x in [1, 2, 3, 4]} # 或: in range(1, 5)
>>> D {1: 1, 2: 4, 3: 9, 4: 16}
>>> D = {c: c4 for c in 'HACK'} # 遍历任何可迭代对象
>>> D {'H': 'HHHH', 'A': 'AAAA', 'C': 'CCCC', 'K': 'KKKK'}
>>> D = {c.lower(): (c + '!') for c in ['HACK', 'PY', 'CODE']}
```

']}] # 表达式: 表达式>>> D {'hack':'HACK!','py':'PY!','code':'CODE!'} 字典推导式对于从键列表初始化字典也很有用, 方式与上一节末尾遇到的 fromkeys 方法非常相似: >>> D = dict.fromkeys(['a','b','c'], 0) # 从键列表初始化字典>>> D {'a': 0,'b': 0,'c': 0} >>> D = {k: 0 for k in ['a','b','c']} # 一样, 但用推导式>>> D {'a': 0,'b': 0,'c': 0} >>> D = dict.fromkeys('code') # 其他可迭代对象, 默认值>>> D {'c': None,'o': None,'d': None,'e': None} >>> D = {k: None for k in 'code'} >>> D {'c': None,'o': None,'d': None,'e': None} 与它的列表和集合近亲一样, 字典推导式还支持这里未展示的额外语法, 包括用于从结果中过滤值的 if 子句和嵌套的 for 循环。遗憾的是, 要真正理解字典推导式, 你还需要更多关于 Python 迭代语句和概念的知识, 而本书目前还没透露足够的信息来把这个故事讲好。你会在第 14 章和第 20 章学到所有推导式变体——列表、集合、字典和生成器——的大量内容, 所以我们把更多细节推迟到后面。

深度理解

·核心概念: 字典推导式 = zip + dict 的一般化——它允许"键"和"值"都是任意表达式, 是"把一批数据变换成键值映射"的一行式表达。

·底层实现: {k: v for ...} 编译为独立的推导式代码对象, 内部用 BUILDMAP + MAPADD 字节码 (MAPADD 直接做哈希插入, 比普通推导式里的 D[k]=v (两次名字查找) 更快)。与列表推导式一样, 它独立作用域, 循环变量不会泄漏到外层。

·设计原因：文档/哲学上 dict comprehension 让"构建映射"这一高频动作从"先初始化再赋值"的三行代码压缩为一行，且可读性不降反升——因为你一眼看到"键=什么、值=什么"的规则。

·实际问题：{v: k for k, v in somedict.items()} 是"字典反转键值"的一行式；{x: f(x) for x in data} 是"MVP 预处理"；{k: 0 for k in keys} 等价 fromkeys 但运行时可对 k 做变换。

·初学者误区：① 逗号位置——列表推导式一个表达式、字典推导式键:值一对，忘写冒号就语法错；② 同键覆盖：循环中不同 x 算出相同键时，后算出的覆盖先算出的（如 {c.lower(): c for c in [...]} 中两个键名相同则后者胜）；③ 想同时"改原字典"——推导式永远建立新的字典。

代码分析

```
list(zip(['a', 'b', 'c'], [1, 2, 3]))          # →
D = dict(zip(['a', 'b', 'c'], [1, 2, 3]))      # zip + dict
D = {k: v for (k, v) in zip(['a', 'b', 'c'], [1, 2, 3])}  #

D = {x: x ** 2 for x in [1, 2, 3, 4]}          # ==
D = {c: c * 4 for c in 'HACK'}                 #
D = {c.lower(): (c + '!') for c in ['HACK', 'PY', 'CODE']} #

D = dict.fromkeys(['a', 'b', 'c'], 0)          # {'a':0, 'b':0, 'c...
D = {k: 0 for k in ['a', 'b', 'c']}            #
```

·解释器如何处理：{x: x² for x in [1,2,3,4]} 被编译为：创建空字典 → 对 [1,2,3,4] 取迭代器 → 循环 FORITER: 取 x → 计算 x² → MAPADD (该字节码一个时钟周期完成"以 x 为键、结果为目标"的插入)。迭代器在循环结束时

自动被引用计数回收。生成新字典，原可迭代对象不受影响。

·设计思想：三种推导式 (list/set/dict) 彼此只是"收集容器"不同——[...]、 {...}、 {k:v ...}。语法一致性让 Python 程序员"学会一种，通三种"；等到第 14 章把括号换成圆括号，就得到生成器 (generator) ——一个统一的家族。

8.5.7 键的插入顺序 (Key Insertion Ordering)

英文原文

By now, you've probably noticed that dictionaries remember their keys' order—no matter what sort of syntax we use to make them. As noted both earlier here and in Chapter 4, as of Python 3.7 (and CPython 3.6), dictionaries have shed their former random key order and adopted insertion order instead: keys are now ordered left to right from oldest to newest additions. This order is maintained for literals, dict calls, comprehensions, and new keys added on the fly.

Among other things, insertion order makes dictionary outputs arguably more coherent and readable and may avoid key sorts in some programs. This isn't the same as a list (e.g., there's no way to add a key in the "middle") and doesn't make dictionaries sequences. The effect, however, is close. Here's a brief illustration:


```
>> D = dict(a=1, b=2) # Literal keys stored left-to-right
>> D['c'] = 3 # New keys always added at the end
>> D {'a': 1, 'b': 2, 'c': 3}
>> D.pop('b') # Method pop removes any key by name
2
>> D # Other keys still in insertion order {'a': 1, 'c': 3}
>> D['b'] = 2 # Earlier key goes at the end too
>> D {'a': 1, 'c': 3, 'b': 2}
>> D['c'] = 0 # Changing values doesn't impact order
>> D['d'] = 1 # And new arrivals always go at the end
>> D {'a': 1, 'c': 0, 'b': 2, 'd': 1}
```

If you stare at this long enough, you'll probably notice that the insertion order of dictionaries is much like the LIFO stack ordering we coded with list append and pop methods earlier. In fact, the dictionary popitem is defined to return the key/value pair (really, tuple) for the key added most recently (i.e., at the end):

```
>> D.popitem() ('d', 1)
>> D.popitem() ('b', 2)
>> D {'a': 1, 'c': 0}
```

While useful in some contexts, keep in mind that you may still need to sort dictionary keys manually, using techniques we'll study ahead. Ordering filenames for display, for example, will often warrant string-value sorts instead of a program's internal insertion order.

中文翻译

到这一步，你可能已经注意到字典会记住它们键的顺序——无论我们用什么语法创建它们都如此。正如前面以及第 4

章提到的，从 Python 3.7（以及 CPython 3.6）开始，字典抛弃了以前的随机键顺序，采用插入顺序（insertion order）：键现在按照从最旧到最新的添加顺序从左到右排列。这个顺序对字面量、dict 调用、推导式以及运行中新添加的键都一视同仁地维护。除了别的方面，插入顺序让字典输出可以说更连贯、更易读，也可能让某些程序免去按键排序。这不同于列表（比如，没有办法把键加到“中间”），也不会让字典变成序列。但效果上很接近。这里做一个简短的演示：

```
>>> D = dict(a=1, b=2) # 字面量键从左到右存储>>> D['c'] = 3 # 新键总是加在末尾>>> D {'a': 1, 'b': 2, 'c': 3}>>> D.pop('b') # pop 方法按名字删除任意键2>>> D # 其他键仍保持插入顺序{'a': 1, 'c': 3}>>> D['b'] = 2 # 早先的键重插也会跑到末尾>>> D {'a': 1, 'c': 3, 'b': 2}>>> D['c'] = 0 # 改值不影响顺序>>> D['d'] = 1 # 新来的键总在末尾>>> D {'a': 1, 'c': 0, 'b': 2, 'd': 1}
```

如果你盯着这段代码看足够久，很可能会发现字典的插入顺序很像我们之前用列表 append 和 pop 方法写过的 LIFO 栈（stack）顺序。事实上，字典的 popitem 被定义为返回最近添加的那个键的键值对（实际上是元组）（即在末尾的那个）：

```
>>> D.popitem() ('d', 1)>>> D.popitem() ('b', 2)>>> D {'a': 1, 'c': 0}
```

虽然在某些场景下有用，但请记住，你可能仍需要手动排序字典的键，所用的技巧我们后面会学。例如，为显示而对文件名排序，常常需要按字符串值排序，而不是按程序内部的插入顺序。

深度理解

·核心概念：插入有序（insertion order）是 3.7 的语言级保证（3.6 在 CPython 中实现）。规律记住三条：新键

排末尾、改值不动序、删除不影响其他键的相对顺序（被删键重插会回到末尾）。

·底层实现：这是 compact dict 的直接产物。在古老版本（3.5 及更早）里，dict 直接把键值对放进哈希槽，槽位置由哈希决定，迭代顺序 = 槽内存布局 = 伪随机。3.6 的 compact dict 把"索引表 (indices)"与"条目表 (entries)"分离：插入的条目按先后顺序追加进 entries 数组，索引表只记录"某个哈希槽对应 entries 的下标"。迭代两条表取 entries 的物理顺序 → 恰好等于插入顺序。因为条目追加到数组末尾是 $O(1)$ 摊还，"popitem 弹末尾、 $O(1)$ "也就顺理成章。

·设计原因：老 dict 的顺序随机化本是哈希表正常现象，但无数程序其实需要"稳定顺序"（输出可复现、LRU 缓存语义、日志打印顺序）。与其让 API 修饰成 OrderedDict，不如直接把基础 dict 做成有序，既省一点内存（紧凑布局本就少 ~20% 内存）又消灭一大类 bug。

·实际问题：现在 collections.OrderedDict 几乎只剩"move_to_end"等额外操作才需要；普通"按加入顺序遍历数据"（如配置文件解析顺序、API 参数顺序）直接靠 dict。但注意它不是序列：没有下标索引、没有切片、不能"插到中间"。

·初学者误区：① 仍套用旧观念"dict 无序"（读老教程抓狂）；② 认为插入有序 = 排序——它是"插入先后序"，不是"键的大小序"，想要后者必须 sorted(D)；③ 以为 popitem 是"随机弹一个"——3.7 后是确定性的"弹最后添加的"。

代码分析

```
D = dict(a=1, b=2)      # a → b
D['c'] = 3               # a → b → c
D.pop('b')              # b a → c
D['b'] = 2               # b → a → c → b
D['c'] = 0               # c
D['d'] = 1               # a → c → b → d
D.popitem()             # ('d', 1)
D.popitem()             # ('b', 2)
D                       # {'a': 1, 'c': 0}
```

·解释器如何处理：D['b'] = 2 在 b 已被删除、键不存在时走插入路径：在 entries 数组末尾追加新条目（O(1) 摊还），并在索引表登记哈希位置——所以尽管键 b 历史上先于 c 存在，此时它在 entries 中的位置却靠后，迭代时排在 c 之后。而 D['c'] = 0 走更新路径：找到旧条目原地替换 value 引用，不移动条目，顺序不变。popitem 直接取 entries 数组最后一个条目并收缩。

·设计思想："按插入先后记账"天然契合"目录/日志/缓存"类应用的语义；同时用"entries 追加"代替"哈希散布"让哈希与顺序两个概念解耦，堪称 Python 3.6 最优雅的一次内部革命。

8.5.8 字典"合并"运算符 (Dictionary "Union" Operator)

英文原文

Last up in the dictionary toolbox is a bit of an oddball: as of Python 3.9, dictionaries have sprouted a "unio

n" operation that's kicked off with the | operator—the same syntax used for union in the set objects you met in Chapter 5.

Per the quotes, though, this operation isn't really mathematical set union at all. It's a positionally sensitive, key-based merge of dictionaries, that works the same as the dictionary update method but returns its result as a new dictionary instead of updating a subject in place. Hence, dictionary | is like the combination of running the dictionary's copy and update methods in turn, and really just a minor convenience for narrow roles. Here it is at work:

```
>> D = dict(a=1, b=2) >> D {'a': 1, 'b': 2} >> D | {'b': 3, 'c': 4} # Update, return, rightmost wins {'a': 1, 'b': 3, 'c': 4} >> D | {'b': 3, 'c': 4} | dict(a=5, d=6) {'a': 5, 'b': 3, 'c': 4, 'd': 6}
```

You get roughly the same mileage with the dictionary's longstanding update method, though it requires a manual copy to avoid in-place changes in programs that don't want them (as a spoiler, the |= in-place assignment form of dictionary union that you'll meet later in this book is identical to calling update with a single argument sans copy):

```
>> D {'a': 1, 'b': 2} >> C = D.copy() >> C.update({'b': 3, 'c': 4}) # "Union" sans the | expression >> C.update(dict(a=5, d=6)) >> C {'a': 5, 'b': 3, 'c': 4, 'd': 6}
```

The Python 3.5 dictionary-literal unpacking we explored earlier can have the same effect too, though, at least when used with gnarly embedded literals as in the following, perhaps less readably:

```
>> D {'a': 1, 'b': 2} >> {D, {'b': 3, 'c': 4}, dict(a=5, d=6)}  
{'a': 5, 'b': 3, 'c': 4, 'd': 6}
```

Which—true sports fans will note—raises the number of dedicated dictionary merge operations from one to three since this book's prior edition. This bloat is especially grievous, given that it takes just one line of simple code to accomplish what the update method, the newer unpacking, and the newest | union operator all redundantly do in their main use cases:

```
>> D1 = dict(a=1, b=1) >> D2 = dict(a=2, c=2) >> f  
or k in D2: D1[k] = D2[k] # What update(), , and | red  
undantly do >> D1 # Did this really justify 3 alternati  
ves? {'a': 2, 'b': 1, 'c': 2}
```

Dictionary | "union" may be handy in limited contexts. But whether this justifies convoluting dictionaries with just one binary operator, just one of many set operations, and an operation that's almost entirely redundant with two earlier tools that were already almost entirely redundant with very simple code, is a riddle left to the reader to solve.

中文翻译

字典工具箱里最后登场的是一匹“怪马”：从 Python 3.9 起，字典长出了一个用 `|` 运算符启动的“合并（union）”操作——正是你在第 5 章见过的集合（set）对象所用合并的同一种语法。不过按这里的引号所言，这个操作其实根本不是什么数学意义上的集合并。它是对字典的一种对位置敏感的、基于键的合并（merge），工作方式与字典 update 方法相同，但把结果作为新字典返回，而不是就地更新主体。因此，字典 `|` 就像是依次执行字典的 copy 和 update 方法，本质上只是为狭窄场景提供的一个小便利。它实际运行如下：

```
>>> D = dict(a=1, b=2) >>> D {'a': 1, 'b': 2} >>>
D | {'b': 3, 'c': 4} # 更新后返回，最右侧胜出{'a': 1, 'b': 3, 'c': 4}
>>> D | {'b': 3, 'c': 4} | dict(a=5, d=6) {'a': 5, 'b': 3, 'c': 4, 'd': 6}
```

字典由来已久的 update 方法也能达到大致相同的效果，只是在不希望就地修改的程序里，它需要手动 copy 一份（剧透：本书后面会讲到的字典合并的就地赋值形式 `|=`，与用单个参数调用 update（不 copy）完全等价）：

```
>>> D {'a': 1, 'b': 2} >>> C = D.copy() >>> C.update({'b': 3, 'c': 4})
# 不用 | 表达式的“合并” >>> C.update(dict(a=5, d=6)) >>> C {'a': 5, 'b': 3, 'c': 4, 'd': 6}
```

我们前面探索过的 Python 3.5 字典字面量解包也能产生同样效果——至少在像下面这样使用复杂的嵌套字面量时可以，尽管可读性或许差一些：

```
>>> D {'a': 1, 'b': 2} >>> {D, {'b': 3, 'c': 4}, dict(a=5, d=6)} {'a': 5, 'b': 3, 'c': 4, 'd': 6}
```

——真正的体育迷会注意到——这使专门的字典合并操作从此书上一版起，从一个增加到了三个。这种臃肿尤其令人痛心，因为用一行简单的代码就能做到 update 方法、较新的解包和最

年轻的 | 合并运算符在主要用途上冗余地做的事：>>> D1 = dict(a=1, b=1) >>> D2 = dict(a=2, c=2) >>> for k in D2: D1[k] = D2[k] # update()、和 | 冗余地做的事>>> D1 # 这真的值得三个替代方案吗？{'a': 2, 'b': 1, 'c': 2} 字典 | "合并"在有限的场景下可能很方便。但这是否足以证明：给字典卷进一个二元运算符——还是众多集合运算中的那一个、一个几乎与两个更早工具完全冗余、而那两个更早工具又几乎与极其简单的代码完全冗余——这个谜题留给读者去解答。

深度理解

- 核心概念：| 是 copy + update 的语法糖 (3.9+)：返回新字典，右侧键胜出；|= 等价于 update。它与解包 (3.5+)、update 方法 (远古) 构成"三个等价物"。
- 底层实现：D | D2 编译为 BINARYOP (操作码调度器按左操作数的类型查找"合并"槽位)。实现 dictor：先 PyDictCopy(D)，再对 D2 执行批量插入 (复用 dictmerge)。因此它比 update 多一次 O(n) 的复制开销，在"纯合并"场景下与字面量解包速度相当。
- 设计原因 (含作者吐槽)：Mark Lutz 对 | 的态度微妙——他指出三个 API 功能冗余，一行 for 循环就搞定一切。这背后其实是 Python 社区"多路径达成同一目标"式的演化史：核心语法在回应不同时期的需求，有时就会累积冗余。| 的优势：表达式而非语句，可链式、可放在表达式里 (如函数实参内的联合并)。
- 实际问题：config = defaults | overrides | user 的链式合并、state = base | {'page': n} 的"复制并加一键" (避

免副作用) 都是 | 的主场; 老代码里习惯 `d = {a, b}` 或 `d = a.copy(); d.update(b)`。

·初学者误区: ① 把 | 当成集合运算——它按键合并、不是集合并; ② 忘了 | 不改原字典 (想要就地用 |=); ③ 在 3.9 以下环境用 | 直接 `SyntaxError` (版本兼容陷阱)

。

代码分析

```
D = dict(a=1, b=2)
D | {'b': 3, 'c': 4}                # {'a':1,'b':3,'c':4}...
D | {'b': 3, 'c': 4} | dict(a=5, d=6)  # → {'a':5,'b':3,...

C = D.copy()                        # |
C.update({'b': 3, 'c': 4})           # copy + update
C.update(dict(a=5, d=6))

{**D, **{'b': 3, 'c': 4}, **dict(a=5, d=6)}  # **

D1 = dict(a=1, b=1); D2 = dict(a=2, c=2)
for k in D2: D1[k] = D2[k]           #
D1                                    # {'a': 2, 'b': 1, 'c': 2}
```

·解释器如何处理: `D | {'b': 3,'c': 4}`: ① 字节码 `BINARY OP |` 触发, 解释器按左操作数 `dict` 的类型查找"合并"槽位 (`dictor` 实现, 3.9 加入); ② `dictor` 先调用 `PyDictCopy` 复制 `D` (新字典), 再循环把右侧条目插入新字典——遇到已存在的键直接覆盖; ③ 返回新字典。因此原 `D` 完全不受影响。而 `{D, ...}` 走 `BUILDMAPUNPACK`, 本质同样"建新字典+顺序插入"。三者都是"顺序写入、后者覆盖", 所以"右侧胜出"的规则到处一致。

·设计思想：作者的灵魂拷问值得深思：语法工具是"少而精"还是"多而全"？Python 的真实答案是两者兼有——核心语义最小化，但表达手段随社区需求不断增殖。对学习者而言，选一个惯用法（如或 |）用熟即可，不必追求全部掌握。

8.5.9 插曲：图书数据库 (Intermission: Books Database)

英文原文

Let's take a break from the fine points and code a more concrete example of dictionaries at work. In honor of this book's quarter-century milestone, the following builds a simple in-memory editions database that maps edition years (the keys) to titles (the values). As coded, you fetch edition names by indexing on year strings:

```
>> table = {'2024': 'Learning Python, 6th Edition', # Year => title
```

```
...'2013': 'Learning Python, 5th Edition', ...'1999': 'Learning Python'} >> table['2024'] # Key => Value 'Learning Python, 6th Edition' >> for year in table: # Keys iteration
```

```
... print(year + '\t' + table[year]) 2024 Learning Python , 6th Edition 2013 Learning Python, 5th Edition 1999 Learning Python
```

The last command uses a for loop, which we've used several times since its Chapter 4 preview. The full tale of the for isn't told until Chapter 13, but this particular loop simply iterates through each key in the table to print a tab-separated list of keys and their values (recall from Chapter 7 that `\t` in a Python string means vertical tab).

Dictionaries aren't sequences like lists and strings, but if you need to step through the items in a dictionary, it's easy—either call the dictionary `keys` method or use the dictionary itself. Given a dictionary `D`, saying `for key in D` works the same as saying `for key in D.keys()`, and both use the iteration protocol that we'll expand on later in this book. Either way, you can index from key to value inside the for loop as you go, as this code does.

Mapping values to keys Notice how the prior table maps years to titles, but not vice versa. If you want to map the other way—titles to years—you can either code the dictionary differently:

```
>> table = {'Learning Python, 6th Edition': 2024, # Title => year
...         'Learning Python, 5th Edition': 2013, ... 'Learning Python': 1999}
>> table['Learning Python'] # Key => value
1999
```

Or use other dictionary methods like `items` that give searchable sequences (the list call is required in the f

ollowing because `items` returns an iterable view, a topic coming up shortly):

```
>> list(table.items())[2] [('Learning Python, 6th Edition', 2024), ('Learning Python, 5th Edition', 2013)] >>
[title for (title, year) in table.items() if year == 1999]
# Value => key ['Learning Python']
```

The last command here uses the list comprehension we explored earlier, as well as tuple assignment mentioned earlier and covered in this book's next part (synopsis: it unpacks items into variables). The combo scans the (key, value) pairs returned by the dictionary's `items` method, selecting keys for values matching a search target (1999).

The net effect of all this is to index backward—from value to key, instead of key to value. This is useful if you want to store data just once and map backward from values only rarely (searching through sequences like this is generally much slower than a direct key-to-value index, though not all programs need to care).

In fact, although dictionaries by nature map keys to values unidirectionally, there are multiple ways to map values back to keys with a bit of extra generalizable code:

```
>> K = 'Learning Python' >> V = 1999 >> table[K] #
Key => Value (normal usage) 1999 >> [key for (key, value) in table.items() if value == V] # Value => Key ['Learning Python'] >> [key for key in table.keys() if ta
```

```
ble[key] == V] # Same: keys() ['Learning Python']
```

Note that both of the last two commands return a list of titles: in dictionaries, there's just one value per key, but there may be many keys per value if a given value is stored under multiple keys, and a value might be a collection itself to represent many values per key. It's also possible to invert a dictionary for indexing values by zipping its values and keys—but only if its values are all immutable and don't appear in multiple keys:

```
>> dict(zip(table.values(), table.keys()))[1999] # Key/  
value inversion? 'Learning Python'
```

Sharp-eyed readers may notice that this yields a dictionary with integer keys, which works naturally per a tip in the next section. For more on this front, watch for a less lossy dictionary inversion function in the `mapattrs.py` example from "Example: Mapping Attributes to Inheritance Sources"—code that would surely stretch this preview past its breaking point if included here.

For this chapter's purposes, let's wrap up by adding in a few pragmatic pieces to the dictionary puzzle.

中文翻译

让我们从细节里休息一下，写一个更具体的字典应用示例。为了纪念本书诞生二十五周年，下面的代码构建了一个简单的内存版次数据库：把版年份（键）映射到书名（值）。如

代码所示，你通过用年份字符串索引来取出版本名：>>>

```
table = {'2024': 'Learning Python, 6th Edition', # 年份
=> 书名...'2013': 'Learning Python, 5th Edition', ...'1999': 'Learning Python'}>>> table['2024'] # 键 => 值'Learning Python, 6th Edition'>>> for year in table: # 键迭代... print(year + '\t' + table[year]) 2024 Learning Python, 6th Edition 2013 Learning Python, 5th Edition 1999 Learning Python
```

最后一个命令用到了 for 循环，自第 4 章预览以来我们已经用过好几次。for 的完整故事要到第 13 章才讲，但这个循环只是简单遍历表中的每个键，打印一份用制表符分隔的键值列表（回想第 7 章：Python 字符串中的 \t 表示制表符 tab）。字典不像列表和字符串那样是序列，但如果你需要遍历字典中的条目，也很容易——要么调用字典的 keys 方法，要么直接用字典本身。给定字典 D，写 for key in D 与写 for key in D.keys() 效果相同，两者都使用本书后面会展开讲解的迭代协议（iteration protocol）。无论哪种方式，你都可以在 for 循环体内一边遍历一边用键索引取值，就像这段代码做的那样。从值映射回键（Mapping values to keys）注意前面的表把年份映射到书名，但反过来不行。如果你想反向映射——从书名到年份——你可以换一种方式编码字典：>>>

```
table = {'Learning Python, 6th Edition': 2024, # 书名 => 年份...'Learning Python, 5th Edition': 2013, ...'Learning Python': 1999}>>> table['Learning Python'] # 键 => 值1999 或者用 items 等其他字典方法，它们给出可搜索的序列（下面用 list 调用是必需的，因为 items 返回可迭代的视图（view），这个话题马上就来）：>>> list(table.items())[2][('Learning Python, 6th Edition', 2024), ('Learning Python, 5th Edition', 2013), ('Learning Python', 1999)]
```

hon, 5th Edition', 2013)] >>> [title for (title, year) in table.items() if year == 1999] # 值 => 键['Learning Python'] 这里的最后一个命令用到了我们之前探索过的列表推导式，以及之前提过、本书下一部分会详细讲的元组赋值（tuple assignment）（摘要：它把条目解包到变量里）。这个组合扫描字典 items 方法返回的（键，值）对，挑选出值匹配搜索目标（1999）的键。这一切的净效果就是反向索引（index backward）——从值到键，而不是从键到值。如果你希望数据只存一份、只是偶尔需要从值反向映射，这很有用（像这样在序列中搜索一般比直接的键到值索引慢得多，不过并非所有程序都需要在意）。事实上，尽管字典天生是单向的“键→值”映射，但用一点点可泛化的代码，有不止一种方式把值映射回键：>>> K = 'Learning Python'>>> V = 1999>>> table[K] # 键 => 值（正常用法）1999>>> [key for (key, value) in table.items() if value == V] # 值 => 键['Learning Python']>>> [key for key in table.keys() if table[key] == V] # 一样：keys()['Learning Python'] 注意最后两条命令都返回一个列表：在字典里，每个键只有一个值，但如果同一个值存放在多个键名下，那么每个值可能对应多个键；而一个值本身也可能是一个集合，用来表达“每个键对应多个值”。还可以通过把值和键 zip 起来反转字典以实现“按值索引”——但仅当它的值全部不可变且不在多个键下出现时才可行：>>> dict(zip(table.values(), table.keys()))[1999] # 键/值反转？'Learning Python'眼尖的读者可能注意到，这产生了一个带整数键的字典，根据下一节的提示，这可以自然工作。关于这个方向还有更多内容：请留意 mapattrs.py 示例（来自“示例：把属性映射到继承来源”）中一个更少信息损失的字典反

转函数——如果在这里展开它，肯定会把这篇预览撑到极限。就本章而言，让我们往字典谜题里再添几块实用拼图，然后收尾。

深度理解

·核心概念：字典是单向映射（键→值 $O(1)$ ），反向查找（值→键）要么换种存法，要么 $O(n)$ 线性扫描——两种方案各有取舍。这是“用数据建模思想”的第一次实战演练。

·底层实现：for year in table 走 dict 的 iter——它返回的是键迭代器（dictkeyiterator），因此 for k in D $\equiv$ for k in D.keys()。反向扫描 [key for ... if value == V] 是 $O(n)$ 逐项比较，而 table[K] 是 $O(1)$ 哈希查找——相差一个数量级。

·设计原因：为什么不提供“反向查找”API？因为值不唯一（多键可共享同值），“值→键”的结果是列表而非单值——这超出“映射”的最小契约。反过来也印证了“一个值可挂多个键”的数据现实。

·实际问题：数据库表（如 SQLite 的表单行）其实就是“记录集合”；dict(zip(values, keys)) 反转法有前提条件（值必须不可变、唯一），否则会丢数据——生产代码宁可显式扫描或维护第二张索引字典。

·初学者误区：① 把“正向查”的习惯直接套到反向，以为有 table[1999] 这种魔法；② 忘了值可能不唯一，结果只取到“第一个匹配”的键；③ dict(zip(vals, keys)) 在值含可变对象（如列表）时直接 TypeError。

代码分析

```
table = {'2024': 'Learning Python, 6th Edition',
         '2013': 'Learning Python, 5th Edition',
         '1999': 'Learning Python'}
table['2024']                                # => 0(1)
for year in table:                            # table.keys()
    print(year + '\t' + table[year])

# => 0(n)
[title for (title, year) in table.items() if year == 1999]
                                     # ['Learning Python']
[key for key in table.keys() if table[key] == V]    #

#
dict(zip(table.values(), table.keys()))[1999]      # 'Learning Pyt...
```

·解释器如何处理：for year in table 时，解释器调用 Py ObjectGetIter(table) → 得到 dictkeyiterator 对象；随后每次迭代调用其 tpiternext：从 entries 数组当前位置取出下一个键，递增位置直到数组末尾抛出 StopIteration 终止循环。table.items() 返回的 dictitemiterator 每次产出 (键, 值) 二元元组——推导式里的 (title, year) 元组解包就是消费这个元组。dict(zip(table.values(), table.keys())) 中，zip 先取全部值、再取全部键，两两配成对喂给 dict 构造器——注意当某值出现两次时，后一个键会覆盖前一个，这就是“信息丢失”风险所在。

·设计思想：“先存一次、反向偶尔查” vs “建双份索引”是经典时空权衡；本例教育我们要根据访问模式设计数据结构，而不是迷信单一工具。

8.5.10 字典使用技巧 (Dictionary Usage Tips)

英文原文

Dictionaries are fairly straightforward tools once you get the hang of them, but here are a few additional pointers and reminders you should be aware of when using them: Sequence operations don't work. Dictionaries are mappings, not sequences. Because they deal with keys and values and are ordered by insertion-time only, things like concatenation (an ordered joining of values) and slicing (extracting a contiguous section of values) simply don't apply—and Python raises an error if your code tries to use them on dictionaries. Assigning to new indexes adds entries. Keys can be created when you write a dictionary literal (embedded in the code of the literal itself), or when you assign values to new keys of an existing dictionary object individually. The end result is the same. Keys need not always be strings. Our examples so far have used strings as keys, but any other immutable objects work just as well. For instance, you can use integers as keys, which makes the dictionary look much like a list (when indexing, at least). Tuples may be used as dictionary keys too, allowing compound key values—such as dates and IP addresses—to have associated values. User-defined class instance objects (discussed in Part VI) can also be used as keys, as long as they define the proper methods; roughly, they need to tell Python that

their values are "hashable" and thus won't change, as otherwise they would be useless as fixed keys. Mutable objects such as lists, sets, and other dictionaries don't work as keys because they may change, but are allowed as values. The following sections delve deeper into these and other mysteries of dictionary-processing code.

中文翻译

一旦上手，字典是非常直接的工具，但这里还有一些额外的提示和提醒，你在使用它们时应该注意：序列操作不适用（Sequence operations don't work）字典是映射，不是序列。因为它们处理的是键和值、且只按插入时间排序，诸如拼接（值的顺序连接）和切片（提取值的一段连续区间）这类操作根本不适用——如果你的代码试图在字典上使用它们，Python 会抛出错误。给新索引赋值会添加条目（Assigning to new indexes adds entries）键可以在写字典字面量时被创建（嵌在字面量自身的代码里），也可以在对现有字典对象的新键逐个赋值时被创建。最终结果是一样的。键不一定是字符串（Keys need not always be strings）到目前为止我们的例子都用字符串做键，但任何其他不可变（immutable）对象同样适用。例如，你可以用整数做键，这会让字典看起来很像列表（至少在索引时如此）。元组（tuple）也可以用作字典键，从而让复合键值——比如日期和 IP 地址——拥有关联的值。用户自定义的类实例对象（第 VI 部分讨论）只要定义了恰当的方法，也可以用作键；粗略地说，它们需要告诉 Python 它们的值是可“哈希（hashable）”的、不会改变，否则作为固定键就没有用处。

列表、集合和其他字典等可变对象不能作为键，因为它们可能改变，但可以作为值。

深度理解

·核心概念：三大纪律：① 字典禁拼接/切片；② 赋新键自动建条目；③ 键必须不可变（可哈希），值不限。

·底层实现：为什么键必须可哈希？因为哈希表的查找依赖“键的哈希值始终不变”。若键可变，先插入后改键，它的哈希值变化，查找时定位到错误的槽——表就“烂”了。整数、字符串、元组（含不可变项）可哈希；列表、集合、字典不可哈希（`hash = None`，试图当键直接 `TypeError: unhashable type: 'list'`）。

·设计原因：“值必须可变、键必须不可变”是哈希表的物理定律，不是语言任性。它同时解释了为什么“字典能当记录”（键=字段名）、“字典能当稀疏结构”（键=坐标元组）。

·实际问题：整数键 → 稀疏数组模拟；元组键 → 多维坐标、复合标识（日期、IP）；字符串键 → 绝大多数“命名数据”。

·初学者误区：① 试图把列表当键（报 `unhashable`）；② 以为键只能字符串；③ 用可变对象当键失败后“灵机一动”用其字符串表示——可以，但注意身份与内容语义的区别。

用字典模拟灵活列表：整数键 (Using dictionaries to simulate flexible lists: Integer keys)

英文原文

The last point in the preceding list is important enough to demonstrate with a few examples. When you use lists, it is illegal to assign to an offset that is off the end of the list:

```
>> L = [] >> L[99] = 'hack'
IndexError: list assignment index out of range
```

Although you can use repetition to preallocate as big a list as you'll need (e.g., [0]100), you can also do something that looks similar with dictionaries that does not require such space allocations—and potential waste. By using integer keys, dictionaries can emulate lists that seem to grow on offset assignment:

```
>> D = {} >> D[99] = 'hack' >> D[99]
'hack' >> D {99: 'hack'}
```

Here, it looks as if D is a 100-item list, but it's really a dictionary with a single entry; the value of the key 99 is the string 'hack'. You can access this structure with offsets much like a list, catching nonexistent keys with get or in tests if required, but you don't have to allocate space for all the positions to which you might need to assign values in the future. When used like this, dictionaries are like more flexible equivalents of li

sts:

```
>> D[62] = 'code'>> D[30] = 'write'>> D[62]'code'>>
D {99:'hack', 62:'code', 30:'write'}
```

As another example, we might also employ integer keys in the first book-database code we wrote earlier to avoid quoting the year, albeit at the expense of expressiveness (integer keys cannot contain nondigit characters):

```
>> table = {2024:'Learning Python, 6th Edition',
... .. etc ... >> table[2024]'Learning Python, 6th Edition'
```

中文翻译

上面列表中的最后一点重要到值得用几个例子演示。使用列表时，给超出列表末尾的偏移赋值是非法的：

```
>>> L = []
>>> L[99] = 'hack'IndexError: list assignment index out of range
```

虽然你可以用重复操作预分配一个足够大的列表（例如 `[0]100`），但你也可以用字典做类似的事，而且不需要这样的空间分配——也就不存在潜在浪费。通过使用整数键，字典可以模拟“在偏移赋值时似乎自己会长大”的列表：

```
>>> D = {}
>>> D[99] = 'hack'>>> D[99]'hack'
>>> D {99:'hack'}
```

这里，看起来 `D` 像一个 100 项的列表，但它其实是一个只有单个条目的字典；键 99 的值是字符串 `'hack'`。你可以像列表一样用偏移访问这个结构，如果需要还可以用 `get` 或 `in` 测试捕获不存在的键——但你不必为将来可能需要赋值的所有位置预先分配空间。这样用时，字

典就像更灵活的列表等价物：
`>>> D[62] = 'code'>>> D[30] = 'write'>>> D[62]['code']>>> D {99:'hack', 62:'code', 30:'write'}` 作为另一个例子，我们也可以在前面的图书数据库代码中使用整数键，省去给年份加引号，尽管牺牲了一些表现力（整数键不能包含非数字字符）：
`>>> table = {2024:'Learning Python, 6th Edition', ... etc ...>>> table[2024]'Learning Python, 6th Edition'`

深度理解

·核心概念：稀疏 vs 连续。列表是“连续必填”结构（索引 $0..n-1$ 全部占位，物理上连续分配），字典是“按需记账”结构（只存出现过的键）。用整数键的字典模拟“会长大的列表”，本质是牺牲顺序语义换取稀疏存储。

·底层实现：L[99] = 'hack' 时 listassubscript 检查 99 > = obsize，越界抛 IndexError——因为数组物理上只有 0 个槽。而 D[99] = 'hack' 走 dict 插入路径：99 的哈希值定位槽，直接在哈希表中登记——不需要“第 0..98 号”存在。内存只花在“实际存在的键”上。

·设计原因：这展示了哈希表相对数组的天然优势——分配与索引脱钩。数组必须先申请后使用，哈希表“用多少、占多少”。代价是：整数键字典失去切片、有序遍历（插入序）、连续下标运算（D[i+1] 必须手动）等序列能力。

·实际问题：游戏地图、稀疏矩阵、ID→对象缓存（用户 ID 不连续时）、按事件编号记录的日志——“键是整数但完全不连续”的场景，正是整数键字典的主场。

·初学者误区：① 误以为 D[99] 会让字典“有 100 个槽”——

一只有 1 个条目；② 想当然给字典做切片 D[0:5]——TypeError（它不是序列）；③ 把"整数键字典"与"列表"混为一谈，忘了字典的迭代顺序是插入顺序而非数值大小。

代码分析

```
L = []
L[99] = 'hack'           # IndexError 99

D = {}
D[99] = 'hack'           # OK
D[99]                     # 'hack'
D                         # {99: 'hack'}
D[62] = 'code'           #
D[30] = 'write'
D                         # {99: 'hack', 62: 'code', 30: 'write'}

table = {2024: 'Learning Python, 6th Edition'} #
table[2024]               # 'Learning Python, 6th Edition'
```

·解释器如何处理：D[99] = 'hack' 编译为 STORESUBSCR，解释器发现目标是 dict，调用 dictasssub：对键 99 调用 int.hash（Python 整数哈希 = 值本身（对 64 位平台模 $2^{61}-1$ 后的哈希），因此是常数时间），定位槽位，写入条目。没有"必须为 0..98 预留"的概念——哈希表只在插入时占用必要的槽。对比之下 L[99] = 'hack' 在 listassubscript 里检查 idx > PyListGETSIZE 立即抛 IndexError。

·设计思想："字典 = 更灵活的列表"这句总结妙在道出了两种容器的本质分野：列表承诺"连续"，字典承诺"存在"。按数据形态（连续 vs 稀疏）选择容器，是数据建模的第一课。

用字典实现稀疏数据结构：元组键 (Using dictionaries for sparse data structures: Tuple keys)

英文原文

In a similar way, dictionary keys are also commonly leveraged to implement sparse—mostly empty—data structures, such as multidimensional arrays where only a few positions have values stored in them:

```
>> Matrix = {} >> Matrix[(2, 3, 4)] = 88 >> Matrix[(7, 8, 9)] = 99 >> X = 2; Y = 3; Z = 4 # A; separates state  
ments: see Chapter 10 >> Matrix[(X, Y, Z)] 88 >> Mat  
rix {(2, 3, 4): 88, (7, 8, 9): 99}
```

Here, we've used a dictionary to represent a three-dimensional array that is empty except for the two positions (2,3,4) and (7,8,9). The keys are tuples that record the coordinates of nonempty slots. Rather than allocating a large and mostly empty three-dimensional matrix to hold these values, we can use a simple two-item dictionary. In this scheme, accessing an empty slot triggers a nonexistent key exception, as these slots are not physically stored:

```
>> Matrix[(2,3,6)] KeyError: (2, 3, 6)
```

Avoiding missing-key errors Errors for nonexistent key fetches like the foregoing are common in sparse matrixes, but you probably won't want them to shut down your program.

There are at least three ways to fill in a default value instead of getting such an error message—you can use the dictionary get method shown earlier to provide a default for keys that do not exist, test for keys ahead of time in if statements, or use a try statement to catch and recover from the error. Though straightforward, the last of these are previews of statement syntax we'll begin studying in Chapter 10:

```
>> Matrix.get((2, 3, 4), 0) # Exists: fetch and return 8
8 >> Matrix.get((2, 3, 6), 0) # Doesn't exist: use passed
    default 0 >> if (2, 3, 6) in Matrix: # Check for key before
    fetch
```

```
... print(Matrix[(2, 3, 6)]) ... else: # See Chapters 10 and
    12 for if/else ... print(0) ... 0 >> try:
```

```
... print(Matrix[(2, 3, 6)]) # Try to index ... except KeyE
    rror: # Catch and recover ... print(0) # See Chapters 1
    0 and 34 for try/except ... 0
```

Of these, the get method is the most concise in terms of coding requirements, but the if and try statements are much more general in scope—as you'll start seeing for yourself soon in Chapter 10.

中文翻译

与此类似，字典键也常被用来实现稀疏（sparse）——大部分为空的——数据结构，比如只有少数几个位置存有值的多维数组：

```
>>> Matrix = {} >>> Matrix[(2, 3, 4)] = 88 >>> Matrix[(7, 8, 9)] = 99 >>> X = 2; Y = 3; Z = 4
```

; 分隔语句：详见第 10 章>>> Matrix[(X, Y, Z)] 88 >>> Matrix {(2, 3, 4): 88, (7, 8, 9): 99} 这里，我们用字典表示一个三维数组，除 (2,3,4) 和 (7,8,9) 两个位置外全为空。键是记录非空槽坐标的元组。与其分配一个庞大且大部分为空的三维矩阵来存放这些值，不如用一个简单的两条目字典。在这种方案里，访问空槽会触发"键不存在"异常，因为这些槽根本没有被物理存储：>>> Matrix[(2,3,6)] KeyError: (2, 3, 6) 避免缺键错误 (Avoiding missing-key errors) 像上面这样"取不存在的键"的错误在稀疏矩阵中很常见，但你可能不希望它们让程序崩溃。至少有三种方法可以填充默认值而不是得到错误消息——你可以用前面展示的字典 get 方法为不存在的键提供默认值，在 if 语句中提前测试键是否存在，或者用 try 语句捕获并恢复错误。最后这两种虽然直接，但都是第 10 章才开始学习的语句语法的预告：
>>> Matrix.get((2, 3, 4), 0) # 存在：取出并返回88
>>> Matrix.get((2, 3, 6), 0) # 不存在：使用传入的默认值0
>>> if (2, 3, 6) in Matrix: # 取出前先检查键是否存在... print(Matrix[(2, 3, 6)]) ... else: # if/else 详见第 10、12 章.. print(0) ... 0
>>> try: ... print(Matrix[(2, 3, 6)]) # 尝试索引... except KeyError: # 捕获并恢复... print(0) # try/except 详见第 10、34 章... 0 其中，就编码量而言 get 方法最简洁，但 if 和 try 语句的适用范围要广泛得多——第 10 章你就会亲眼看到。

深度理解

·核心概念：元组键 = 复合坐标。三维矩阵 (x, y, z) 直接作为键，是"稀疏多维数组"最优雅的 Python 表达。防缺键三大法：get（最简）、if in（预检）、try/except（异

常处理，最通用）。

·底层实现：元组能当键因为元组可哈希（若其元素全部可哈希）：`tuple.hash` 会把各元素的哈希值按固定公式组合 $((a1000003 \wedge b) \dots)$ ，得到元组级哈希，再走正常的哈希表插入。`Matrix.get((2,3,6), 0)`：查不到键就返回 0，不抛异常。`if in` 与 `try` 两条路径的差别在于——`try` 先尝试再兜底，`if` 先询问再行动（Look Before You Leap vs Easier to Ask Forgiveness，后文第 10 章的主题）。

·设计原因："稀疏"问题的本质是绝大多数坐标为 0 或空——为它们分配内存是纯浪费。字典按需存储，把"维度乘积"的内存需求压缩成"非空条目数"。这也是科学计算里稀疏矩阵（如 `scipy.sparse`）背后思想的最简原型。

·实际问题：棋盘/地图状态（只有几步有棋子）、大型稀疏矩阵运算、按(坐标,频道)聚合的统计表——凡是"空间巨大但内容稀疏"的数据，都可以考虑"元组键字典"。同时，"取不到=0"的默认值语义让矩阵加法、累加器都能天然表达。

·初学者误区：① 忘了元组键必须全部元素可哈希（`(1, [2])` 当键报错）；② 访问空槽抛 `KeyError` 后以为程序完蛋——那是"没被存储"的正常语义；③ 试图对稀疏字典做"三维切片"——做不到，需要另写逻辑。

代码分析

```

Matrix = {}
Matrix[(2, 3, 4)] = 88          # (2,3,4)
Matrix[(7, 8, 9)] = 99
Matrix[(X, Y, Z)]              # 88 X,Y,Z
Matrix                        # {(2, 3, 4): 88, (7, 8, 9): 99}
Matrix[(2, 3, 6)]              # KeyError

#
Matrix.get((2, 3, 4), 0)       # 88
Matrix.get((2, 3, 6), 0)       # 0

if (2, 3, 6) in Matrix:       # LBYL
    print(Matrix[(2, 3, 6)])
else:
    print(0)

try:                           # KeyErrorEAFP
    print(Matrix[(2, 3, 6)])
except KeyError:
    print(0)

```

·解释器如何处理：Matrix[(2,3,4)] = 88 中，(2,3,4) 首先作为一个元组字面量被构建 (BUILDTUPLE)，随后作为下标传给 STORESUBSCR。dict 对该键调用 tuple.hash（一次 O(k) 计算，k=3 个元素），得到哈希后定位槽位。Matrix[(X, Y, Z)] 在运行时构造一个内容相同的新元组——但哈希值相同、== 判定相等，于是命中同一槽位——这演示了“哈希查找基于值相等而非身份”的关键性质。

·设计思想：LBYL（先检查后行动）与 EAFP（请求宽恕比允许更容易）是 Python 中两种防错哲学，if in 与 try 分别是代表；两者在本例中结果相同，但 EAFP 在“检查与使用之间存在竞态”的场景（文件、网络）中更稳——第 34 章会深入。

字典中的嵌套 (Nesting in dictionaries)

英文原文

As you can tell, dictionaries can play many roles in Python. In general, they can replace search data structures (because indexing by key is a search operation) and can represent many types of structured information.

For example, dictionaries are one of many ways to describe the properties of an item in your program's domain; that is, they can serve the same role as "records" or "structs" in other language, and JSON content in language-neutral roles. The following, for example, fills out a dictionary describing a book, by assigning to new keys:

```
>> rec = {} >> rec['title'] = 'Learning Python, 5th Edition' >> rec['year'] = 2013 >> rec['isbn'] = '9781449355739' >> rec['year'], rec['isbn'] (2013, '9781449355739')
```

Especially when nested, though, Python's built-in data types allow us to easily represent structured information. The following again uses a dictionary to capture object properties, but it codes it all at once rather than assigning to each key separately, and nests a dictionary and list to represent structured property values:

```
>> rec = {'title': 'Learning Python, 5th Edition',  
... 'date': {'year': 2013, 'month': 'July'}, ... 'isbns': ['144935  
5730', '9781449355739']} To fetch components of nes  
ted objects, simply string together indexing operatio  
ns:
```

```
>> rec['title'] 'Learning Python, 5th Edition' >> rec['is  
bns'] ['1449355730', '9781449355739'] >> rec['isbns'][  
1] '9781449355739' >> rec['date']['year'] 2013
```

Although you'll learn in Part VI that classes (which group both data and logic) can sometimes be better in this record role, dictionaries are an easy-to-use tool for simpler requirements. For more on record representation choices, see also the upcoming sidebar "Why You Will Care: List Versus Dictionary Versus Set".

Also notice that while we've focused on a single "record" with nested data here, there's no reason we couldn't nest the record itself in a larger, enclosing database collection coded as a list or dictionary, though an external file or formal database interface often plays the role of top-level container in realistic programs. The following abstract snippets would both print a record's two-item isbns list if run live and provided with an other record omitted here:

```
db = [] db.append(rec) # A list "database" db.append  
(other) db[0]['isbns']
```

```
db = {} db['lp5e'] = rec # A dictionary "database" db['l
```

```
p6e'] = other db['lp5e']['isbns']
```

Later in the book you'll meet tools such as Python's `shelve`, which works much the same way, but automatically maps objects to and from files to make them permanent. Python objects really can be database records.

中文翻译

如你所见，字典在 Python 中可以扮演很多角色。总的来说，它们可以替代搜索数据结构（因为按键索引本身就是一次搜索操作），也可以表示许多类型的结构化信息。例如，字典是描述“程序领域内某事物属性”的众多方式之一；也就是说，它可以扮演其他语言中“记录（record）”或“结构体（struct）”的角色，以及在语言中立场合下的 JSON 内容角色。例如，下面通过给新键赋值，填充了一个描述某本书的字典：

```
>>> rec = {} >>> rec['title'] = 'Learning Python, 5th Edition' >>> rec['year'] = 2013 >>> rec['isbn'] = '9781449355739' >>> rec['year'], rec['isbn'] (2013, '9781449355739')
```

尤其是嵌套时，Python 的内置数据类型让我们能轻松表示结构化信息。下面同样用字典捕捉对象属性，但它一次写完而非逐个按键赋值，并嵌套了一个字典和一个列表来表示结构化的属性值：

```
>>> rec = {'title': 'Learning Python, 5th Edition', ... 'date': {'year': 2013, 'month': 'July'}, ... 'isbns': ['1449355730', '9781449355739']}
```

要取出嵌套对象的组件，只需把索引操作串起来：

```
>>> rec['title'] 'Learning Python, 5th Edition' >>> rec['isbns'] ['1449355730', '9781449355739'] >>> rec['isbns'][1] '9781449355739' >>> rec['date']['year'] 2013
```

虽然第 VI 部分

会告诉你类 (class) (把数据和逻辑组合在一起) 在"记录"这个角色上有时更好, 但对于更简单的需求, 字典是一个易用的工具。关于记录表达方式的更多选择, 还可以参考后面即将出现的专栏"你会在意什么: 列表 vs 字典 vs 集合"。还要注意, 虽然我们这里聚焦于一个带嵌套数据的单一"记录", 但我们完全可以把记录本身再嵌套进一个更大的、用列表或字典编码的外层数据库集合中——不过在真实程序里, 顶层容器角色往往由外部文件或正式的数据库接口担当。下面两个抽象片段在真实运行并提供此处省略的 other 记录时, 都会打印一条记录的两项 isbnns 列表: db = [] db.append(rec) # 列表式"数据库" db.append(other) db[0]['isbnns'] db = {} db['lp5e'] = rec # 字典式"数据库" db['lp6e'] = other db['lp5e']['isbnns'] 在本书后面, 你还会遇到像 Python 的 shelve 这样的工具, 它的工作方式与此非常相似, 但会自动把对象映射到文件、从文件映射回对象, 使其持久化。Python 对象真的可以成为数据库记录。

深度理解

- 核心概念: 字典即记录 (record), 嵌套即结构。{'date': {'year': ..., 'month': ...}} 正是 JSON 的天然映射——Python 的 json 模块就是在这两种格式间无缝转换。这是"无模式 (schemaless) 数据"的核心载体。
- 底层实现: rec['isbnns'][1] 分两步: 先按 'isbnns' 哈希查得列表引用 (O(1)), 再对该列表做数组索引 (O(1))。rec['date']['year'] 同理是两次哈希查找。嵌套只增加"引用链"的深度, 不复制任何数据。
- 设计原因: 为什么字典在"简单记录"场景胜过类? 因为类

需要先定义、绑定逻辑，而字典即写即用、无需声明——对脚本化、原型化、数据交换（JSON）场景零成本。类在“数据+行为的封装”上才显现优势（第 27 章）。

·实际问题：Web API 的请求/响应体、配置文件（YAML/JSON/TOML）解析结果、数据库查询行、测试用例数据——全部是嵌套字典的形态。掌握“串式索引”是读取它们的基本功。

·初学者误区：① 试图 `rec['date.year']` 这种“点式路径”索引——不行，必须逐层 `['date']['year']`；② 误以为嵌套字典会“复制”内层——全是引用；③ 把嵌套当“银弹”，层数过深（超过 3 层）可读性崩塌，此时应考虑类或数据类（`dataclass`）。

代码分析

```
rec = {'title': 'Learning Python, 5th Edition',
       'date': {'year': 2013, 'month': 'July'},
       'isbns': ['1449355730', '9781449355739']}

rec['title']           # 'Learning Python, 5th Edition'
rec['isbns']           # ['1449355730', '9781449355739']
rec['isbns'][1]        # '9781449355739' 1
rec['date']['year']    # 2013

db = []                #
db.append(rec)
db[0]['isbns']         #

db = {}                #
db['lp5e'] = rec
db['lp5e']['isbns']    #
```

·解释器如何处理：`rec['date']['year']` 的字节码序列是 BI

NARYSUBSCR (date) → BINARYSUBSCR (year) : 第一次查找返回内层字典对象 (引用传递, 无复制), 第二次在它上面继续哈希查找。若中间任何键不存在, 立即抛 `KeyError`——所以"串式索引"要保证路径上的每个键都存在。db['lp5e']['isbn'] 同理: 先定位记录 ($O(1)$), 再取字段。

·设计思想: 记录 = 字典 + 名字; 数据库 = 记录 + 容器。Python 用"任意嵌套"统一了"单条记录"与"整库数据"的表达——shelve 等持久化工具正是站在这个统一模型上的。

字典键/值/项视图对象 (Dictionary key/value/item view objects)

英文原文

When we explored the dictionary keys, values, and items methods earlier, we wrapped their results in list calls for display at the REPL prompt. Technically, this is because these methods return view objects that produce results on demand, instead of physical lists. Displaying their raw values suggests as much, but collects their values anyhow:

```
>> D = dict(program=1, script=2, app=3) >> D.keys()
dictkeys(['program', 'script', 'app'])
```

View objects are iterables, which we've seen simply means objects that generate result items one at a time

e, instead of producing the result list all at once in memory. Besides being iterable, dictionary views are insertion ordered, reflect future changes to the dictionary, and support set operations.

On the other hand, because they are not lists, they do not directly support operations like indexing or the list sort method, and do not display as a normal list when printed.

We'll discuss the notion of iterables more formally in Chapter 14, but for our purposes here it's enough to know that we have to run the results of these three methods through the list built-in if we want to apply list operations or display their values as lists. For example:

```
>> D = dict(a=1, b=2, c=3) >> D {'a': 1, 'b': 2, 'c': 3} >
> K = D.keys() # Makes a view object, not a list >> K
dictkeys(['a', 'b', 'c']) >> list(K) # Force a real list when
needed ['a', 'b', 'c'] >> V = D.values() # Ditto for values
s and items views >> V dictvalues([1, 2, 3]) >> list(V)
[1, 2, 3] >> D.items() dictitems([('a', 1), ('b', 2), ('c', 3)])
>> list(D.items()) [('a', 1), ('b', 2), ('c', 3)] >> K[0] # Li
st operations fail unless converted TypeError: 'dictkey
s' object is not subscriptable >> K.sort() AttributeError:
'dictkeys' object has no attribute 'sort' >> list(K)[0],
list(K).sort() ('a', None)
```

Unlike lists, though, dictionary views don't take up space for their full results, and are not carved in stone

when created—they dynamically reflect future changes made to the dictionary after the view object has been created:

```
>> D = {'a': 1, 'b': 2, 'c': 3} >> D {'a': 1, 'b': 2, 'c': 3} >>
K = D.keys() >> V = D.values() >> list(K), list(V) # Views maintain same order as dictionary (['a', 'b', 'c'], [1, 2, 3])
>> del D['b'] # Change the dictionary in place >
> D {'a': 1, 'c': 3} >> list(K), list(V) # Reflected in any current view objects (['a', 'c'], [1, 3])
```

That said, use cases for dynamic view morph are likely rare. In fact, apart from result displays at the interactive prompt, you probably won't even notice views very often in practice, because looping constructs in Python automatically force them to produce one result on each:

```
>> for k in D.keys(): print(k)
... # View iterators used automatically in loops
```

As we've seen, it's also often unnecessary to call keys directly in such cases because dictionaries themselves provide implicit key iterators; for better or worse, `for k in D` is usually as much code as you need to type.

中文翻译

之前探索字典的 `keys`、`values` 和 `items` 方法时，我们在 REPL 提示符下把它们的返回值包进 `list` 调用来显示。从技术上讲，这是因为这些方法返回视图对象（view objects）——

—按需产生结果，而不是物理列表。显示它们的原始值也能看出这一点，但（原书示例）仍会收集它们的值：

```
>>> D = dict(program=1, script=2, app=3) >>> D.keys() dictkeys(['program', 'script', 'app'])
```

视图对象是可迭代对象（iterables）——我们说过，这仅仅意味着它们一次生成一个结果项，而不是在内存里一次性产出整个结果列表。除了可迭代之外，字典视图还是插入有序的、会反映字典的后续变化、并且支持集合操作。另一方面，因为它们不是列表，它们不直接支持索引或列表 `sort` 方法等操作，打印时也不会像普通列表那样显示。我们会在第 14 章更正式地讨论可迭代对象的概念，但就我们这里的目的而言，只需要知道：如果想应用列表操作，或把它们的值作为列表显示，就必须把这三种方法的结果交给 `list` 内置函数处理。例如：

```
>>> D = dict(a=1, b=2, c=3) >>> D {'a': 1, 'b': 2, 'c': 3} >>> K = D.keys() # 生成的是视图对象，不是列表 >>> K dictkeys(['a', 'b', 'c']) >>> list(K) # 需要时才强制出真列表 ['a', 'b', 'c'] >>> V = D.values() # values 和 items 视图同理 >>> V dictvalues([1, 2, 3]) >>> list(V) [1, 2, 3] >>> D.items() dictitems([('a', 1), ('b', 2), ('c', 3)]) >>> list(D.items()) [('a', 1), ('b', 2), ('c', 3)] >>> K[0] # 列表操作不转换会失败 TypeError: 'dictkeys' object is not subscriptable >>> K.sort() AttributeError: 'dictkeys' object has no attribute 'sort' >>> list(K)[0], list(K).sort() ('a', None)
```

与列表不同，字典视图不会为完整结果占用空间，而且创建后不是一成不变的——它们会动态反映视图对象创建之后对字典所做的后续修改：

```
>>> D = {'a': 1, 'b': 2, 'c': 3} >>> D {'a': 1, 'b': 2, 'c': 3} >>> K = D.keys() >>> V = D.values() >>> list(K), list(V) # 视图与字典保持相同顺序(['a', 'b
```

```
['c'], [1, 2, 3]) >>> del D['b'] # 原地修改字典>>> D {'a': 1, 'c': 3} >>> list(K), list(V) # 已存在的视图对象同步反映出来(['a', 'c'], [1, 3]) 话虽如此, "动态视图变形"的用例可能很少见。事实上, 除了在交互提示符下显示结果之外, 实践中你可能甚至不太会注意到视图——因为 Python 的循环结构会自动迫使它们每次产生一个结果: >>> for k in D.keys(): print(k) ... # 循环中自动使用视图迭代器a b c 正如我们所见, 这种情况下通常会调用 keys 也常常不是必须的, 因为字典本身提供隐式的键迭代器; 无论好坏, for k in D 通常就是你需要敲的全部代码。
```

深度理解

·核心概念: 视图 (view) 是"按需迭代 + 零拷贝 + 实时反映"的动态窗口。它既不是快照列表, 也不是完全静态的东西——而是字典底表的活映射。

·底层实现: D.keys() 返回 dictkeys 对象, 内部持有对字典本身的引用 (而非数据拷贝)。每次迭代时实时从字典的 entries 数组逐个读数——这就是它"反映后续变化"的原理: 它读的是当前表, 而不是创建时的快照。同理不占 O(n) 内存。K[0] 之所以失败, 是因为 dictkeys 没有实现 getitem——解释器返回 "not subscriptable"。

·设计原因: 老版本 Python 里 keys() 返回真列表 (占用 O(n) 内存、还是快照)。视图的引入 (3.0) 是为大数据与一致性: ① 不浪费内存; ② 避免"快照过时"导致的困惑 (判断成员后再取却找不到)。代价是丢失了列表的部分能力 (索引/sort) 。

·实际问题：视图动荡在并发修改或"先取视图后改字典"时会出现——如果你预期遍历期间字典不变，建议先 `list(D.keys())` 固化。日常 `"for k in D"` 完全不碰视图对象，所以多数人无感知。

·初学者误区：① 对 `K = D.keys()` 后调用 `K[0]/K.sort()` 报错不解；② 以为视图是"某时刻的照片"，忘了它实时；③ 在 `for` 循环里修改字典（增删键）可能引发 `RuntimeError: dictionary changed size during iteration`——这是"活视图"的另一面。

代码分析

```
D = dict(a=1, b=2, c=3)
K = D.keys()           # dict_keys(['a', 'b', 'c'])
list(K)                # ['a', 'b', 'c']
K[0]                   # TypeError
K.sort()               # AttributeError sort

#
V = D.values()         #
del D['b']              #
list(V)                # [1, 3]"""

for k in D.keys(): print(k)  # list()
for k in D: print(k)        #
```

·解释器如何处理：`K = D.keys()` 时解释器调用 `PyDictKeys`——创建一个 `dictkeys` 对象，没有复制任何键，只是把指向 `D` 的指针存进去。`list(K)` 则对它做完整迭代：每次 `tpiternext` 从 `D` 的 `entries` 数组实况读取一个键，全部读完后构造真列表。当 `del D['b']` 修改字典后，`V` (`values` 视图) 在下次迭代时重新扫描 `entries`，自然就看不到 `b`

了。视图的"实时性"本质是"读时再找", 而非"创建时快照"。

·设计思想: "视图"是 Python 对"懒求值 (laziness)"的一次完整示范: 列表是热切物化 (eager), 视图是惰性窗口 (lazy)。它也是第 14 章"迭代器协议"的提前预演——list() 对视图的消费行为与对任何迭代器完全一致。

字典视图与集合 (Dictionary views and sets)

英文原文

Though perhaps even more obscure than view objects in general, some dictionary views are also set-like and support set operations such as union and intersection that we used on true sets in Chapter 5. Specifically, views returned by the keys method are set-like, values views are not, and items views are if their (key, value) pairs are unique and hashable (immutable).

This reflects logical symmetry: set items are unique and immutable just like dictionary keys, and sets themselves behave like unordered and valueless dictionaries (and are even coded in curly braces).

Here is what keys views look like when used in set operations (continuing the prior section's session); as also shown, dictionary value views are never set-like, because their items are not necessarily unique or immutable:

```
>> K, V (dictkeys(['a','c']), dictvalues([1, 3])) >> K | {'x': 4} # Keys (and some items) views are set-like {'x','a','c'} # Results are unordered sets, not views >> V | {'x': 4}
TypeError: unsupported operand type(s) for |: 'dictvalues' and 'dict'
>> V | {'x': 4}.values()
TypeError: unsupported operand type(s) for |: 'dictvalues' and 'dictvalues'
```

Be careful not to confuse the `|` here with the dictionary union operation we met earlier in "Dictionary "Union" Operator"; when run on a view, `|` is not a full key-based dictionary merge. In set operations, views may be mixed with other views, sets, and dictionaries, and dictionaries are treated the same as their keys views:

```
>> D = {'a': 1, 'b': 2, 'c': 3} >> D.keys() & D.keys() # Intersect views {'b','a','c'}
>> D.keys() & {'b'} # Intersect view and set {'b'}
>> D.keys() & {'b': 1} # Intersect view and dict {'b'}
>> D.keys() | {'b','c','d'} # Union view and set {'b','a','d','c'}
```

Items views are set-like too, but only if they are hashable—that is, if they contain only immutable objects:

```
>> {'a': [1, 2]}.items() | {0, 1} # Immutable value: no set ops
TypeError: unhashable type: 'list'
>> D = {'a': 1}
>> D.items() # Items set-like if and only if hashable
dictitems([('a', 1)])
>> D.items() | D.keys() # Union view and view {'(a', 1),'a'}
>> D.items() | D # Dictionary treated same as its keys {'(a', 1),'a'}
>> D.items() | {'(c', 3), ('d', 4)} # Set of key/value pairs {'(a', 1), ('c', 3), ('d',
```

```
4)} >> dict(D.items() | {'c': 3}, {'d': 4}) # dict accepts i  
terable sets too {'a': 1,'c': 3,'d': 4}
```

See Chapter 5's coverage of sets if you need a refresher on these operations. Their role in dictionary views is probably uncommon and may even be academic; but they work when helpful (and at least you now have a fighting chance when they crop up in sadistic final exams!). Here, let's wrap up with two more quick coding notes for dictionaries.

中文翻译

虽然可能比一般的视图对象更冷门，但有些字典视图也是集合式 (set-like) 的，支持我们在第 5 章对真正的集合使用过的并集、交集等集合运算。具体来说：keys 方法返回的视图是集合式的，values 视图不是，而 items 视图在它们的 (键, 值) 对唯一且可哈希 (不可变) 时也是集合式的。这反映了逻辑上的对称性：集合项就像字典键一样唯一且不可变，而集合本身的行为就像“无序、无值的字典” (甚至也用花括号编码)。下面看看 keys 视图在集合运算中长什么样 (延续上一节的会话)；同时也展示出：字典的值视图从来不是集合式的，因为它们的项不保证唯一或不可变：

```
>>> K, V (dictkeys(['a','c']), dictvalues([1, 3])) >>> K | {'x': 4} # keys (以及部分 items) 视图是集合式的{'x','a','c'}  
# 结果是无序集合，不再是视图>>> V | {'x': 4} TypeError:  
: unsupported operand type(s) for |: 'dictvalues' and 'dict'  
>>> V | {'x': 4}.values() TypeError: unsupported operand type(s) for |: 'dictvalues' and 'dictvalues'
```

注意不要把这里的 | 与前面“字典 |”合并运算符章节讨论的字典合并

混淆了；当作用于视图时，| 不是一次完整的基于键的字典合并。在集合运算中，视图可以与其他视图、集合以及字典混合，且字典会被视同其 keys 视图处理：

```
>>> D = {'a': 1, 'b': 2, 'c': 3}
>>> D.keys() & D.keys() # 视图交集{'b', 'a', 'c'}
>>> D.keys() & {'b'} # 视图与集合交集{'b'}
>>> D.keys() & {'b': 1} # 视图与字典交集{'b'}
>>> D.keys() | {'b', 'c', 'd'} # 视图与集合并集{'b', 'a', 'd', 'c'}
Items 视图也是集合式的，但仅在它们可哈希时成立——也就是说，仅当它们只包含不可变对象时：
```

```
>>> {'a': [1, 2]}.items() | {0, 1}
# 值不可变做不到：没有集合运算
TypeError: unhashable type: 'list'
>>> D = {'a': 1}
>>> D.items() # items 可哈希时才是集合式的
dictitems([('a', 1)])
>>> D.items() | D.keys() # 视图与视图并集({'a', 1}, 'a')
>>> D.items() | D # 字典被视同其 keys
{'a', 1}, 'a'
>>> D.items() | {'c', 3}, ('d', 4)} # 键值对集合({'a', 1}, ('c', 3), ('d', 4))
>>> dict(D.items() | {'c', 3}, ('d', 4)) # dict 也接受可迭代的集合
{'a': 1, 'c': 3, 'd': 4}
如果需要复习这些运算，请参见第 5 章关于集合的内容。它们在字典视图中的角色可能并不常见，甚至可能有点学术化；不过在有用时它们确实能用（而且至少在变态的期末考试里出现时，你现在有了一战之力！）。这里让我们再用两条笔记给字典收尾。
```

深度理解

·核心概念：视图的“集合式”能力取决于元素的唯一性 + 可哈希性：keys 天然唯一可哈希 → 集合式；values 可能重复、不可哈希 → 永非集合式；items 只有在“（键，值）元组可哈希”（即值不可变）时才集合式。隐蔽但对称：集合 = “无值字典”，字典键 = “集合元素”。

·底层实现：K | {'x': 4} 时，dictkeys 实现了 tpasnumber -> nbor 槽位 (dictviewsor 等函数)，内部把操作数转成临时集合执行 set.union，返回真集合——所以结果类型是 set，并失去字典的语义。当 V | {'x': 4} 时，dictvalues 没有实现该槽位，解释器尝试"反射操作"失败后抛 TypeError。

·设计原因：让 keys 视图支持集合运算，是让"两个字典的共同键"这类查询成为内置能力——D1.keys() & D2.keys() 一行完成"求共有字段"，无需手写循环；setdefault/Counter 与它一起构成字典高级建模三件套。

·实际问题：快捷运算「哪些键两边都有」「哪些键只在 A 里」；items 视图的 & 还能按"同时有某键某值"筛选。但注意：这些返回的是无序集合，且不是完整合并序列——需要有序结果请先 list(...) 再自行处理。

·初学者误区：① 把视图上的 | 误当字典合并（它返回集合、非视图）；② V | ... 失败后不知道为何——值不唯一/不可哈希；③ 试图把集合运算结果再当字典用（先 dict(..) 转换才行）。

代码分析

```
K = {'a': 1, 'c': 3}.keys()      # dict_keys(['a', 'c'])
K | {'x': 4}                    # {'x', 'a', 'c'} set

D = {'a': 1, 'b': 2, 'c': 3}
D.keys() & D.keys()             # {'c', 'a', 'b'}
D.keys() & {'b'}                # {'b'}
D.keys() & {'b': 1}             # {'b'} keys
D.keys() | {'b', 'c', 'd'}     # {'b', 'a', 'd', 'c'}
```

```
{'a': [1, 2]}.items() | {0, 1}    # TypeError:items
D.items() | D.keys()              # ({'a',1), 'a'}
D.items() | D                    # ({'a',1), 'a'} keys
dict(D.items() | {'c': 3, 'd': 4}) # {'a':1,'c':3,'d':4}...
```

·解释器如何处理：D.keys() & {'b': 1} 中解释器调用 dict keys 的 tpasnumber->nband 槽位 (dictviewsand)：它先对右侧操作数求"键的集合"（对字典取 keys），再与左视图执行标准集合交集，工厂函数返回一个新的 set 对象。这就是为什么字典"被视同其 keys 视图"。{'a':[1,2]}.items() 在创建时就成功，但一旦参与集合运算，内部要哈希 ('a', [1,2]) 这个元组——元组含不可哈希元素，于是 TypeError: unhashable type:'list'。

·设计思想：这套"视图即集合"的设计证明了：唯一性+可哈希性是集合与字典键的共同数学基础——两种看似不同的容器，在底层是同一棵树的枝叶。

排序字典键 (Sorting dictionary keys)

英文原文

First of all, keys lists must be sorted when the inherent insertion order doesn't meet your goals. Beware, though, that a common coding pattern for scanning a collection in sorted order won't work, because keys does not return a list:

```
>> D = {'c': 3,'b': 2,'a': 1} >> D {'c': 3,'b': 2,'a': 1} >>
Ks = D.keys() # Sorting a view object doesn't work! >
```

> Ks.sort() AttributeError:'dictkeys' object has no attribute'sort'

To work around this, you can either convert keys to a list manually, or use the sorted call, applied to lists earlier in this chapter, on either a keys view or the dictionary itself:

```
>> Ks = list(D.keys()) # Convert keys to a list
>> Ks.sort() # And then sort with list.sort()
>> for k in Ks: print(k, D[k])
```

```
... a 1 b 2 c 3
>> Ks = D.keys()
>> for k in sorted(Ks): print(k, D[k]) # sorted() accepts any iterable
```

```
... # sorted() returns its result
a 1 b 2 c 3
```

Of these, using the dictionary's keys iterator is simplest and common, though also arguably implicit:

```
>> for k in sorted(D): print(k, D[k]) # Or use sorted() on the dict
```

```
... # dict iterators produce keys
a 1 b 2 c 3
```

Dictionary magnitude comparisons Finally, dictionaries can be compared for equality directly with `==`, which automatically compares each key/value pair and ignores insertion order:

```
>> D1 = dict(a=1, b=2, c=3)
>> D2 = dict(c=3, b=2, a=1)
>> D1, D2 ({'a': 1, 'b': 2, 'c': 3}, {'c': 3, 'b': 2, 'a': 1})
>> D1 == D2, D1 != D2 (True, False)
```

Magnitude comparisons like `<` and `>` do not work on dictionaries themselves, though you can implement them by comparing key/value items views manually, if you also sort them to discount any insert-order difference (subtlety: `a > b` on the items results directly runs the set superset test because the views are set-like—not greater-than!):

```
>> D1 = dict(a=1, b=2, c=4) >> D2 = dict(c=3, b=2, a=1) >> D1 > D2 # D1 > D2: D1['c'] > D2['c'] TypeError: '>' not supported between instances of 'dict' and 'dict' >> list(D1.items()), list(D2.items()) ([('a', 1), ('b', 2), ('c', 4)], [('c', 3), ('b', 2), ('a', 1)]) >> list(D1.items()) > list(D2.items()) # Fails: insertion order False >> D1.items() > D2.items() # Fails: superset, not magnitude False >> sorted(D1.items()) > sorted(D2.items()) # OK: sorted to neutralize order True
```

Because we'll revisit this near the end of the next chapter to reinforce them in the context of comparisons at large, we'll postpone further coverage here.

中文翻译

首先，当固有的插入顺序不能满足你的目标时，就必须对键列表排序。不过要小心：一种常见的“按排序顺序扫描集合”的写法在这里行不通，因为 `keys` 返回的不是列表：

```
>>> D = {'c': 3, 'b': 2, 'a': 1} >>> D {'c': 3, 'b': 2, 'a': 1} >>> Ks = D.keys() # 对视图对象排序行不通！ >>> Ks.sort() AttributeError: 'dictkeys' object has no attribute 'sort'
```

要绕过这个问题，你有两种办法：手动把键转换成列表，或用本

章早先用于列表的 `sorted` 调用——无论作用于 `keys` 视图还是字典本身都行：

```
>>> Ks = list(D.keys()) # 把键转成列表
>>> Ks.sort() # 再用 list.sort() 排序
>>> for k in Ks: print(k, D[k]) ... a 1 b 2 c 3
>>> Ks = D.keys()
>>> for k in sorted(Ks): print(k, D[k]) # sorted() 接受任何可迭代对象... # sorted() 返回结果a 1 b 2 c 3
其中，使用字典的键迭代器最简单、最常用，不过也可以说最隐式：
```

```
>>> for k in sorted(D): print(k, D[k]) # 或对字典本身用 sorted() ... # 字典迭代器产出键a 1 b 2 c 3
字典的大小比较 (Dictionary magnitude comparisons) 最后，字典可以直接用 == 比较相等，它会自动比较每个键值对，并忽略插入顺序：
```

```
>>> D1 = dict(a=1, b=2, c=3)
>>> D2 = dict(c=3, b=2, a=1)
>>> D1, D2 ({'a': 1, 'b': 2, 'c': 3}, {'c': 3, 'b': 2, 'a': 1})
>>> D1 == D2, D1 != D2 (True, False)
< 和 > 这样的大小比较不能直接作用于字典本身，不过你可以手动比较键/值的 items 视图来实现，前提是还要对它们排序以抵消插入顺序的差异（这里有个隐蔽点：对 items 结果直接做 > 会运行集合的超集（superset）测试，因为这些视图是集合式的——那不是大于！）：
```

```
>>> D1 = dict(a=1, b=2, c=4)
>>> D2 = dict(c=3, b=2, a=1)
>>> D1 > D2 # D1 > D2? D1['c'] > D2['c']
TypeError: '>' not supported between instances of 'dict' and 'dict'
>>> list(D1.items()), list(D2.items()) ([('a', 1), ('b', 2), ('c', 4)], [('c', 3), ('b', 2), ('a', 1)])
>>> list(D1.items()) > list(D2.items()) # 失败：插入顺序False
>>> D1.items() > D2.items() # 失败：是超集测试，不是大于False
>>> sorted(D1.items()) > sorted(D2.items()) # 可行：排序以中和顺序True
因为我们会在下一章末尾在"比较"的大背景下再次讨
```

论它们来强化，所以这里的覆盖就到此为止。

深度理解

·核心概念：① 排序字典键 = `sorted(D) / sorted(D.keys()) / list(D.keys()) + .sort()` 三选一；② 字典 `==` 比较键值对内容且无视插入顺序；③ 字典 `</>` 不支持 (`TypeError`) ——但 `items` 视图的 `</>` 会偷偷变成集合的超集/子集测试，这是隐蔽陷阱。

·底层实现：`sorted(D)` 对字典调用其 `iter` (产键)，把所有键收集成列表后走 `Timsort`，返回新列表。`D1 == D2` 走 `dictequal`：先比较长度，再对每个键检查 `D2` 是否有且值相等 (值相等用 `==` 递归)。`D1.items() > D2.items()` 调用 `dictitems` 的集合比较槽位——`set.gt` 是“真超集”判断 (`D1.items() D2.items()`)，返回 `False` 因为两个 `items` 集合互不包含。

·设计原因：“大小比较”对无序映射本身无意义 (“字典 `a` 比字典 `b` 大”数学上没定义)，所以语言直接禁掉；而“相等”有意义 (内容是否一致)，所以保留。视图的集合式 `>` 是“超集”这一不同概念复用符号——Python 对同一个 `>` 在不同类型上给出不同语义，是其动态分派的又一例证。

·实际问题：输出报表要“按键排序”时 `for k in sorted(D)` 是最短写法；比较两个配置字典是否一致直接用 `==` (与顺序无关，符合直觉)；要真正按“排序后的 `items`”比较大小，先 `sorted()` 中和顺序再做序列比大小。

·初学者误区：① `D.keys().sort()` 报错后手足无措——视图没有 `sort`；② `D1 > D2` 报 `TypeError` 就以为"字典不能比"——而 `items` 视图的 `>` 能跑，但语义是超集；③ 忘了 `==` 忽略顺序——以为 `dict(a=1)` 与 `dict(a=1, b=2)` 的某种有序概念。

代码分析

```
D = {'c': 3, 'b': 2, 'a': 1}
Ks = D.keys()
Ks.sort()                                # AttributeError sort

Ks = list(D.keys()); Ks.sort()           #
for k in Ks: print(k, D[k])              # a 1 / b 2 / c 3

for k in sorted(D.keys()): ...            # sorted
for k in sorted(D): ...                  # sorted

D1 = dict(a=1, b=2, c=3)
D2 = dict(c=3, b=2, a=1)
D1 == D2                                # True

D1 = dict(a=1, b=2, c=4); D2 = dict(c=3, b=2, a=1)
D1 > D2                                  # TypeErrordict
list(D1.items()) > list(D2.items())      # False
D1.items() > D2.items()                  # False""
sorted(D1.items()) > sorted(D2.items())  # True
```

·解释器如何处理：`sorted(D)` 中 `sorted` 接收的虽是 `dict`，但它不知道、也不关心具体类型——它只调用 `PyObjectGetIter(D)` 得到键迭代器，全部消费后对新列表排序。这就是"sorted 接受任何可迭代对象"在底层的样子（多态/鸭子类型）。`D1 == D2` 走 `dictrichcompare(PyEQ)`：先比长度，相等则对每个键做 `dictlookup` 找另一字典中

的同键并比较值（值可能递归嵌套）。`D1.items() > D2.items()` 走集合语义：`setrichcompare` 判断 `items1` `items2`——两集合互不为子集，返回 `False`。而 `sorted(...) > sorted(...)` 走列表的序列比较：逐三元组比较，`('c',4)` 在 `('c',3)` 之后更大 $\rightarrow$ `True`。

·设计思想：符号 `>` 的"多面性"（数字`>`、序列字典序`>`、集合超集`>`）是 Python"运算符即协议、含义由类型决定"的集中体现——它在简洁性与"同符异义"的迷惑性之间，选择了前者。理解这一点，比背每种情况的结论重要得多。

8.6 本章小结 (Chapter Summary)

英文原文

In this chapter, we explored the list and dictionary types—probably the two most common, flexible, and powerful collection types you will see and use in Python code. We learned that the list type supports positionally ordered collections of arbitrary objects, and that it may be freely nested and grown and shrunk on demand. The dictionary type is similar, but it stores items by key instead of by position and orders keys by insertion time only. Both lists and dictionaries are mutable, and so support a variety of in-place change operations not available for strings: for example, lists can be grown by slice assignment and append calls, and

dictionaries by key assignment and update. In the next chapter, we will wrap up our in-depth core object-type tour by looking at tuples and files. After that, we'll move on to statements that code the logic that processes our objects, taking us another step toward writing complete programs. Before we tackle those topics, though, here are some chapter quiz questions to review what you've learned.

中文翻译

在本章中，我们探索了列表和字典这两种类型——它们大概是你 Python 代码中会看到和用到的最常见、最灵活、最强大的集合类型。我们学到了：列表类型支持任意对象的位置有序集合，可以自由嵌套、按需增长和收缩。字典类型与此类似，但按键而不是按位置存储条目，并且键只按插入时间排序。列表和字典都是可变的，因此支持字符串所没有的各种原地修改操作：例如，列表可以通过切片赋值和 `append` 调用增长，字典可以通过键赋值和 `update` 增长。在下一章，我们将通过研究元组 (tuple) 和文件 (file) 来为深入的核心对象类型之旅收尾。之后，我们将转向编写处理对象的逻辑的语句 (statement)，向写出完整程序再迈进一步。不过在着手这些主题之前，这里有一些章节测验题，用来复习你学到的东西。

深度理解

·核心概念：本章的“知识骨架”：`list` = 位置容器（有序/可变/任意嵌套），`dict` = 键容器（有序插入/可变/映射）。两者的共同点（可变、引用存储、异构、可嵌套）与差异

- 点（位置 vs 键、序列 vs 映射）构成全部细节的两根主轴。
- 底层实现：回顾全章两条主线：list 的动态数组（连续指针数组 + 约 12.5% 的扩容增量 + memmove 搬移，中段操作 $O(n)$ 、尾部摊还 $O(1)$ ）；dict 的紧凑哈希表（indices + entries 双表，开放寻址 + 哈希随机化，平均 $O(1)$ 查找，插入有序）。这两套机制解释了本章几乎所有“为什么”。
- 设计原因：为什么这两个类型成为主力？因为真实世界的的数据天然分为两种：有位置的一组（名单、序列、坐标）与有名字的一组（属性、配置、记录）。两种思维模型，两种容器。
- 实际问题：几乎所有脚本都同时需要两者：读文件成行列表（list）、解析配置成键值（dict）、统计计数（dict 或 Counter）、按序处理结果（list）。
- 初学者误区：把本章的知识当“API 大全”死记——正确做法是抓住“位置 vs 键”和“可变 vs 不可变”两条主线，其余全部可以在 REPL 里用 `help(list)/help(dict)` 随时查证。

8.7 测验 (Test Your Knowledge: Quiz)

英文原文

1. Name two ways to build a list containing five integer zeros.
2. Name two ways to build a dictionary with two keys, 'a' and 'b', each having an associated value o

f 0. 3. Name four operations that change a list object in place. 4. Name four operations that change a dictionary object in place. 5. Why might you use a dictionary instead of a list?

中文翻译

1. 说出两种构建"包含五个整数 0"的列表的方法。2. 说出两种构建"包含'a' 和'b' 两个键、每个键关联值为 0"的字典的方法。3. 说出四种原地修改列表对象的操作。4. 说出四种原地修改字典对象的操作。5. 为什么你可能用字典而不是列表?

8.8 测验答案 (Test Your Knowledge: Answers)

英文原文

1. A literal expression like `[0, 0, 0, 0, 0]` and a repetition expression like `[0] * 5` will each create a list of five zeros. In practice, you might also build one up with a loop that starts with an empty list and appends 0 to it in each iteration, with `L.append(0)`. A list comprehension like `[0 for i in range(5)]` could work here, too, but this is more work than you need to do for this answer.

2. A literal expression such as `{'a': 0, 'b': 0}` or a series of assignments like `D = {}`, `D['a'] = 0`, and `D['b'] = 0` would create the desired dictionary. You can also use the newer and simpler-to-code `dict(a=0, b=0)` keyword

d form, or the more flexible `dict([('a', 0), ('b', 0)])` key/value sequences form. Because all the values are the same, you can also use the special form `dict.fromkeys('ab', 0)`, and dictionary comprehension like `{k: 0 for k in 'ab'}` suffices too, though again, this may be overkill here.

3. The `append` and `extend` methods grow a list in place, the `sort` and `reverse` methods order and reverse lists, the `insert` method inserts an item at an offset, the `remove` and `pop` methods delete from a list by value and by position, the `del` statement deletes an item or slice, and index and slice assignment statements replace an item or entire section. Pick any four of these for the quiz.

4. Dictionaries are primarily changed by assignment to a new or existing key, which creates or changes the key's associated value in the table. Also, the `del` statement deletes a key's entry, the dictionary update method merges one dictionary into another in place, and `D.pop(key)` removes a key and returns the value it had. Dictionaries also have other, more exotic in-place change methods presented tersely or not at all in this chapter, such as `popitem` and `setdefault`; see reference resources for more details.

5. This question is a bit unfair, given that the following sidebar gives the answer, but you may have already figured this out on your own. Dictionaries are generally better when the data is labeled (a record with field names, for example); lists are best suited to collections of unlabeled items (such as all the files in a dire

ctory)。Dictionary lookup is also usually quicker than searching a list, though this might vary per program and Python.

中文翻译

1. 像 `[0, 0, 0, 0, 0]` 这样的字面量表达式和像 `[0] * 5` 这样的重复表达式，都会创建包含五个零的列表。实践中，你也可以用循环构建：从空列表开始，每次迭代用 `L.append(0)` 追加一个 0。像 `[0 for i in range(5)]` 这样的列表推导式也能实现，但对这道题来说，付出的工作量超出了需要。2. 像 `{'a': 0, 'b': 0}` 这样的字面量表达式，或像 `D = {}`、`D['a'] = 0`、`D['b'] = 0` 这样的一系列赋值，都能创建所需的字典。你还可以用更新、更易写的 `dict(a=0, b=0)` 关键字形式，或更灵活的 `dict([('a', 0), ('b', 0)])` 键/值序列形式。因为所有值都相同，你也可以用特殊形式 `dict.fromkeys('ab', 0)`；字典推导式 `{k: 0 for k in 'ab'}` 也能胜任——尽管这可能有点大材小用。3. `append` 和 `extend` 方法原地增长列表，`sort` 和 `reverse` 方法给列表排序和反转，`insert` 方法在偏移处插入项，`remove` 和 `pop` 方法分别按值和按位置从列表删除，`del` 语句删除一项或一段，索引与切片赋值语句替换一项或整个区间。从其中任选四个作为本题答案即可。4. 字典主要通过给新键或已有键赋值来修改——这会在表中创建或改变键的关联值。此外，`del` 语句删除某个键的条目，字典的 `update` 方法把另一个字典就地合并进来，`D.pop(key)` 移除一个键并返回它原来的值。字典还有其他本章只是简略提及甚至没提的比较冷门的原地修改方法，比如 `popitem` 和 `setdefault`；更多细节请查阅参考资源。5. 这道题有点不公平，因为下面的专栏就给出了答案，不过你可能

已经自己想明白了。当数据是带标签的（例如带字段名的记录）时，字典通常更好；列表最适合存放无标签的项集合（例如一个目录下的所有文件）。字典查找通常也比搜索列表更快，尽管这可能因程序和 Python 版本而异。

8.9 专栏：你会在意什么——列表 vs 字典 vs 集合 (Why You Will Care: List Versus Dictionary Versus Set)

英文原文

With all the objects in Python's core types arsenal, some readers may be puzzled over the choice between lists and dictionaries. In short, although both are flexible collections of other objects, lists assign items to positions, and dictionaries assign them to more mnemonic keys. Because of this, dictionary data often carries more meaning to human readers. For example, a nested list structure can always be used to record info:

```
>> L = ['Pat', 40.5, ['dev','mgr']] # List-based "record"
>> L[0]'Pat'>> L[1] # Positions/numbers for fields 4
0.5 >> L[2][1]'mgr'
```

For some types of data, the list's access-by-position makes sense—a list of employees in a company, the files in a directory, or numeric matrixes, for example. But a more symbolic record like this may be more meaningfully coded as a dictionary, with labeled fields r

eplacing field positions:

```
>> D = {'name':'Pat','age': 40.5,'jobs': ['dev','mgr']} >  
> D['name'] 'Pat' >> D['age'] # Dictionary-based "record"  
40.5 >> D['jobs'][1] # Names mean more than numbers  
'mgr'
```

For variety, here is the same record recoded with dict and keywords, which may seem even more readable to some human readers:

```
>> D = dict(name='Pat', age=40.5, jobs=['dev','mgr'])  
>> D['name'] 'Pat' >> D['jobs'].remove('mgr') >> D {  
'name':'Pat','age': 40.5,'jobs': ['dev']}
```

In practice, dictionaries tend to be best for data with labeled components, as well as structures that can benefit from quick, direct lookups by name, instead of slower linear (left-to-right) searches. As we've seen, they also may be better for sparse collections and collections that grow flexibly.

Python programmers also have access to the sets we studied in Chapter 5, which are much like the keys of a valueless dictionary; they don't map keys to values, but can often be used like dictionaries for fast lookups when there is no associated value, especially in search routines:

```
>> D = {} >> D['state1'] = True # A visited-state dictionary  
>> 'state1' in D True >> S = set() >> S.add('state1')  
# Same, but with sets >> 'state1' in S True
```

Watch for a rehash of this record representation thread in the next chapter, where you'll see how tuples and named tuples compare to dictionaries in this role, as well as in Chapter 27, where you'll learn how user-defined classes factor into this picture, combining both data and logic to process it.

中文翻译

面对 Python 核心类型军火库里的这么多对象，一些读者可能会在列表和字典之间左右为难。简而言之：虽然两者都是其他对象的灵活集合，但列表把项分配给位置，字典则把它们分配给更有助记忆的键。正因为如此，字典数据对人类读者往往携带更多含义。例如，嵌套列表结构总是可以用来记录信息：

```
>>> L = ['Pat', 40.5, ['dev','mgr']] # 列表式"记录"
>>> L[0]'Pat'>>> L[1] # 字段用位置/数字表示40.5
>>> L[2][1]'mgr'
```

对某些类型的数据，列表的按位置访问是有意义的——比如公司员工列表、目录中的文件、数值矩阵。但像这样更有符号意义的记录，用带标签字段替换字段位置的字典来编码可能更有意义：

```
>>> D = {'name':'Pat','age': 40.5,'jobs': ['dev','mgr']}
>>> D['name']'Pat'
>>> D['age'] # 字典式"记录" 40.5
>>> D['jobs'][1] # 名字比数字更有意义'mgr'换个花样，下面是同一个记录用dict和关键字重新编码的版本，对某些读者来说可能可读性更强：
```

```
>>> D = dict(name='Pat', age=40.5, jobs=['dev','mgr'])
>>> D['name']'Pat'>>> D['jobs'].remove('mgr')
>>> D {'name':'Pat','age': 40.5,'jobs': ['dev']}
```

在实践中，字典最适合“带标签组件”的数据，以及能受益于“按名字快速直接查找”而非“较慢的线性（从左到右）搜索”的

结构。正如我们所见，它们对稀疏集合和需要灵活增长的集合也可能更好。Python 程序员还有我们在第 5 章学过的集合 (set) 可用，它非常像一个“无值字典”的键：它们不把键映射到值，但在没有关联值时，可以像字典一样用于快速查找，尤其是在搜索例程中：

```
>>> D = {} >>> D['state1'] = True # 已访问状态的字典 >>> 'state1' in D True >>> S = set() >>> S.add('state1') # 一样的效果，但用集合 >>> 'state1' in S True
```

请留意下一章对这条“记录表示”主线的重新讨论——你会看到元组和具名元组 (named tuples) 在这个角色上与字典的对比；以及第 27 章，你会学到用户自定义的类如何把数据和逻辑结合起来、进入这幅图景。

深度理解

·核心概念：选型三原则：带标签的数据 → 字典；无标签的序列 → 列表；只关心“有没有” → 集合。字典的查找是 $O(1)$ 哈希，列表的“查找”是 $O(n)$ 线性扫描。

·底层实现：“状态集合”例子里，字典方案是“键→True”的映射，集合方案干脆只存键——两者在哈希表内部几乎一样（set 的条目布局还更省内存）。`'state1' in D` 与 `'state1' in S` 都是 $O(1)$ 哈希查找。

·设计原因：三个容器对应人类三种信息组织习惯：“按顺序排列”（列表）、“按名字归档”（字典）、“按是否出现过标记”（集合）。Python 让三种思维各有一个类型，互不越界。

·实际问题：真实项目里“记录”的演进路线往往是：`dict` → `namedtuple` → `dataclass/class`——随着“需要逻辑”的

程度提高逐级升级（本书第 9、27 章覆盖）。去重、成员判断优先考虑集合；顺序处理用列表；字段访问用字典。

·初学者误区：① 用列表去重 ($O(n^2)$)，应该用集合；② 用列表模拟“按名字取”（线性扫描），应该用字典；③ 在“只需要成员判断”的场景却保存了一堆无用的值（应该用集合）。

8.10 脚注 (Footnotes)

英文原文

[1] In practice, you won't see many lists written out like this in list-processing programs. It's more common to see code that processes lists constructed dynamically (at runtime), from user inputs, file contents, and so on. In fact, although it's important to master literal syntax, many data structures in Python are built by running program code at runtime. [2] This description requires elaboration when the value and the slice being assigned overlap: `L[2:5]=L[3:6]`, for instance, works fine because the value to be inserted is fetched before the deletion happens on the left. Hence, slice assignment is really a fetch + delete + insert, but the fetch part matters too rarely to make the marquee. [3] Unlike + concatenation, append doesn't have to generate new objects, so it's usually faster than + too. You can also mimic append with the clever slice assignments of the prior section: `L[len(L):]=[X]` is like `L.append(X)`.

X), and `L[:0]=[X]` is like appending at the front of a list. Both delete an empty slice and insert X, changing L in place quickly, like `append`. Both are arguably more complex than list methods, though. For instance, `L.insert(0, X)` can also append an item to the front of a list, and seems noticeably more mnemonic. `L.insert(len(L), X)` inserts one object at the end too, but unless you like typing, you might as well use `L.append(X)`! [4]

As for lists, you might not see dictionaries coded in full using literals very often—programs rarely know all their data before they are run, and more typically extract it dynamically from users, files, and so on. Lists and dictionaries are grown in different ways, though. In the next section you'll see that you often build up dictionaries by assigning to new keys at runtime; this approach fails for lists, which are commonly grown with `append` or `extend` instead.

中文翻译

[1] 在实践中，列表处理程序里很少见到这样把列表完整写出来的写法。更常见的是处理运行时（动态）构建的列表的代码——来自用户输入、文件内容等。事实上，虽然掌握字面量语法很重要，但 Python 中的许多数据结构是靠运行时执行程序代码构建出来的。[2] 当被赋的值与目标切片重叠时，这个描述需要补充：例如 `L[2:5]=L[3:6]` 也能正常工作，因为要插入的值是在左侧删除发生之前取出的。因此，切片赋值实际上是“取出 + 删除 + 插入”三步，只是“取出”这一步太少见，不值得写进大字标题。[3] 与 + 拼接不同，

append 不必生成新对象，所以它通常也比 + 快。你也可以用上一节巧妙的切片赋值模仿 append: `L[len(L):]=[X]` 就像 `L.append(X)`，而 `L[:0]=[X]` 就像在列表前部追加。两者都是删除一个空切片并插入 X，像 append 一样快速原地修改 L。不过两者都比列表方法更复杂。例如，`L.insert(0, X)` 也能在列表前部加一项，而且显然更好记。`L.insert(len(L), X)` 也能在末尾插入一个对象，但除非你喜欢多打字，不然还是直接用 `L.append(X)` 吧！[4] 与列表一样，你可能不常看到用字面量完整编码字典——程序很少在运行前就知道自己的全部数据，更典型的是从用户、文件等处动态提取。不过列表和字典的增长方式不同。在下一节你会看到，你常常在运行时通过给新键赋值来构建字典；这种方法对列表行不通，列表通常改用 append 或 extend 增长。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 列表：日志行列表、CSV 行集、队列/栈、排序输出的中间结果、分页数据切片 (`page = rows[i:i+n]`)、解析 AST 的节点栈。
- 字典：JSON 配置、HTTP 请求头/参数、数据库行 (`dict(zip(cursor.description, row))`)、统计计数 (`count[w] = count.get(w, 0) + 1`)、缓存 (LRU 用 `OrderedDict` 或 3.7+ 的 `dict`)、API 响应映射。

·组合拳: json.load() 返回嵌套 dict/list, 配合"串式索引 + 推导式"直接加工; shelve/pickle 把"字典数据库"持久化到磁盘。

与其他语言 (Java/C++) 的区别

维度	Python	Java	C/C++
动态数组	内置 list (自动扩容、异构、可嵌套)	ArrayList<T> (泛型、必须装箱/拆箱)	std::vector (手动内存管理、类型固定)
哈希表	内置 dict (插入有序 3.7+, 字面量语法)	HashMap<K,V> (无序, 需 import)	std::unordered_map (需 include, 哈希函数自定义)
切片	L[i:j] 一等语法, 返回新列表	subList 视图 (Java 8+ List.copyOf 曲折)	手写 memcpy 循环
扩容策略	约 1.125 倍, 均摊 O(1)	约 1.5~2 倍, 均摊 O(1)	各实现不同 (gcc 2 倍/LLVM 1.5 倍)
不可变哈希键	键必须可哈希 (不可变)	键须实现 hashCode>equals 且不可变	需自定义哈希函数, 易错
成员判断	x in list (O(n)) / x in dict (O(1))	contains (O(n)) / HashMap.containsKey	手写循环 / find

Python 发展历史背景

- 1.x~3.5: dict 是经典哈希表 (槽位即条目), 迭代顺序伪随机——依赖顺序的代码须自排序; keys()/values()/items() 返回真列表 (O(n) 内存快照)。
- 3.6 (CPython): Raymond Hettinger 提出 compact dict 提案 (PEP 520 相关讨论), 把"索引表"与"条目表"分离: 内存节省约 20%, 且迭代顺序 = 插入顺序成为 CP

python 的实现细节。

- 3.7: dict 的插入顺序从"CPython 实现细节"升格为语言规范 (在 3.7 发布周期中被正式接受, 任何 Python 实现都必须遵守)。
- 3.5: / 解包进字面量 (PEP 448) ; 3.9: dict 加入 `|/|` = 合并运算符 (PEP 584) 。三条"合并途径" (`update`、`|`、`|`) 并存正是本版作者调侃的"历史冗余"。

高级开发者需要掌握的相关知识

- 底层: 读懂 CPython 源码 `Objects/listobject.c` (`listresize` 扩容公式: `newsize + (newsize >> 3) + 3`) 与 `Objects/dictobject.c` (`dictlookup` 开放寻址、`indices/entries` 双表) ; 了解哈希随机化 (`PYTHONHASHSEED`) 与 hash collision 攻击防护。
- 性能: 列表中段插入/删除 $O(n)$ 的替代品 (`collections.deque`、`bisect` 维护有序插入) ; dict 的 slots 类 vs dict 内存对比; `sys.getsizeof` 实测两种容器内存。
- 协议: `iter/getitem` 与 `for` 循环的关系 (第 14 章展开) ; dict 与 `collections.abc.Mapping/Sequence` 抽象基类的注册关系。
- 进阶类型: `collections.defaultdict`、`OrderedDict` (3.7 后主要保留 `movetoend`)、`Counter`、`namedtuple` (第 9 章)、`dataclasses` (类与记录的结合)。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5) 。列表与字典是 Python 使用频率最高、面试与项目出现率 100% 的两种类型；本章是全书后续（推导式、函数、类、文件、Web 开发）的地基。

·应该掌握到什么程度：

1. 必须脱口而出：五类列表操作（增长/删除/重排/查询/组合）、五种字典构造法、get/setdefault/pop/update/keys/values/items、切片赋值的"删除+插入"语义、sort vs sorted、视图的三性质（惰性/实时/集合式）。

2. 必须形成肌肉记忆：for k, v in D.items()、L.append vs L.extend、x in D 判键、sorted(D)。

3. 必须能口头讲清：为什么 dict 查找 $O(1)$ 、为什么键必须不可变（哈希不变性）、为什么列表尾部操作 $O(1)$ 而头部 $O(n)$ 、3.7 插入有序从何而来。

4. 建议动手验证：在 REPL 中跑一遍表 8-1 与表 8-2 的每个操作；用 sys.getsizeof 对比 [0]100 与 dict.fromkeys(range(100), 0) 的内存；写一个稀疏矩阵类练手。

·后续应该学习哪些相关内容：

·第 9 章：元组 (tuple) 与文件——元组是"不可变列表"、可作字典键；文件是"文本序列"。

·第 11 章：+= 对列表的原地语义、扩展解包赋值 (a, b = ...) 。

- 第 13~14 章: for/while 语句、迭代协议、推导式与 map/filter 的完整展开。
 - 第 20 章: 列表推导式与矩阵编程的深度应用。
 - 第 27 章: 类如何结合数据与逻辑, 成为"升级版记录"。
-

(本章完)